DEVOPS ENGINEER COURSE

Docker for DevOps Engineers

Containers from zero to production

Beginner | Intermediate | Advanced

Concepts · hands-on labs · troubleshooting · interview Q&A · cheat sheet

Contents

Part 1 — Foundations 5

1.1 Why Containers? 5

1.2 Virtual Machines vs Containers 6

1.3 Docker Architecture 7

1.4 Installing Docker 8

1.5 Your First Containers 11

1.6 Lab: Install, Inspect, and Explore the Engine 12

Key takeaways 13

Part 2 — Images, Containers and the CLI 14

2.1 Images in Depth 14

2.2 Containers and Their Lifecycle 16

2.3 The docker run Flags You Must Know 17

2.4 CLI Deep Dive 19

2.5 Environment Variables and Configuration 21

2.6 Lab: Lifecycle and CLI Fluency 22

Key takeaways 23

Part 3 — Writing Dockerfiles 24

3.1 The Build Context and .dockerignore 24

3.2 Every Dockerfile Instruction 25

3.3 Layers and the Build Cache 29

3.4 Multi-Stage Builds 31

3.5 Dockerfile Best-Practice Checklist 33

3.6 Lab: Build, Measure and Optimize a Python API 33

Key takeaways 35

Part 4 — BuildKit, Image Optimization and Registries 36

4.1 BuildKit 36

4.2 Image Optimization 40

4.3 Tagging Strategy 42

4.4 Registries 43

4.5 Lab: Multi-Platform Build to a Local Registry With Remote Cache 45

Key takeaways 47

Part 5 — Storage and Networking 48

5.1 Storage 48

5.2 Networking Fundamentals 51

5.3 Networking Troubleshooting Toolkit 55

5.4 Lab: Volumes, Isolated Networks and Firewalling 56

Key takeaways 57

Part 6 — Docker Compose in Depth 58

6.1 Core Concepts 58

6.2 A Complete Example 58

6.3 Service Configuration Reference 61
6.4 Profiles 65
6.5 Multiple Files: Overrides, extends, include 65
6.6 Day-to-Day Commands 66
6.7 Compose in CI and Production 67
6.8 Lab: Build a Full Dev Environment With Compose 68
Key takeaways 69

Part 7 — Runtime Internals: cgroups, Namespaces, Logging, Monitoring and Filesystems 70

7.1 Resource Limits and cgroups 70
7.2 Namespaces 72
7.3 Logging 73
7.4 Monitoring 74
7.5 Storage Drivers and overlayfs 76
7.6 Building a Container by Hand 77
7.7 Lab: Limits, OOM, Throttling and Logs 79
Key takeaways 79

Part 8 — Container Security 81

8.1 Threat Model in One Table 81
8.2 Securing the Host and Daemon 81
8.3 Securing Images 82
8.4 Securing the Runtime 85
8.5 Secrets at Runtime 88
8.6 Security Checklist 88
8.7 Lab: Harden and Verify an Image 89
Key takeaways 90

Part 9 — Docker in CI/CD, Swarm, the Path to Kubernetes, and Production Practices 91

9.1 Principles of Container CI/CD 91
9.2 GitHub Actions 91
9.3 GitLab CI 93
9.4 Jenkins 95
9.5 CI Caching Strategies Compared 96
9.6 Docker Swarm Basics 96
9.7 The Path to Kubernetes 98
9.8 Production Best Practices 100
9.9 Lab: A GitHub Actions Pipeline With Cache, Scan and Sign 101
Key takeaways 101

Part 10 — Troubleshooting Playbook 103

10.1 The General Method 103
10.2 Container Exits Immediately 103
10.3 Restart Loops 104
10.4 OOMKilled 105
10.5 Networking Problems 105
10.6 Disk Space 106
10.7 Build Failures 107

10.8 Permission Problems 108
10.9 Performance Problems 108
10.10 Daemon-Level Problems 109
10.11 Quick Reference: Symptom to First Command 109
10.12 Lab: Break It and Fix It 110
Key takeaways 111

Part 11 — Capstone Project, Interview Preparation and Cheat Sheet 112
11.1 Capstone: “Linkly”, a URL Shortener Platform 112
11.2 Interview Questions and Answers 121
11.3 Command Cheat Sheet 125
11.4 Where to Go Next 128
Key takeaways 128

Part 1 — Foundations

This course takes you from "I have run `docker run hello-world` once" to the level expected of a senior DevOps or platform engineer: someone who can build small, secure images, debug a broken container at 3 a.m., wire Docker into CI/CD, and explain exactly what happens in the kernel when a container starts.

Part 1 lays the groundwork. You will learn why containers exist, how they differ from virtual machines, what the moving parts of the Docker platform are (dockerd, containerd, runc, the OCI specifications), and how to install Docker properly on Linux and macOS, including rootless mode.

1.1 Why Containers?

1.1.1 The problem containers solve

Before containers, shipping software meant shipping *instructions*: "install Python 3.11, then these 40 packages, set these environment variables, make sure libssl is version X". Every environment (laptop, CI runner, staging VM, production fleet) was a slightly different snowflake. The classic symptom was "works on my machine".

Containers change what you ship. Instead of instructions, you ship the **result**: a filesystem snapshot containing the application and all of its user-space dependencies, plus metadata describing how to run it. That artifact is an **image**. A running instance of it is a **container**.

The key benefits for a DevOps engineer:

Benefit	What it means in practice
Reproducibility	The same image digest runs identically in CI, staging and production.
Isolation	Processes cannot see each other's files, processes or network by default.
Density	Containers share the host kernel, so you can run dozens or hundreds per host.
Speed	A container starts in milliseconds; there is no OS boot.
Immutable deployments	You replace containers rather than patching them, which makes rollbacks trivial.
Standard interface	Every app is started, stopped, logged and monitored the same way.

1.1.2 What a container actually is

A container is **not** a lightweight VM. It is an ordinary Linux process (or group of processes) that the kernel has been told to isolate and constrain using three main features:

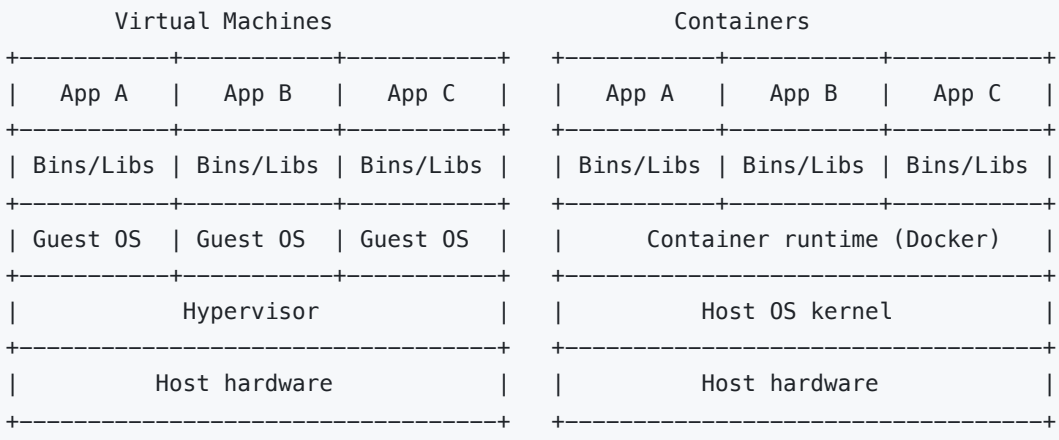
- **Namespaces** control what a process can *see*: its own process tree (PID), network stack (NET), mount table (MNT), hostname (UTS), IPC objects, user IDs (USER) and cgroup view (CGROUP).
- **Control groups (cgroups)** control what a process can *use*: CPU, memory, I/O, number of PIDs.

- **A layered root filesystem** (usually overlayfs) gives the process its own `/` assembled from read-only image layers plus a thin writable layer.

Add security filters (capabilities, seccomp, AppArmor/SELinux) and you have a container. We will build one by hand in Part 7.

> **Note:** Because containers share the host kernel, a Linux container needs a Linux kernel. On macOS and Windows, Docker Desktop runs a small Linux VM and your containers live inside it.

1.2 Virtual Machines vs Containers



Aspect	Virtual machine	Container
Isolation boundary	Hardware virtualization (hypervisor)	Kernel features (namespaces, cgroups)
Kernel	Each VM has its own	Shared with host
Boot time	Tens of seconds to minutes	Milliseconds
Size	GBs (full OS)	MBs (just app + libs)
Density	Tens per host	Hundreds per host
Security isolation	Strong	Good, but weaker; a kernel exploit affects all containers
Run a different OS kernel	Yes (Windows on Linux host)	No (Linux containers need a Linux kernel)

The two are complementary, not rivals. In the cloud, your containers almost always run *inside* VMs (EC2 instances, GKE nodes). Sandboxed runtimes such as **gVisor** and **Kata Containers** blur the line further by giving each container a user-space kernel or a micro-VM for stronger isolation.

> **Interview:** “Is a container a VM?” No. A container is a process isolated with namespaces and limited with cgroups, sharing the host kernel. A VM virtualizes hardware and runs its own kernel. Mention the security implication: kernel vulnerabilities cross container boundaries.

1.3 Docker Architecture

When you type `docker run nginx`, several components cooperate. Understanding them is essential for troubleshooting and for understanding Kubernetes, which reuses most of the stack.

```
docker CLI  ---REST over /var/run/docker.sock--->  dockerd (Docker Engine)
                                                       |
                                                       | gRPC
                                                       v
                                                  containerd
                                                       |
                                          containerd-shim-runc-v2 (one per container)
                                                       |
                                                       v
                                                  runc  --> creates namespaces,
                                                            cgroups, mounts,
                                                            then exec()s your process
                                                            and exits
```

1.3.1 The components

- **Docker CLI (`docker`)**: a client. It translates commands into REST API calls. It can talk to a local daemon over a Unix socket or a remote one via `DOCKER_HOST` or contexts.
- **dockerd (Docker Engine)**: the daemon. It owns the high-level concepts: images, builds (via BuildKit), volumes, networks, the API, logging drivers, Swarm.
- **containerd**: a CNCF-graduated container supervisor. It manages the container lifecycle, pulls and stores images, manages snapshots (filesystem layers). Kubernetes can talk to containerd directly through its CRI plugin, without dockerd.
- **containerd-shim**: a tiny process that sits between containerd and each container. It keeps the container’s stdio open and reports its exit status, which is why you can restart dockerd or containerd without killing running containers (with `live-restore` enabled).
- **runc**: the reference low-level OCI runtime. It reads an OCI bundle (a root filesystem plus `config.json`), sets up namespaces, cgroups, capabilities and seccomp, then `exec`s the container’s entrypoint and exits.
- **BuildKit**: the build engine, default since Docker Engine 23. It parallelizes builds, supports cache mounts, secrets, and multi-platform output.

1.3.2 The OCI specifications

The **Open Container Initiative** (Linux Foundation) standardizes containers so that tools are interchangeable:

Spec	Defines	Why you care
Image spec	Image manifest, config, layers, index (multi-arch)	Images built by Docker run in Podman, containerd, Kubernetes.
Runtime spec	`config.json` + rootfs bundle, lifecycle	runc, crun, gVisor (runsc), Kata are swappable.
Distribution spec	HTTP API for pushing and pulling	Any OCI registry works: Docker Hub, ECR, GHCR, Harbor.

So “Docker image” really means “OCI image”. Kubernetes removed its built-in Docker shim (dockershim) in v1.24, but Docker-built images still run fine there, because they are OCI images.

1.3.3 The Docker socket is root

The daemon traditionally runs as root, and anyone who can write to `/var/run/docker.sock` can start a privileged container that mounts the host’s `/`. **Access to the Docker socket equals root on the host.** Keep that in mind whenever someone suggests adding a user to the `docker` group or mounting the socket into a CI container.

```
# Why the docker group is root-equivalent:
docker run --rm -it -v /:/host alpine chroot /host sh
# You now have a root shell on the host filesystem.
```

1.3.4 Docker Desktop, Engine, and alternatives

- **Docker Engine**: the open-source daemon on Linux. Free.
- **Docker Desktop**: a GUI app for macOS, Windows and Linux that bundles a Linux VM, Engine, Compose, BuildKit, Scout, Kubernetes and extensions. Commercial use in larger organizations (more than 250 employees or more than USD 10M revenue) requires a paid subscription.
- **Alternatives**: Podman (daemonless, rootless-first, mostly CLI-compatible), Colima and OrbStack (lightweight VMs on macOS that expose a Docker-compatible socket), Rancher Desktop, nerdctl (Docker-like CLI for containerd).

1.4 Installing Docker

1.4.1 Linux (Ubuntu/Debian) from the official repository

Distribution packages (`docker.io`) are often outdated. Use Docker’s own apt repository.

```
# Remove conflicting packages
for pkg in docker.io docker-doc docker-compose podman-docker containerd runc; do
  sudo apt-get remove -y "$pkg"
done

# Add Docker's GPG key and repository
```

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] \
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
```

The packages:

- `docker-ce`: the Engine (dockerd)
- `docker-ce-cli`: the `docker` client
- `containerd.io`: containerd and runc
- `docker-buildx-plugin`: `docker buildx`
- `docker-compose-plugin`: `docker compose` (v2, written in Go, a CLI plugin)

For RHEL, Fedora, CentOS Stream or Amazon Linux, the procedure is the same idea using `dnf config-manager --add-repo https://download.docker.com/linux/<distro>/docker-ce.repo`. On Amazon Linux 2023, `sudo dnf install docker` gives you a supported Engine from Amazon's repository.

Enable and verify:

```
sudo systemctl enable --now docker
sudo docker run --rm hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

Optionally allow your user to run `docker` without sudo (remember: root-equivalent):

```
sudo usermod -aG docker "$USER"
newgrp docker   # or log out and back in
```

1.4.2 macOS

Options:

1. **Docker Desktop**: download from docker.com, choose Apple Silicon or Intel. Easiest, includes everything.
2. **Colima** (free, CLI-only):

```
brew install colima docker docker-compose docker-buildx
colima start --cpu 4 --memory 8 --disk 60 --vm-type vz --vz-rosetta
docker context ls
```

3. **OrbStack**: a fast commercial alternative popular with Mac users.

> **Tip:** On Apple Silicon, images default to `linux/arm64`. If you need to test an `amd64`-only image, use `--platform linux/amd64`; Docker Desktop and Colima (with Rosetta) will emulate it.

1.4.3 Rootless mode

Rootless mode runs the daemon *and* containers as an unprivileged user using user namespaces. A container breakout lands the attacker as your normal user, not root.

```
# Prerequisites (Ubuntu)
sudo apt-get install -y uidmap dbus-user-session slirp4netns
# Ensure subordinate ID ranges exist
grep "^$USER:" /etc/subuid /etc/subgid

# Install as the normal user (not root)
dockerd-rootless-setuptool.sh install

systemctl --user enable --now docker
export DOCKER_HOST=unix:///run/user/$(id -u)/docker.sock
docker info --format '{{.SecurityOptions}}'
```

```
[name=seccomp,profile=builtin name=rootless name=cgroupns]
```

Limitations to know:

- Binding ports below 1024 needs `sysctl net.ipv4.ip_unprivileged_port_start=0` or `setcap cap_net_bind_service`.
- Networking goes through a user-space stack (slirp4netns, or the faster pasta / RootlessKit port driver), which is slower than the rootful bridge.
- Resource limits require cgroup v2 with systemd delegation.
- Some storage drivers and features (AppArmor, certain overlay setups) behave differently.

Rootless mode is different from **userns-remap**, where a rootful daemon maps container root to an unprivileged host UID range. Both reduce the blast radius of a breakout.

1.4.4 Post-install configuration: daemon.json

The daemon reads `/etc/docker/daemon.json` (rootless: `~/.config/docker/daemon.json`). A sensible production baseline:

```
{
  "log-driver": "local",
  "log-opts": { "max-size": "10m", "max-file": "3" },
  "live-restore": true,
  "default-address-pools": [
    { "base": "10.200.0.0/16", "size": 24 }
  ],
  "features": { "containerd-snapshotter": true },
  "userland-proxy": false,
  "no-new-privileges": true
}
```

Why each matters:

- `log-driver` / `log-opts`: the default `json-file` driver has **no rotation** and will fill your disk. `local` is compact and rotates.
- `live-restore`: containers survive a daemon restart or upgrade.
- `default-address-pools`: avoids clashes between Docker's default `172.17.0.0/16`-style subnets and your corporate network or VPN.
- `containerd-snapshotter`: uses containerd's image store, which enables multi-platform images locally and is the default for fresh installs in recent Docker Engine releases.
- `no-new-privileges`: blocks setuid escalation in all containers by default.

```
sudo systemctl restart docker
docker info | grep -iE 'logging driver|live restore|storage driver|cgroup'
```

```
 Storage Driver: overlayfs
 Logging Driver: local
 Cgroup Driver: systemd
 Cgroup Version: 2
 Live Restore Enabled: true
```

1.4.5 Docker contexts

A context bundles an endpoint (local socket, SSH host, remote TLS) so you can switch between daemons:

```
docker context create prod-host --docker "host=ssh://deploy@10.0.0.15"
docker context use prod-host
docker ps              # runs against the remote daemon over SSH
docker context use default
```

1.5 Your First Containers

```
docker run --rm hello-world
docker run -it --rm alpine:3.20 sh
```

```
docker run -d --name web -p 8080:80 nginx:1.27-alpine
curl -s localhost:8080 | head -n 4
docker logs web
docker stop web && docker rm web
```

What happened during `docker run -d --name web -p 8080:80 nginx:1.27-alpine`:

1. The CLI POSTed to `/containers/create` on the daemon.
2. The daemon checked the local image store; on a miss it pulled `docker.io/library/nginx:1.27-alpine` (resolving the tag to a digest, choosing the manifest for your CPU architecture, downloading layers).
3. It created a writable layer on top of the image layers, a network endpoint on the default bridge, and a port-forwarding rule for 8080 to 80.
4. containerd started a shim, which invoked runc to create namespaces and cgroups and exec `nginx -g 'daemon off;'`.
5. The CLI printed the container ID and returned (because of `-d`).

1.6 Lab: Install, Inspect, and Explore the Engine

Goal: install Docker, confirm every component, and prove that a container is just a process.

1. Install Docker using section 1.4 for your OS. Then:

```
docker version
docker info
docker buildx version
docker compose version
```

Note the separate Client and Server sections in `docker version`. If the Server section errors, the daemon is not running or you lack socket permissions.

2. Start a long-ish running container:

```
docker run -d --name sleeper alpine:3.20 sleep 600
```

3. On a Linux host, find it as a normal process:

```
ps -ef | grep "sleep 600" | grep -v grep
pstree -sp "$(pgrep -f 'sleep 600')"
```

```
root  48211  48190  0 10:02 ?  00:00:00 sleep 600
systemd(1)---containerd-shim(48190)---sleep(48211)
```

The parent is the containerd shim, not dockerd. (On macOS, run these inside the VM, e.g. `colima ssh`.)

4. Compare views from inside and outside:

```
docker exec sleeper ps
docker exec sleeper hostname
readlink /proc/$(pgrep -f 'sleep 600')/ns/pid
readlink /proc/self/ns/pid
```

Inside, `sleep` is PID 1; outside it has a large PID. The namespace inode numbers differ.

5. Configure `daemon.json` with log rotation and `live-restore`, restart the daemon, and confirm with `docker info`. Verify `sleeper` is still running after the restart (thanks to live-restore).
6. Clean up:

```
docker rm -f sleeper
```

Key takeaways

- A container is an isolated, constrained Linux process: namespaces for visibility, cgroups for resources, a layered filesystem for its root.
- Containers share the host kernel; VMs do not. That gives speed and density at the cost of a weaker isolation boundary.
- The stack is CLI -> dockerd -> containerd -> shim -> runc. Kubernetes reuses containerd and runc without dockerd.
- OCI specs make images and runtimes interoperable across Docker, Podman, containerd and Kubernetes.
- Access to the Docker socket is equivalent to root on the host. Prefer rootless mode where possible.
- Configure log rotation and address pools in `daemon.json` on day one.

Part 2 — Images, Containers and the CLI

With Docker installed, you now need a precise mental model of the two core objects, **images** and **containers**, and fluency with the CLI that manipulates them. Most day-to-day DevOps work with Docker is CLI work: inspecting, filtering, exec-ing, copying, and cleaning up.

2.1 Images in Depth

2.1.1 What an image is

An image is an immutable, content-addressed bundle made of:

- **Layers**: tar archives of filesystem changes, each identified by a SHA-256 digest.
- **A config object**: JSON describing the default command, entrypoint, environment, user, working directory, exposed ports, labels, and the ordered list of layer diff IDs.
- **A manifest**: JSON pointing to the config and the layers for one platform.
- **An index** (optional, "manifest list"): JSON pointing to several manifests, one per platform (e.g. `linux/amd64`, `linux/arm64`).

```
nginx:1.27-alpine  (tag, mutable pointer)
       |
       v
Image index  sha256:ab12...   (multi-platform)
  |-- manifest linux/amd64  sha256:c3d4...
  |      |-- config  sha256:e5f6...
  |      |-- layer   sha256:1111...  (alpine base)
  |      |-- layer   sha256:2222...  (nginx binaries)
  |      `-- layer   sha256:3333...  (config files)
  `-- manifest linux/arm64  sha256:7788...
```

2.1.2 Image references: names, tags, digests

A full reference looks like:

```
registry.example.com:5000/team/app:1.4.2@sha256:9f86d0...
|------ registry ------| |-repo-| |tag| |---- digest ----|
```

- If you omit the registry, Docker assumes `docker.io`.
- If you omit the namespace on Docker Hub, it assumes `library/` (official images).
- If you omit the tag, it assumes `latest`. `latest` is just a tag name with no special meaning; it is not "the newest".
- A **tag** is mutable: someone can push a new image to `app:1.4.2`.
- A **digest** is immutable: `app@sha256:...` always refers to exactly the same bytes.

> **Warning:** For production deployments, pin by digest (or use immutable tags enforced by your registry). A tag can be moved under you, silently changing what runs.

2.1.3 Working with images

```
docker pull nginx:1.27-alpine
docker pull --platform linux/arm64 nginx:1.27-alpine
docker image ls
docker image ls --digests
docker image ls --filter dangling=true
docker image inspect nginx:1.27-alpine --format '{{.Config.Cmd}} {{.Architecture}}'
docker image history nginx:1.27-alpine
docker tag nginx:1.27-alpine myregistry.local/web:v1
docker image rm myregistry.local/web:v1
docker image prune          # remove dangling images
docker image prune -a       # remove all images not used by a container
```

```
$ docker image history nginx:1.27-alpine
IMAGE          CREATED       CREATED BY                                      SIZE
4ff1...        3 weeks ago   CMD ["nginx" "-g" "daemon off;"]                0B
<missing>      3 weeks ago   STOPSIGNAL SIGQUIT                              0B
<missing>      3 weeks ago   EXPOSE map[80/tcp:{}]                           0B
<missing>      3 weeks ago   ENTRYPOINT ["/docker-entrypoint.sh"]            0B
<missing>      3 weeks ago   RUN /bin/sh -c set -x && addgroup -g 101 ...    24.1MB
<missing>      3 weeks ago   ADD alpine-minirootfs-3.20.3-x86_64.tar.gz /    7.8MB
```

`<missing>` simply means those intermediate layers were built elsewhere and have no local image ID; it is normal.

Inspect a remote image without pulling it:

```
docker buildx imagetools inspect nginx:1.27-alpine
```

```
Name:      docker.io/library/nginx:1.27-alpine
MediaType: application/vnd.oci.image.index.v1+json
Digest:    sha256:ab12...
Manifests:
  Name:      docker.io/library/nginx:1.27-alpine@sha256:c3d4...
  Platform:  linux/amd64
  Name:      docker.io/library/nginx:1.27-alpine@sha256:7788...
  Platform:  linux/arm64/v8
  ...
```

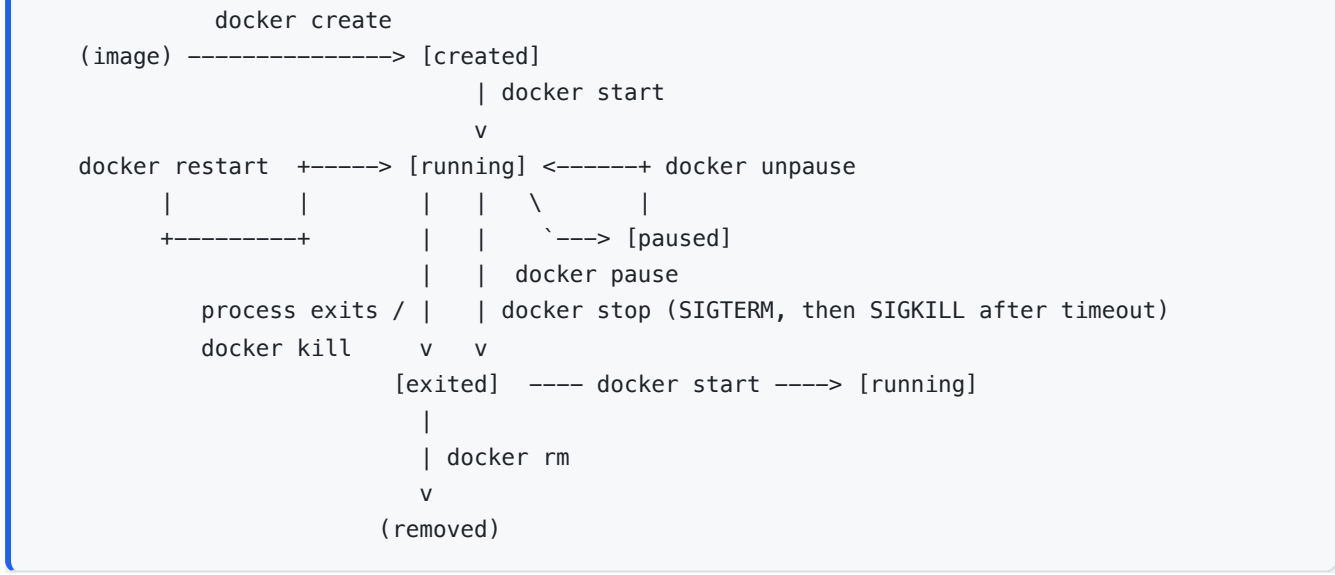
2.1.4 Saving, loading, exporting

```
docker image save -o nginx.tar nginx:1.27-alpine   # image with layers + metadata
docker image load -i nginx.tar
docker container export web -o web-fs.tar          # flattened filesystem of a container, no metadata
docker image import web-fs.tar myflat:latest        # new single-layer image
```

`save` / `load` is how you move images into air-gapped environments. `export` / `import` loses history, CMD, ENV and so on.

2.2 Containers and Their Lifecycle

2.2.1 States



Additional states: `restarting` (restart policy is acting), `removing`, and `dead` (removal failed, usually storage driver trouble).

2.2.2 PID 1 and signals

The container's main process runs as PID 1 in its PID namespace. Two consequences:

1. **Signal handling**: the kernel does not apply default signal actions to PID 1. If your app does not install a SIGTERM handler, `docker stop` will wait the full timeout (10 seconds by default) and then SIGKILL it. That is slow and skips graceful shutdown.
2. **Zombie reaping**: PID 1 must `wait()` on orphaned children. Many apps do not, leaving zombies.

Fixes:

```
docker run --init myapp        # injects tini as PID 1, which forwards signals and reaps zombies
```

```
docker stop -t 30 myapp        # give a longer grace period
docker run --stop-signal SIGQUIT nginx   # or STOPSIGNAL in the Dockerfile
```

Also use the **exec form** of `CMD` / `ENTRYPOINT` (JSON array) so your app is PID 1, not `/bin/sh -c` which does not forward signals. More in Part 3.

2.2.3 Exit codes worth memorizing

Exit code	Meaning
0	Clean exit
1	Generic application error
125	The `docker run` command itself failed (bad flag, daemon error)
126	Command found but not executable (permissions, wrong arch)
127	Command not found (typo, missing binary, wrong PATH)
128 + n	Killed by signal n
137	128 + 9 = SIGKILL, often OOM-killed or `docker kill` /stop timeout
139	128 + 11 = SIGSEGV, segmentation fault
143	128 + 15 = SIGTERM, graceful stop

2.2.4 Restart policies

Policy	Behavior
`no` (default)	Never restart.
`on-failure[:N]`	Restart on non-zero exit, optionally up to N times.
`always`	Always restart; also starts on daemon boot, even if manually stopped before reboot.
`unless-stopped`	Like `always`, but stays stopped across daemon restarts if you stopped it manually.

```
docker run -d --restart unless-stopped --name api myorg/api:1.2.0
docker update --restart on-failure:5 api
```

Docker applies exponential backoff between restarts (starting at 100 ms, doubling, capped at 1 minute) to avoid a tight crash loop.

2.3 The `docker run` Flags You Must Know

`docker run` = `create` + `start` (+ `attach` unless `-d`). Its flags are the vocabulary of container configuration; Compose and Kubernetes have equivalents for nearly all of them.

Flag	Purpose	Example
`-d`	Detached (background)	`-d`
`-it`	Interactive + TTY	`-it alpine sh`
`--rm`	Remove on exit	`--rm`
`--name`	Fixed name	`--name api`
`-p host:ctr`	Publish port	`-p 127.0.0.1:8080:80`
`-P`	Publish all EXPOSEd ports to random host ports	`-P`
`-e` , `--env-file`	Environment variables	`-e LOG_LEVEL=debug`
`-v` , `--mount`	Volumes and bind mounts	`--mount type=volume,src=data,dst=/data`
`--network`	Attach to network	`--network backend`
`-w`	Working directory	`-w /app`
`-u`	User / UID:GID	`-u 1000:1000`
`--entrypoint`	Override ENTRYPOINT	`--entrypoint sh`
`--memory` , `--cpus`	Resource limits	`--memory 512m --cpus 1.5`
`--restart`	Restart policy	`--restart unless-stopped`
`--health-cmd`	Healthcheck	`--health-cmd "curl -f localhost/healthz"`
`--read-only`	Read-only root FS	`--read-only --tmpfs /tmp`
`--cap-drop/--cap-add`	Linux capabilities	`--cap-drop ALL --cap-add NET_BIND_SERVICE`
`--init`	tini as PID 1	`--init`
`--platform`	Choose image platform	`--platform linux/amd64`
`--pull`	`always` , `missing` , `never`	`--pull always`
`-l`	Labels	`-l team=payments`

Everything after the image name replaces `CMD` :

```
docker run --rm alpine:3.20 echo "hello"          # CMD replaced by: echo hello
docker run --rm --entrypoint ls alpine:3.20 -la / # ENTRYPOINT replaced by ls, args -la /
```

2.4 CLI Deep Dive

2.4.1 Management commands

Modern Docker groups commands by object: `docker container ...`, `docker image ...`, `docker volume ...`, `docker network ...`, `docker system ...`, `docker builder ...`. The short forms (`docker ps`, `docker rmi`) are aliases and remain valid.

Short	Long form
`docker ps`	`docker container ls`
`docker rm`	`docker container rm`
`docker rmi`	`docker image rm`
`docker images`	`docker image ls`
`docker exec`	`docker container exec`
`docker build`	`docker buildx build` (BuildKit)

2.4.2 Listing and filtering

```
docker ps                       # running
docker ps -a                    # all, including exited
docker ps -q                    # IDs only (for scripting)
docker ps --filter status=exited --filter label=team=payments
docker ps --filter ancestor=nginx:1.27-alpine
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
docker ps -s                    # include writable-layer size
```

```
NAMES     STATUS                   PORTS
api       Up 2 hours (healthy)     0.0.0.0:8080->8080/tcp
redis     Up 2 hours               6379/tcp
worker    Exited (137) 5 min ago
```

2.4.3 Inspect and Go templates

`docker inspect` returns the full JSON state. Go templates (`--format`) extract fields; pipe to `jq` for anything complex.

```
docker inspect api --format '{{.State.Status}} {{.State.ExitCode}} {{.State.OOMKilled}}'
docker inspect api --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
docker inspect api --format '{{json .Config.Env}}' | jq
docker inspect api --format '{{.HostConfig.RestartPolicy.Name}}'
docker inspect api --format '{{json .State.Health}}' | jq '.Log[-1]'
docker inspect api | jq '.[0].Mounts'
```

2.4.4 Logs

```
docker logs api
docker logs -f --tail 100 api        # follow last 100 lines
docker logs --since 15m api
docker logs -t api                   # timestamps
docker logs api 2>&1 | grep -i error # stderr is a separate stream
```

`docker logs` only works for logging drivers that support reading (`json-file`, `local`, `journald`, and others via dual logging). Apps should log to stdout/stderr, not to files inside the container.

2.4.5 Exec, attach, cp

```
docker exec -it api sh                     # shell into running container
docker exec -u 0 -it api sh                # as root, for debugging
docker exec api env                        # one-off command
docker exec -e DEBUG=1 -w /app api ./diag.sh
docker attach api                          # attach to PID 1 stdio (detach: Ctrl-P Ctrl-Q)
docker cp api:/app/config.yaml ./config.yaml
docker cp ./fix.conf api:/etc/nginx/conf.d/
```

> **Warning:** Ctrl-C while attached sends SIGINT to PID 1 and may stop the container. Use `docker logs -f` to watch output, and Ctrl-P Ctrl-Q to detach.

For minimal images with no shell (distroless, scratch), use `docker debug` (Docker Desktop / Pro feature) or attach a debug container sharing the target's namespaces:

```
docker run --rm -it --pid container:api --network container:api \
  nicolaka/netshoot
```

2.4.6 Stats, top, events, diff

```
docker stats                         # live CPU/mem/net/IO per container
docker stats --no-stream --format 'table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}'
docker top api                       # processes in the container
docker events --since 30m --filter type=container --filter event=die
docker diff api                      # A=added, C=changed, D=deleted in writable layer
docker port api
```

```
$ docker diff api
C /tmp
A /tmp/cache.db
C /etc/nginx/conf.d
```

`docker diff` is a quick way to discover that an app is writing to its container filesystem when it should be writing to a volume.

2.4.7 Updating, renaming, waiting

```
docker update --memory 1g --cpus 2 api   # change limits live
docker rename api api-old
docker wait worker                       # block until exit, print exit code
docker container prune                   # remove all stopped containers
```

2.4.8 Bulk operations and scripting

```
docker stop $(docker ps -q)                         # stop everything running
docker rm $(docker ps -aq --filter status=exited)   # remove exited
docker ps -aq --filter label=env=test | xargs -r docker rm -f
```

Use `xargs -r` so nothing runs when the list is empty.

2.4.9 docker system

```
docker system df          # disk usage summary
docker system df -v       # per image/container/volume
docker system info
docker system prune       # stopped containers, unused networks, dangling images, build cache
docker system prune -a --volumes   # aggressive: also unused images and volumes
```

```
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          23        6         8.42GB    6.10GB (72%)
Containers      9         4         120MB     88MB (73%)
Local Volumes   11        5         2.3GB     1.1GB (47%)
Build Cache     154       0         3.9GB     3.9GB
```

> **Warning:** `--volumes` deletes data in volumes not attached to a container, including stopped databases' data if their container was removed. Never run it blindly on a server.

2.5 Environment Variables and Configuration

Twelve-factor apps take configuration from the environment.

```
docker run -e DB_HOST=db -e DB_PORT=5432 myapp
docker run -e DB_PASSWORD myapp          # passes the value from the current shell
docker run --env-file ./app.env myapp
```

```
# app.env  (KEY=VALUE, no quotes processing, no export keyword)
DB_HOST=db
DB_PORT=5432
LOG_LEVEL=info
```

> **Warning:** Environment variables are visible in `docker inspect`, in `/proc/<pid>/environ`, and often in crash dumps. For real secrets prefer mounted files (Compose/Swarm secrets, Kubernetes Secrets as volumes, or a secrets manager). See Part 8.

2.6 Lab: Lifecycle and CLI Fluency

Goal: observe every lifecycle state, exit codes, signal handling, and practice inspection.

1. Create without starting, then start:

```
docker create --name lc alpine:3.20 sh -c 'echo started; sleep 300'
docker ps -a --filter name=lc --format '{{.Status}}'   # Created
docker start lc
docker pause lc && docker ps --filter name=lc --format '{{.Status}}'  # (Paused)
docker unpause lc
```

2. Observe the stop timeout problem. `sh` as PID 1 ignores SIGTERM:

```
time docker stop lc
docker inspect lc --format '{{.State.ExitCode}}'
```

Expect about 10 seconds and exit code 137 (SIGKILL after timeout).

3. Repeat with `--init`:

```
docker run -d --init --name lc2 alpine:3.20 sleep 300
time docker stop lc2
docker inspect lc2 --format '{{.State.ExitCode}}'
```

Expect a near-instant stop and exit code 143.

4. Produce exit codes 127 and 126:

```
docker run --rm alpine:3.20 nosuchcmd; echo "exit=$?"
docker run --rm alpine:3.20 /etc/passwd; echo "exit=$?"
```

5. Restart policy with backoff:

```
docker run -d --name crashy --restart on-failure:3 alpine:3.20 sh -c 'exit 1'
sleep 5; docker inspect crashy --format '{{.RestartCount}} {{.State.Status}}'
```

6. Inspection drill: start `nginx:1.27-alpine` with `-p 8080:80`, then use templates to print its IP, port bindings, and the image's default CMD. Use `docker diff` after `curl localhost:8080` to see what nginx writes.

7. Clean up:

```
docker rm -f lc lc2 crashy $(docker ps -aq --filter ancestor=nginx:1.27-alpine)
docker system df
```

Key takeaways

- Images are immutable stacks of content-addressed layers plus config; tags are mutable pointers, digests are immutable.
- `latest` means nothing special. Pin versions, and pin digests in production.
- PID 1 does not get default signal handling: use exec-form commands, handle SIGTERM, or use `--init`.
- Learn the exit codes: 125/126/127 for startup problems, 137 for SIGKILL/OOM, 143 for SIGTERM.
- `docker inspect` with `--format` or `jq` answers almost any "what is this container doing" question.
- `docker system df` and targeted `prune` commands keep hosts healthy; be careful with `--volumes`.

Part 3 — Writing Dockerfiles

A Dockerfile is a recipe that BuildKit executes to produce an image. Writing good Dockerfiles is the single highest-leverage Docker skill: it determines image size, build speed, security posture and runtime behavior. This Part covers every instruction, how layers and the build cache work, and multi-stage builds.

3.1 The Build Context and .dockerignore

When you run `docker build .`, the `.` is the **build context**: the set of files the builder may access via `COPY` and `ADD`. With BuildKit, only files actually referenced are transferred, but the context still defines what is reachable, and a large or sensitive context slows builds and risks leaking secrets.

Always add a `.dockerignore`:

```
# .dockerignore
.git
.github
node_modules
dist
build
*.log
.env
.env.*
**/*.pem
**/__pycache__
.venv
Dockerfile*
docker-compose*.yaml
compose*.yaml
```

> **Warning:** Without `.dockerignore`, `COPY . .` copies your `.env`, `.git` history (which may contain old secrets) and local `node_modules` (built for the wrong OS) into the image.

Basic build invocations:

```
docker build -t myapp:dev .
docker build -t myapp:dev -f docker/Dockerfile.prod .
docker build --build-arg APP_VERSION=1.4.2 -t myapp:1.4.2 .
docker build --target test .            # build a specific stage
docker build --no-cache .
docker build --progress=plain .         # full log output, useful in CI
```

3.2 Every Dockerfile Instruction

3.2.1 Parser directives and syntax

```
# syntax=docker/dockerfile:1
# check=error=true
```

The `syntax` directive must be the very first line. It tells BuildKit to fetch the latest stable Dockerfile frontend (version 1.x), giving you new features (heredocs, `COPY --exclude`, `--chmod`, build checks) without upgrading Docker. `check=error=true` turns build-check warnings (e.g. `JSONArgsRecommended`, `SecretsUsedInArgOrEnv`) into errors.

3.2.2 FROM

```
FROM python:3.12-slim-bookworm AS base
FROM --platform=$BUILDPLATFORM golang:1.23 AS build
FROM scratch
```

Starts a new build stage. `AS name` names the stage for later reference. Choose base images deliberately: official images, slim or alpine variants, distroless for runtime. `--platform=$BUILDPLATFORM` is for cross-compilation (Part 4).

3.2.3 ARG

```
ARG BASE_TAG=3.12-slim
FROM python:${BASE_TAG}
ARG APP_VERSION=dev
RUN echo "building ${APP_VERSION}"
```

Build-time variables, set with `--build-arg`. Scoping rules trip people up:

- An `ARG` before the first `FROM` is only usable in `FROM` lines. To use it inside a stage, redeclare `ARG BASE_TAG` (without a value) in that stage.
- An `ARG` declared in a stage is only visible within that stage.
- Values are recorded in image history. **Never pass secrets as build args**; use secret mounts (Part 4).

Predefined args: `HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY` (not persisted in history), plus BuildKit's `TARGETPLATFORM`, `TARGETOS`, `TARGETARCH`, `BUILDPLATFORM`, `BUILDOS`, `BUILDARCH`.

3.2.4 ENV

```
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    PATH="/app/.venv/bin:${PATH}"
```

Sets environment variables that persist in the image and in running containers. Use `ARG` for build-only values and `ENV` for values the runtime needs. A common pattern to make an arg available at runtime: `ARG APP_VERSION` then `ENV APP_VERSION=${APP_VERSION}`.

3.2.5 WORKDIR

```
WORKDIR /app
```

Sets the working directory for subsequent `RUN`, `CMD`, `ENTRYPOINT`, `COPY`, `ADD`. Creates it if missing. Use absolute paths and prefer `WORKDIR` over `RUN cd /app && ...`.

3.2.6 COPY

```
COPY requirements.txt .
COPY --chown=app:app --chmod=0755 entrypoint.sh /usr/local/bin/
COPY --from=build /out/server /server
COPY --from=nginx:1.27-alpine /etc/nginx/nginx.conf /tmp/ref.conf
COPY --link package.json package-lock.json ./
COPY --exclude=*.md . /src
```

Copies files from the build context (or another stage/image with `--from`). Key flags:

- `--chown`, `--chmod`: set ownership and mode without an extra `RUN chown` layer (which would duplicate file data).
- `--from`: copy from a stage name, stage index, or any image reference.
- `--link`: copies into an independent layer that does not depend on the previous layers, so changes to earlier layers do not invalidate it and it can be rebased efficiently.
- `--exclude`: skip matching paths (Dockerfile frontend 1.7+ / stable in recent versions).
- `--parents`: preserve directory structure for glob matches.

3.2.7 ADD

```
ADD https://github.com/org/tool/releases/download/v1.2/tool.tar.gz /tmp/
ADD --checksum=sha256:4b1c... https://example.com/file.tar.gz /tmp/
ADD rootfs.tar.gz /
ADD git@github.com:org/repo.git#v1.2.0 /src
```

Like `COPY` plus extras: local tar archives are auto-extracted, remote URLs are fetched, and Git repositories can be cloned. Because of the implicit magic, **prefer `COPY` unless you need these features**. When fetching URLs, always use `--checksum` for integrity.

3.2.8 RUN

```
# Shell form: runs via /bin/sh -c
RUN apt-get update && apt-get install -y --no-install-recommends curl \
 && rm -rf /var/lib/apt/lists/*
```

```
# Exec form: no shell
RUN ["/bin/bash", "-c", "set -euo pipefail; ./build.sh"]

# Heredoc (Dockerfile 1.4+)
RUN <<EOF
set -eux
apt-get update
apt-get install -y --no-install-recommends ca-certificates
rm -rf /var/lib/apt/lists/*
EOF

# BuildKit mounts
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci
RUN --network=none make test
```

Each `RUN` creates a layer. Rules of thumb:

- Combine `update` and `install` in the same `RUN`, otherwise a cached stale `apt-get update` layer can be reused with a new install line.
- Clean caches in the **same** layer they were created in; deleting in a later layer does not shrink the image.
- `/bin/sh -c` does not have `pipefail`; use `SHELL` or `set -o pipefail` inside bash.

3.2.9 SHELL

```
SHELL ["/bin/bash", "-o", "pipefail", "-c"]
```

Changes the shell used by shell-form instructions. Useful for `pipefail` and essential on Windows images (PowerShell).

3.2.10 USER

```
RUN groupadd --system --gid 10001 app && useradd --system --uid 10001 --gid app app
USER 10001:10001
```

Sets the user for subsequent `RUN`, `CMD`, `ENTRYPOINT`, and for the running container. Use a numeric UID so Kubernetes `runAsNonRoot` can verify it without looking up `/etc/passwd`. Running as root is the default and is a security smell (Part 8).

3.2.11 EXPOSE

```
EXPOSE 8080/tcp
EXPOSE 9090/udp
```

Pure documentation plus a hint for `docker run -P`. It does **not** publish ports or open firewalls.

3.2.12 VOLUME

```
VOLUME /var/lib/postgresql/data
```

Declares a mount point; at runtime Docker creates an anonymous volume there if you do not mount one. Pitfalls: any build step that changes that path *after* `VOLUME` is discarded, and anonymous volumes accumulate silently. Many teams avoid `VOLUME` in app images and declare volumes in Compose or Kubernetes instead.

3.2.13 CMD and ENTRYPOINT

These define what runs. **Always prefer exec form (JSON array).**

```
ENTRYPOINT ["python", "-m", "myapp"]
CMD ["--port", "8080"]
```

- `ENTRYPOINT` is the executable; `CMD` provides default arguments.
- `docker run image --port 9000` replaces `CMD` only.
- `docker run --entrypoint sh image` replaces `ENTRYPOINT`.

	No ENTRYPOINT	`ENTRYPOINT ["e1"]`	`ENTRYPOINT e1` (shell)
No CMD	error	`e1`	`/bin/sh -c e1`
`CMD ["c1"]`	`c1`	`e1 c1`	`/bin/sh -c e1` (CMD ignored)
`CMD c1` (shell)	`/bin/sh -c c1`	`e1 /bin/sh -c c1`	`/bin/sh -c e1`

Shell form makes `/bin/sh` PID 1, which does not forward SIGTERM to your app, so `docker stop` waits and then kills it.

The wrapper-script pattern: an entrypoint script that does setup and then `exec`s the real process so it becomes PID 1:

```
#!/bin/sh
# docker-entrypoint.sh
set -e
if [ -n "$WAIT_FOR_DB" ]; then
  until nc -z "$DB_HOST" 5432; do echo "waiting for db"; sleep 1; done
fi
exec "$@"
```

```
COPY --chmod=0755 docker-entrypoint.sh /usr/local/bin/
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["gunicorn", "-b", "0.0.0.0:8080", "app:app"]
```

3.2.14 HEALTHCHECK

```
HEALTHCHECK --interval=30s --timeout=3s --start-period=20s --start-interval=2s --retries=3 \
  CMD ["wget", "-qO-", "http://127.0.0.1:8080/healthz"]
```

Docker runs the command inside the container and sets health status to `starting`, `healthy` or `unhealthy`. Exit 0 is healthy, 1 is unhealthy. Compose uses it for `depends_on: condition: service_healthy`; Swarm replaces unhealthy tasks. Plain `docker run` only reports it (it does not restart). Kubernetes ignores `HEALTHCHECK` and uses its own probes. `HEALTHCHECK NONE` disables an inherited check.

3.2.15 LABEL

```
LABEL org.opencontainers.image.source="https://github.com/acme/api" \
      org.opencontainers.image.revision="${GIT_SHA}" \
      org.opencontainers.image.licenses="Apache-2.0"
```

Key/value metadata. Use the standard `org.opencontainers.image.*` annotation keys; registries and scanners understand them. (`MAINTAINER` is deprecated; use a label.)

3.2.16 STOPSIGNAL, ONBUILD

```
STOPSIGNAL SIGQUIT          # nginx graceful shutdown
ONBUILD COPY . /app         # runs when another image uses this as a base (rarely
recommended)
```

`ONBUILD` hides behavior from downstream users; avoid it except in tightly controlled internal base images.

3.3 Layers and the Build Cache

3.3.1 How layers form

Each `RUN`, `COPY` and `ADD` produces a filesystem layer. Other instructions (`ENV`, `CMD`, `LABEL`, `EXPOSE`, `USER`, `WORKDIR`) change only the image config and produce empty layers. Layers are stacked using overlayfs at runtime (Part 7).

Consequences:

- Deleting a file in a later layer hides it but does not remove its bytes from the image. A secret copied in layer 3 and deleted in layer 4 is still extractable.
- Changing permissions with `RUN chmod -R` on large trees duplicates the data in a new layer. Use `COPY --chmod`.

3.3.2 Cache rules

BuildKit reuses a cached result for a step when:

- For `RUN`: the instruction string is identical and all parent layers are cached. It does **not** look at what the command would fetch from the network.
- For `COPY` / `ADD`: the checksums of the copied files (content and metadata, not mtime) are unchanged.
- Once one step misses the cache, every later step in that stage rebuilds (unless using `--link` layers or independent stages).

Therefore: **order instructions from least frequently changing to most frequently changing.**

Bad:

```
FROM node:22-alpine
WORKDIR /app
COPY . .
RUN npm ci
CMD ["node", "server.js"]
```

Any source change invalidates `COPY . .`, so `npm ci` reruns every build.

Good:

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY . .
USER node
CMD ["node", "server.js"]
```

Now dependency installation is cached until the lockfile changes.

```
$ docker build -t api .
 => [1/5] FROM docker.io/library/node:22-alpine@sha256:...       0.0s
 => CACHED [2/5] WORKDIR /app                                    0.0s
 => CACHED [3/5] COPY package.json package-lock.json ./          0.0s
 => CACHED [4/5] RUN npm ci --omit=dev                           0.0s
 => [5/5] COPY . .                                               0.1s
 => exporting to image                                           0.2s
```

3.3.3 Cache busting deliberately

```
ARG CACHE_BUST
RUN apt-get update && apt-get upgrade -y   # rerun when CACHE_BUST changes
```

```
docker build --build-arg CACHE_BUST=$(date +%F) .
```

Or `docker build --no-cache-filter stage-name .` to rebuild a single stage. Rebuilding base images regularly (with `--pull`) is how you pick up OS security patches.

3.4 Multi-Stage Builds

Multi-stage builds separate the **build environment** (compilers, dev dependencies, test tools) from the **runtime image**. Only what you `COPY --from` reaches the final image.

3.4.1 Go: from 900 MB to under 15 MB

```
# syntax=docker/dockerfile:1
FROM golang:1.23 AS build
WORKDIR /src
COPY go.mod go.sum ./
RUN go mod download
COPY . .
RUN CGO_ENABLED=0 go build -trimpath -ldflags="-s -w" -o /out/server ./cmd/server

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/server /server
USER nonroot:nonroot
EXPOSE 8080
ENTRYPOINT ["/server"]
```

3.4.2 Python with a virtual environment

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim-bookworm AS build
ENV PIP_DISABLE_PIP_VERSION_CHECK=1
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip pip install -r requirements.txt

FROM python:3.12-slim-bookworm AS runtime
ENV PYTHONUNBUFFERED=1 PATH="/opt/venv/bin:$PATH"
RUN useradd --system --uid 10001 app
COPY --from=build /opt/venv /opt/venv
WORKDIR /app
COPY --chown=10001:10001 src/ ./
USER 10001
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

3.4.3 Node.js frontend served by nginx

```
# syntax=docker/dockerfile:1
FROM node:22-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=cache,target=/root/.npm npm ci

FROM deps AS build
COPY . .
RUN npm run build

FROM deps AS test
COPY . .
RUN npm test

FROM nginxinc/nginx-unprivileged:1.27-alpine AS runtime
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 8080
```

```
docker build --target test .        # CI: run tests in the build
docker build --target runtime -t web:1.0 .
```

BuildKit builds only the stages needed for the requested target, and builds independent stages in parallel. A `test` stage that nothing depends on is skipped when you build `runtime`.

3.4.4 Stage patterns

Pattern	Use
`deps` -> `build` -> `runtime`	Cache dependencies separately from source.
`test` stage	Run unit tests during CI builds with `--target test`.
`dev` stage	Includes hot-reload tools; Compose uses `target: dev` locally.
`FROM runtime AS debug`	Adds shell and tools on top of a minimal runtime for troubleshooting.
Export stage	`FROM scratch AS export` + `COPY --from=build /out /` with `--output type=local,dest=./bin` to extract build artifacts without an image.

```
FROM scratch AS export
COPY --from=build /out/server /
```

```
docker build --target export --output type=local,dest=./bin .
ls ./bin   # server
```

3.5 Dockerfile Best-Practice Checklist

Practice	Why
Start with `# syntax=docker/dockerfile:1`	Latest stable frontend features.
Pin base image versions (and digests for prod)	Reproducibility, supply chain safety.
Use `.dockerignore`	Smaller context, no leaked secrets.
Order layers by change frequency	Maximize cache hits.
Multi-stage builds	Small runtime images without build tools.
One concern per container	Easier scaling, logging, restarts.
Exec-form `CMD` / `ENTRYPOINT`	Correct signal handling.
Non-root `USER` with numeric UID	Least privilege, Kubernetes compatibility.
`--no-install-recommends` and clean caches in the same layer	Smaller images.
`COPY` over `ADD`	Explicit, no surprise extraction.
No secrets in `ARG`, `ENV`, or copied files	They persist in layers and history.
OCI labels	Traceability back to source and commit.
Run `docker build --check .`	Built-in linting of Dockerfile issues.

Linting:

```
docker build --check .
docker run --rm -i hadolint/hadolint < Dockerfile
```

```
$ docker build --check .
WARNING: JSONArgsRecommended - JSON arguments recommended for CMD to prevent unintended
behavior related to OS signals
Dockerfile:12
```

3.6 Lab: Build, Measure and Optimize a Python API

Goal: turn a naive Dockerfile into an optimized multi-stage one and measure the difference.

1. Create the app:

```
mkdir -p ~/labs/pyapi/src && cd ~/labs/pyapi
cat > requirements.txt <<'EOF'
flask==3.0.3
gunicorn==23.0.0
```

```
EOF
cat > src/app.py <<'EOF'
from flask import Flask
app = Flask(__name__)

@app.get("/healthz")
def healthz():
    return {"status": "ok"}

@app.get("/")
def index():
    return {"hello": "docker"}
EOF
```

2. Naive Dockerfile:

```
# Dockerfile.naive
FROM python:3.12
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
CMD gunicorn -b 0.0.0.0:8000 --chdir src app:app
```

```
docker build -f Dockerfile.naive -t pyapi:naive .
docker build --check -f Dockerfile.naive .
```

3. Edit `src/app.py` (change the greeting) and rebuild. Observe that `pip install` reruns.

4. Write the optimized multi-stage Dockerfile from section 3.4.2 (adapt the CMD to `--chdir /app app:app` since `src/` is copied to `/app`), add a `.dockerignore`, and a `HEALTHCHECK` using Python:

```
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s \
  CMD ["python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/healthz')"]
```

5. Build and compare:

```
docker build -t pyapi:opt .
docker image ls pyapi
```

```
REPOSITORY   TAG     IMAGE ID       CREATED          SIZE
pyapi        opt     7a1c...        10 seconds ago   148MB
pyapi        naive   3e9b...        3 minutes ago    1.03GB
```

6. Edit the source again and rebuild `pyapi:opt`. Confirm the dependency layer is `CACHED`.

7. Run it and check health and stop behavior:

```
docker run -d --name pyapi -p 8000:8000 pyapi:opt
sleep 15; docker ps --filter name=pyapi --format '{{.Status}}'
time docker stop pyapi     # should be fast: gunicorn is PID 1 and handles SIGTERM
docker rm pyapi
```

Key takeaways

- The build context plus `.dockerignore` controls what can enter an image; always ignore `.git`, `.env` and local dependency folders.
- Every instruction has a precise purpose; know `ARG` vs `ENV`, `COPY` vs `ADD`, `CMD` vs `ENTRYPOINT`, and exec vs shell form.
- Layers are additive: deleting later never shrinks an image or removes a leaked secret.
- Order instructions by change frequency so dependency installs stay cached.
- Multi-stage builds keep compilers and dev tools out of runtime images and enable test, dev and export stages.
- Use `docker build --check` and hadolint to catch mistakes early.

Part 4 — BuildKit, Image Optimization and Registries

Part 3 taught you to write correct Dockerfiles. This Part makes builds fast, images small, and distribution professional: BuildKit features (cache mounts, secrets, SSH, buildx, remote cache, multi-platform), image minimization strategies (slim, alpine, distroless, scratch), and tagging and registry workflows across Docker Hub, ECR, GCR/Artifact Registry, ACR, GHCR and Harbor.

4.1 BuildKit

BuildKit has been the default builder since Docker Engine 23.0. `docker build` is now an alias for `docker buildx build`. Its advantages over the legacy builder:

- Builds a dependency graph (DAG) and executes independent stages in parallel.
- Skips stages the target does not need.
- Content-addressable cache that can be exported to and imported from registries or CI caches.
- `RUN --mount` for caches, secrets, SSH agents and bind mounts.
- Multi-platform builds and multiple output types.

4.1.1 Cache mounts

A cache mount persists a directory across builds **without** baking it into a layer. Package managers keep their download caches warm even when the layer itself is invalidated.

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

FROM golang:1.23 AS build
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go build -o /out/app ./...

FROM debian:bookworm-slim
RUN rm -f /etc/apt/apt.conf.d/docker-clean
RUN --mount=type=cache,target=/var/cache/apt,sharing=locked \
    --mount=type=cache,target=/var/lib/apt,sharing=locked \
    apt-get update && apt-get install -y --no-install-recommends curl
```

Ecosystem	Cache target
pip	`/root/.cache/pip`
uv	`/root/.cache/uv`

Ecosystem	Cache target
npm	`/root/.npm`
yarn	`/usr/local/share/.cache/yarn`
pnpm	`/root/.local/share/pnpm/store`
Go	`/go/pkg/mod`, `/root/.cache/go-build`
Maven	`/root/.m2`
Gradle	`/root/.gradle`
Cargo	`/usr/local/cargo/registry`, `/app/target`
apt	`/var/cache/apt`, `/var/lib/apt` (with `sharing=locked`)

> **Note:** Cache mounts live in the builder's local cache. On ephemeral CI runners they start empty unless you persist the builder state; layer cache export (section 4.1.5) is usually the bigger CI win.

4.1.2 Secret mounts

Secrets are mounted as files (or env vars) only during a single `RUN`, never written into a layer or history.

```
# syntax=docker/dockerfile:1
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc npm ci
RUN --mount=type=secret,id=api_token,env=API_TOKEN ./fetch-assets.sh
```

```
docker build --secret id=npmrc,src=$HOME/.npmrc \
             --secret id=api_token,env=API_TOKEN \
             -t app .
```

Compare with the anti-pattern `ARG NPM_TOKEN` which ends up readable via `docker history`.

4.1.3 SSH mounts

To clone private Git repositories during a build without copying keys:

```
RUN --mount=type=ssh git clone git@github.com:acme/private-lib.git /src/lib
```

```
eval "$(ssh-agent)" && ssh-add ~/.ssh/id_ed25519
docker build --ssh default .
```

Remember to add the host key (`ssh-keyscan github.com >> /etc/ssh/ssh_known_hosts` in an earlier step) so the clone does not prompt.

4.1.4 Bind mounts in RUN

```
RUN --mount=type=bind,source=requirements.txt,target=/tmp/requirements.txt \
    pip install -r /tmp/requirements.txt
```

The file is available during that step but not copied into a layer. Useful for files only needed at build time.

4.1.5 buildx, builders and drivers

`docker buildx` manages builder instances:

```
docker buildx ls
docker buildx create --name ci --driver docker-container --use --bootstrap
docker buildx inspect ci
docker buildx du               # builder cache usage
docker buildx prune --keep-storage 20gb
```

Driver	Description
`docker`	Built into the daemon. Simple, but limited cache export and multi-platform (unless containerd image store is enabled).
`docker-container`	BuildKit in a container. Full features: all cache backends, multi-platform.
`kubernetes`	BuildKit pods in a cluster; scales builds horizontally.
`remote`	Connect to an existing BuildKit daemon.
Docker Build Cloud	Managed remote builders with shared cache (subscription).

4.1.6 Remote cache backends

On ephemeral CI runners the local cache is empty every run. Export the cache somewhere persistent:

```
# Registry cache (works anywhere)
docker buildx build \
  --cache-from type=registry,ref=ghcr.io/acme/api:buildcache \
  --cache-to   type=registry,ref=ghcr.io/acme/api:buildcache,mode=max \
  -t ghcr.io/acme/api:1.4.2 --push .

# GitHub Actions cache
docker buildx build --cache-from type=gha --cache-to type=gha,mode=max .

# Inline cache (metadata embedded in the pushed image, min mode only)
docker buildx build --cache-to type=inline -t ghcr.io/acme/api:1.4.2 --push .
```

`mode=min` caches only layers of the final image; `mode=max` caches all intermediate stages too, which matters for multi-stage builds. Other backends: `type=local`, `type=s3`, `type=azblob`.

4.1.7 Multi-platform builds

ARM servers (AWS Graviton, Ampere) and Apple Silicon laptops make multi-architecture images a normal requirement.

Two strategies:

1. **Emulation (QEMU)**: simple, slow for compile-heavy builds.
2. **Cross-compilation**: build on the native platform and target another. Fast; requires language support (Go, Rust, many others).

```
# One-time: register QEMU emulators (Docker Desktop has them built in)
docker run --privileged --rm tonistiigi/binfmt --install all

docker buildx build --platform linux/amd64,linux/arm64 \
  -t ghcr.io/acme/api:1.4.2 --push .
```

Cross-compiling Go:

```
# syntax=docker/dockerfile:1
FROM --platform=$BUILDPLATFORM golang:1.23 AS build
ARG TARGETOS TARGETARCH
WORKDIR /src
COPY . .
RUN --mount=type=cache,target=/go/pkg/mod \
    CGO_ENABLED=0 GOOS=$TARGETOS GOARCH=$TARGETARCH go build -o /out/api ./cmd/api

FROM gcr.io/distroless/static-debian12:nonroot
COPY --from=build /out/api /api
ENTRYPOINT ["/api"]
```

The build stage always runs natively (`$BUILDPLATFORM`); only the tiny final stage is per-platform. Result: an image index with one manifest per architecture, and each host pulls the right one automatically.

> **Tip:** With the classic image store, a multi-platform result cannot be loaded into the local `docker images` list; you push it. With the containerd image store enabled, you can `--load` multi-platform images locally.

4.1.8 Outputs, attestations and Bake

```
docker buildx build -o type=local,dest=./out .          # files to disk
docker buildx build -o type=tar,dest=out.tar .
docker buildx build -o type=oci,dest=app-oci.tar .
docker buildx build --load -t app:dev .                 # into local image store
```

```
docker buildx build --push -t ghcr.io/acme/app:1.0 .
docker buildx build --sbom=true --provenance=mode=max --push -t ghcr.io/acme/app:1.0 .
```

`--sbom` and `--provenance` attach SBOM and SLSA provenance attestations to the image (Part 8).

Bake defines builds declaratively, like Compose for builds:

```
# docker-bake.hcl
variable "TAG" { default = "dev" }

group "default" { targets = ["api", "web"] }

target "api" {
  context    = "./api"
  tags       = ["ghcr.io/acme/api:${TAG}"]
  platforms  = ["linux/amd64", "linux/arm64"]
  cache-from = ["type=registry,ref=ghcr.io/acme/api:buildcache"]
  cache-to   = ["type=registry,ref=ghcr.io/acme/api:buildcache,mode=max"]
}

target "web" {
  context = "./web"
  tags    = ["ghcr.io/acme/web:${TAG}"]
}
```

```
TAG=1.4.2 docker buildx bake --push
docker buildx bake --print     # show resolved config
```

4.2 Image Optimization

4.2.1 Why size matters

- Faster pulls mean faster scaling and faster deploys, especially on cold nodes.
- Fewer packages mean fewer CVEs and a smaller attack surface.
- Less storage and registry egress cost.

4.2.2 Base image choices

Base	Approx size	Has shell / package manager	libc	Notes
`ubuntu:24.04`	~78 MB	yes / apt	glibc	Familiar, large.
`debian:bookworm-slim`	~75 MB	yes / apt	glibc	Good general default.
`python:3.12-slim`	~125 MB	yes / apt	glibc	Language-specific slim.

Base	Approx size	Has shell / package manager	libc	Notes
`alpine:3.20`	~8 MB	yes / apk	musl	Tiny; musl can break or slow some native wheels and DNS edge cases.
`gcr.io/distroless/base-debian12`	~20 MB	no / no	glibc	Runtime-only, has CA certs, tzdata.
`gcr.io/distroless/static-debian12`	~2 MB	no / no	none	For static binaries.
Chainguard / Wolfi images	small	variants with and without shell	glibc	Minimal, frequently rebuilt, low CVE counts.
Docker Hardened Images	small	minimal	varies	Docker's hardened, low-CVE catalog.
`scratch`	0 MB	no / no	none	Empty; you supply everything.

4.2.3 Distroless and scratch

Distroless images contain only your app's runtime dependencies: no shell, no package manager, no coreutils. Attackers who get code execution have no tools, and scanners have little to flag. `:nonroot` tags default to UID 65532.

scratch is an empty image. For a statically linked binary you need to bring anything else yourself:

```
FROM golang:1.23-alpine AS build
RUN apk add --no-cache ca-certificates tzdata
WORKDIR /src
COPY . .
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /app .

FROM scratch
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=build /usr/share/zoneinfo /usr/share/zoneinfo
COPY --from=build /app /app
USER 65532:65532
ENTRYPOINT ["/app"]
```

Common scratch gotchas: missing CA certificates (TLS fails with "x509: certificate signed by unknown authority"), missing timezone data, no `/etc/passwd` (so use numeric UID), no `/tmp` (mount a tmpfs), and dynamically linked binaries failing with a confusing "no such file or directory" because the loader is absent.

4.2.4 Optimization techniques summary

1. Multi-stage builds; copy only artifacts.
2. Choose the smallest base that works; prefer distroless for runtime.

3. `--no-install-recommends` (apt), `apk add --no-cache`, `pip --no-cache-dir` or cache mounts.
4. Remove build deps and caches in the same `RUN`.
5. Install only production dependencies (`npm ci --omit=dev`, `poetry install --only main`).
6. Strip binaries (`-ldflags="-s -w"`).
7. Keep `.dockerignore` tight.
8. Analyze layers with `dive`:

```
docker run --rm -it -v /var/run/docker.sock:/var/run/docker.sock wagoodman/dive:latest
myapp:1.0
```

`dive` shows each layer's files and an "efficiency score", highlighting wasted space from files overwritten or deleted in later layers.

4.3 Tagging Strategy

Tags should communicate meaning and support rollbacks.

Tag	Example	Mutable?	Use
Semantic version	`1.4.2`	Should be immutable	Releases.
Minor/major float	`1.4`, `1`	Mutable	Consumers that want patches automatically.
Git SHA	`sha-3f9c2ab`	Immutable	Traceability from CI; deploy exactly what was tested.
Branch	`main`, `pr-123`	Mutable	Preview environments.
`latest`	`latest`	Mutable	Convenience only; never deploy it.

A typical CI produces several tags for the same digest:

```
docker buildx build \
  -t ghcr.io/acme/api:1.4.2 \
  -t ghcr.io/acme/api:1.4 \
  -t ghcr.io/acme/api:sha-3f9c2ab \
  --push .
```

Deploy by digest for maximum safety:

```
docker buildx imagetools inspect ghcr.io/acme/api:1.4.2 --format '{{.Manifest.Digest}}'
# deploy: ghcr.io/acme/api@sha256:5e1f...
```

4.4 Registries

4.4.1 Docker Hub

```
docker login                    # interactive, or use a personal access token
echo "$DOCKERHUB_TOKEN" | docker login -u myuser --password-stdin
docker tag api:1.4.2 myuser/api:1.4.2
docker push myuser/api:1.4.2
```

> **Warning:** Docker Hub enforces pull rate limits for unauthenticated and free users. CI systems pulling public images anonymously from shared IPs hit limits quickly. Authenticate, use a pull-through cache, or mirror base images into your own registry.

Credentials are stored in `~/.docker/config.json`, ideally via a credential helper (`osxkeychain`, `desktop`, `pass`, `secretservice`, `ecr-login`) rather than base64 plaintext.

4.4.2 AWS ECR

```
aws ecr create-repository --repository-name api \
  --image-scanning-configuration scanOnPush=true \
  --image-tag-mutability IMMUTABLE

aws ecr get-login-password --region eu-west-1 \
  | docker login --username AWS --password-stdin 123456789012.dkr.ecr.eu-west-1.amazonaws.com

docker tag api:1.4.2 123456789012.dkr.ecr.eu-west-1.amazonaws.com/api:1.4.2
docker push 123456789012.dkr.ecr.eu-west-1.amazonaws.com/api:1.4.2
```

ECR tips: tokens last 12 hours (or use `amazon-ecr-credential-helper`), configure **lifecycle policies** to expire untagged and old images, use **pull-through cache rules** for Docker Hub/GHCR/Quay, and enable tag immutability.

```
{
  "rules": [
    {
      "rulePriority": 1,
      "description": "Expire untagged after 7 days",
      "selection": { "tagStatus": "untagged", "countType": "sinceImagePushed", "countUnit":
"days", "countNumber": 7 },
      "action": { "type": "expire" }
    }
  ]
}
```

4.4.3 Google Artifact Registry (successor to GCR)

Google Container Registry has been replaced by **Artifact Registry**.

```
gcloud artifacts repositories create apps --repository-format=docker --location=europe-west1
gcloud auth configure-docker europe-west1-docker.pkg.dev
docker tag api:1.4.2 europe-west1-docker.pkg.dev/my-project/apps/api:1.4.2
docker push europe-west1-docker.pkg.dev/my-project/apps/api:1.4.2
```

4.4.4 Azure Container Registry

```
az acr create -n acmeregistry -g rg-platform --sku Standard
az acr login -n acmeregistry
docker tag api:1.4.2 acmeregistry.azurecr.io/api:1.4.2
docker push acmeregistry.azurecr.io/api:1.4.2
az acr build -r acmeregistry -t api:1.4.2 .   # build in Azure, no local Docker needed
```

4.4.5 GitHub Container Registry (GHCR)

```
echo "$GITHUB_TOKEN" | docker login ghcr.io -u "$GITHUB_ACTOR" --password-stdin
docker push ghcr.io/acme/api:1.4.2
```

In GitHub Actions, the built-in `GITHUB_TOKEN` with `packages: write` permission is enough.

4.4.6 Harbor

Harbor is a CNCF-graduated, self-hosted registry adding projects with RBAC, vulnerability scanning (Trivy), replication between registries, signature verification policies, quotas, retention rules and proxy caching. It is a common choice for on-prem and air-gapped environments.

```
docker login harbor.corp.example.com
docker tag api:1.4.2 harbor.corp.example.com/payments/api:1.4.2
docker push harbor.corp.example.com/payments/api:1.4.2
```

4.4.7 Running a private registry

The open-source `registry` image (CNCF Distribution) is enough for labs and simple setups:

```
docker run -d --name registry -p 5000:5000 \
  -v registry-data:/var/lib/registry \
  --restart unless-stopped registry:2

docker tag alpine:3.20 localhost:5000/alpine:3.20
docker push localhost:5000/alpine:3.20
```

```
curl -s localhost:5000/v2/_catalog
curl -s localhost:5000/v2/alpine/tags/list
```

```
{"repositories":["alpine"]}
{"name":"alpine","tags":["3.20"]}
```

For anything beyond localhost, you need TLS and authentication. Docker refuses plain-HTTP registries other than `localhost` unless you add them to `insecure-registries` in `daemon.json` (do not do this in production). A production-ish setup uses htpasswd auth and certificates:

```
mkdir -p auth certs
docker run --rm --entrypoint htpasswd httpd:2 -Bbn admin 'S3cure!' > auth/htpasswd
docker run -d --name registry -p 443:5000 \
  -v "$PWD/auth:/auth" -v "$PWD/certs:/certs" -v registry-data:/var/lib/registry \
  -e REGISTRY_AUTH=htpasswd -e REGISTRY_AUTH_HTPASSWD_REALM="Registry" \
  -e REGISTRY_AUTH_HTPASSWD_PATH=/auth/htpasswd \
  -e REGISTRY_HTTP_TLS_CERTIFICATE=/certs/domain.crt \
  -e REGISTRY_HTTP_TLS_KEY=/certs/domain.key \
  registry:2
```

Configure it as a Docker Hub pull-through mirror by setting `REGISTRY_PROXY_REMOTEURL=https://registry-1.docker.io` and adding `"registry-mirrors": ["https://mirror.corp.example.com"]` to clients' `daemon.json`.

Deleting images from the registry requires `REGISTRY_STORAGE_DELETE_ENABLED=true`, a DELETE by digest via the API, then running garbage collection (`registry garbage-collect /etc/docker/registry/config.yml`).

4.4.8 Copying images between registries

Do not `pull`/`tag`/`push` through your laptop for large or multi-arch images. Use registry-to-registry tools:

```
docker buildx imagetools create -t ghcr.io/acme/api:1.4.2 docker.io/acme/api:1.4.2
crane copy docker.io/library/nginx:1.27-alpine harbor.corp.example.com/mirror/nginx:1.27-alpine
skopeo copy --all docker://nginx:1.27-alpine docker://registry.local/nginx:1.27-alpine
```

These preserve all platforms in the index.

4.5 Lab: Multi-Platform Build to a Local Registry With Remote Cache

Goal: build a Go service for amd64 and arm64, push to a local registry, and reuse cache across builder instances.

1. Start a local registry on a network the builder can reach:

```
docker network create buildnet
docker run -d --name registry --network buildnet -p 5000:5000 registry:2
```

2. Create the app:

```
mkdir -p ~/labs/gohello && cd ~/labs/gohello
cat > main.go <<'EOF'
package main

import (
    "fmt"
    "net/http"
    "runtime"
)

func main() {
    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintf(w, "hello from %s/%s\n", runtime.GOOS, runtime.GOARCH)
    })
    http.ListenAndServe(":8080", nil)
}
EOF
cat > go.mod <<'EOF'
module gohello

go 1.23
EOF
```

3. Write the cross-compiling Dockerfile from section 4.1.7 (adjust `./cmd/api` to `.`).

4. Create a `docker-container` builder attached to the network, allowing HTTP to the lab registry:

```
cat > buildkitd.toml <<'EOF'
[registry."registry:5000"]
  http = true
EOF
docker buildx create --name labbuilder --driver docker-container \
  --driver-opt network=buildnet --config buildkitd.toml --use --bootstrap
```

5. Build and push both platforms with registry cache:

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t registry:5000/gohello:1.0 \
  --cache-to type=registry,ref=registry:5000/gohello:cache,mode=max \
  --cache-from type=registry,ref=registry:5000/gohello:cache \
  --push .
```

```
curl -s -H 'Accept: application/vnd.oci.image.index.v1+json' \
  localhost:5000/v2/gohello/manifests/1.0 | jq '.manifests[].platform'
```

You should see entries for `amd64` and `arm64` (plus `unknown/unknown` entries, which are attestation manifests).

6. Remove the builder (and its local cache), recreate it, and rebuild. Observe `CACHED` steps imported from the registry:

```
docker buildx rm labbuilder
docker buildx create --name labbuilder --driver docker-container \
  --driver-opt network=buildnet --config buildkitd.toml --use --bootstrap
# rerun the build command from step 5
```

7. Run each platform (emulated if necessary):

```
docker run --rm -d -p 8081:8080 --platform linux/arm64 --name a localhost:5000/gohello:1.0
docker run --rm -d -p 8082:8080 --platform linux/amd64 --name b localhost:5000/gohello:1.0
curl localhost:8081; curl localhost:8082
```

```
hello from linux/arm64
hello from linux/amd64
```

8. Clean up: `docker rm -f a b registry`, `docker buildx rm labbuilder`, `docker network rm buildnet`.

Key takeaways

- BuildKit is the default builder: use `RUN --mount=type=cache` for package caches, `type=secret` and `type=ssh` for credentials.
- Use a `docker-container` builder plus registry or GHA cache export to make CI builds fast.
- Prefer cross-compilation with `$BUILDPLATFORM` / `$TARGETARCH` over QEMU emulation for multi-arch images.
- Distroless and scratch shrink both size and attack surface; remember CA certs, tzdata and numeric UIDs.
- Tag with semver and Git SHA, never deploy `latest`, and deploy by digest when you can.
- Every cloud has an OCI registry; enable scanning, immutability and lifecycle policies, and mirror Docker Hub to avoid rate limits.

Part 5 — Storage and Networking

Containers are ephemeral by design: their writable layer disappears when they are removed. Real systems need persistent data and need to talk to each other and to the outside world. This Part covers Docker's three storage mechanisms (volumes, bind mounts, tmpfs) and its networking model in depth, down to the iptables rules that make port publishing work.

5.1 Storage

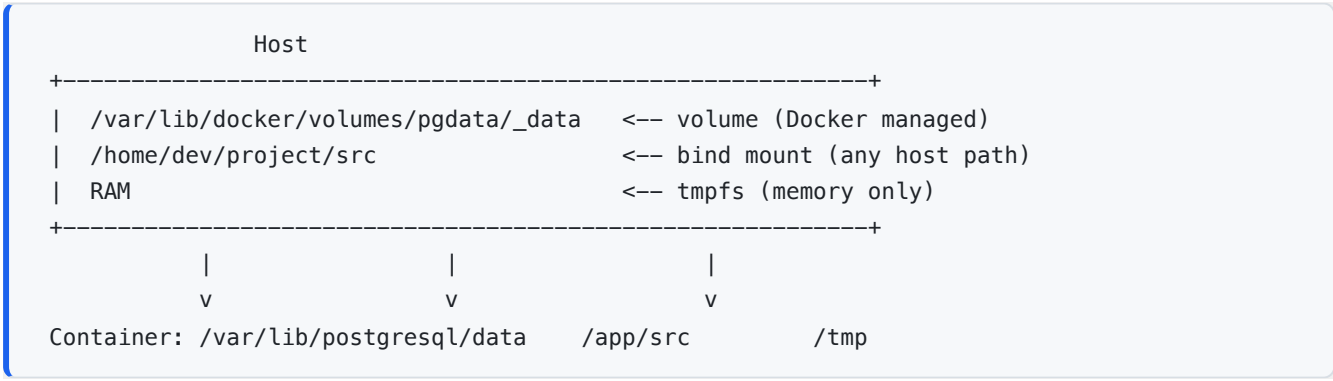
5.1.1 The container writable layer

Every container gets a thin writable layer on top of the read-only image layers. Writes there:

- Are lost when the container is removed.
- Use copy-on-write: modifying a 1 GB file from the image first copies the whole file up into the writable layer.
- Are slower than writing to a volume, because they go through the overlay filesystem.

So: write application data to volumes, write logs to stdout, and treat the container filesystem as read-only when you can.

5.1.2 The three mount types



Type	Managed by	Location	Typical use
Volume	Docker	`/var/lib/docker/volumes/<name>/_data` (or a driver backend)	Databases, persistent app state, sharing between containers.
Bind mount	You	Any host path	Source code in development, config files, host sockets.
tmpfs	Kernel memory	RAM	Scratch space, secrets that must never touch disk, `/tmp` on read-only containers.

5.1.3 `-v` vs `--mount`

Both work; `--mount` is explicit and errors instead of silently creating things.

```
# Named volume
docker run -d -v pgdata:/var/lib/postgresql/data postgres:16
docker run -d --mount type=volume,src=pgdata,dst=/var/lib/postgresql/data postgres:16

# Bind mount (read-only)
docker run -d -v "$PWD/nginx.conf:/etc/nginx/nginx.conf:ro" nginx:1.27-alpine
docker run -d --mount type=bind,src="$PWD/nginx.conf",dst=/etc/nginx/nginx.conf,readonly
nginx:1.27-alpine

# tmpfs
docker run -d --tmpfs /tmp:rw,size=64m,mode=1777 myapp
docker run -d --mount type=tmpfs,dst=/tmp,tmpfs-size=67108864 myapp
```

> **Warning:** With `-v`, if the host path of a bind mount does not exist, Docker creates it as an empty **directory** owned by root. Mounting a "file" that does not exist yet gives you a directory and a confusing app error. `--mount` fails loudly instead.

5.1.4 Volume operations

```
docker volume create pgdata
docker volume ls
docker volume inspect pgdata
docker volume rm pgdata
docker volume prune            # removes unused anonymous volumes
docker volume prune -a         # also unused named volumes
```

```
[
  {
    "CreatedAt": "2026-10-12T09:14:02Z",
    "Driver": "local",
    "Labels": null,
    "Mountpoint": "/var/lib/docker/volumes/pgdata/_data",
    "Name": "pgdata",
    "Options": null,
    "Scope": "local"
  }
]
```

A useful volume behavior: when you mount an **empty** named volume over a directory that contains files in the image, Docker copies the image's files into the volume first. Bind mounts never do this; they always hide the image content.

5.1.5 Volume drivers and NFS

The `local` driver can mount NFS or other filesystems through mount options:

```
docker volume create --driver local \
  --opt type=nfs --opt o=addr=10.0.0.20,rw,nfsvers=4 \
  --opt device=:/exports/shared \
  shared-nfs
```

Plugins exist for cloud block storage and distributed filesystems, though in clustered environments this job has largely moved to Kubernetes CSI drivers.

5.1.6 Permissions and UID mapping

The most common storage problem is “permission denied”. Files in a bind mount keep their **host** numeric UID/GID. If the container runs as UID 10001 and the host directory is owned by UID 1000, writes fail.

Fixes:

- Run the container with a matching user: `-u "$(id -u):$(id -g)"` (great for dev tooling containers).
- `chown` the host directory to the container’s UID.
- For named volumes, create the directory with correct ownership in the Dockerfile before `VOLUME` /mount; the volume copy-up preserves ownership.
- On SELinux hosts (RHEL, Fedora), add `:z` (shared) or `:Z` (private) to relabel bind mounts: `-v "$PWD/data:/data:Z"`.

5.1.7 Backup and restore

Volumes are just directories; back them up by mounting them into a throwaway container:

```
# Backup
docker run --rm -v pgdata:/data:ro -v "$PWD:/backup" alpine:3.20 \
  tar czf /backup/pgdata-$(date +%F).tar.gz -C /data .

# Restore into a new volume
docker volume create pgdata-restored
docker run --rm -v pgdata-restored:/data -v "$PWD:/backup" alpine:3.20 \
  tar xzf /backup/pgdata-2026-10-12.tar.gz -C /data
```

> **Warning:** A file-level copy of a running database can be inconsistent. Use the database’s own tool (`pg_dump`, `mysqldump`, `mongodump`) or stop the container first.

5.1.8 Performance notes for Docker Desktop

On macOS and Windows, bind mounts cross the VM boundary and can be slow for workloads with many small files (like `node_modules`). Docker Desktop uses VirtioFS (fast), and the synchronized file shares feature helps further. A common pattern is to bind mount source code but keep dependencies in a named volume:

```
services:
  web:
    volumes:
      - ./:/app
      - node_modules:/app/node_modules
volumes:
  node_modules:
```

5.2 Networking Fundamentals

5.2.1 Network drivers

Driver	Scope	Description
`bridge`	Single host	Default. Private virtual network with NAT to the outside.
`host`	Single host	Container shares the host's network namespace. No isolation, no port mapping, best performance.
`none`	Single host	Only a loopback interface.
`overlay`	Multi-host (Swarm)	VXLAN-based network spanning cluster nodes.
`macvlan`	Single host, L2	Container gets its own MAC and IP on the physical LAN.
`ipvlan`	Single host, L2/L3	Like macvlan but shares the parent's MAC; works where switches limit MACs.

```
docker network ls
```

```
NETWORK ID     NAME      DRIVER    SCOPE
3c1e5b0d9f2a   bridge    bridge    local
8a7d6c5b4e3f   host      host      local
1f2e3d4c5b6a   none      null      local
```

5.2.2 How the default bridge works

```
                      Host network namespace
+-------------------------------------------------------------+
|  eth0 (192.168.1.50) <---- NAT (iptables MASQUERADE) ----+      |
```

```
|                                                        |      |
|               docker0 bridge  172.17.0.1/16 -----------+      |
|                /                          \                   |
|          vethA1                         vethB1                 |
+-----------|-------------------------------|--------------------+
            |  veth pair                    |  veth pair
+-----------|---------+         +-----------|----------+
|  eth0 172.17.0.2    |         |  eth0 172.17.0.3     |
|  container A netns  |         |  container B netns   |
+---------------------+         +----------------------+
```

Each container gets a network namespace with its own `eth0`, which is one end of a **veth pair**; the other end is plugged into the Linux bridge `docker0`. The bridge forwards between containers; outbound traffic is NATed via iptables `MASQUERADE` so it appears to come from the host's IP.

5.2.3 User-defined bridges: always use them

The default `bridge` network is legacy. It has **no automatic DNS** between containers (only the deprecated `--link`). User-defined bridges provide:

- Embedded DNS: containers resolve each other by name and network alias.
- Better isolation: only containers on the same network can talk.
- Containers can be connected and disconnected at runtime.

```
docker network create --driver bridge --subnet 10.20.0.0/24 backend
docker run -d --name db --network backend -e POSTGRES_PASSWORD=dev postgres:16
docker run --rm --network backend postgres:16 pg_isready -h db
```

```
db:5432 - accepting connections
```

Multi-network isolation:

```
docker network create frontend
docker network create backend --internal      # no outbound internet
docker run -d --name api --network backend myorg/api
docker network connect frontend api           # api is on both networks
docker run -d --name proxy --network frontend -p 443:443 nginx:1.27-alpine
# proxy can reach api; proxy cannot reach db (backend only)
```

5.2.4 Embedded DNS

On user-defined networks, containers use Docker's embedded DNS server at `127.0.0.11`. It resolves container names, service names (in Compose), and network aliases, and forwards everything else to the host's resolvers.

```
docker run --rm --network backend alpine:3.20 cat /etc/resolv.conf
```

```
nameserver 127.0.0.11
options ndots:0
```

Aliases give multiple names, and multiple containers with the same alias get round-robin DNS:

```
docker run -d --network backend --network-alias search elasticsearch:8.15.0
```

Custom DNS settings: `--dns 10.0.0.2`, `--dns-search corp.local`, `--add-host api.local:10.0.0.9`, or `--add-host host.docker.internal:host-gateway` to reach the host from a container on Linux (Docker Desktop provides `host.docker.internal` automatically).

5.2.5 Port publishing

```
docker run -d -p 8080:80 nginx              # all host interfaces, TCP
docker run -d -p 127.0.0.1:8080:80 nginx    # localhost only
docker run -d -p 8080:80/udp myudpapp
docker run -d -p 8000-8010:8000-8010 app
docker run -d -p 80 nginx                   # random host port
docker port <container>
```

> **Warning:** `-p 8080:80` binds to `0.0.0.0` (and `::`), exposing the service on every interface. On a cloud VM with a public IP, that is the internet. Bind databases and admin ports to `127.0.0.1`.

5.2.6 iptables: what Docker does behind the scenes

Docker programs netfilter rules (iptables, or nftables in newer releases where enabled) to implement NAT and isolation. For `-p 8080:80` on a container at `172.17.0.2`:

```
sudo iptables -t nat -L DOCKER -n
```

```
Chain DOCKER (2 references)
target     prot opt source       destination
RETURN     all  --  0.0.0.0/0    0.0.0.0/0
DNAT       tcp  --  0.0.0.0/0    0.0.0.0/0    tcp dpt:8080 to:172.17.0.2:80
```

```
sudo iptables -t nat -L POSTROUTING -n | grep MASQ
```

```
MASQUERADE  all  --  172.17.0.0/16   0.0.0.0/0
```

Chains to know:

Chain	Table	Purpose
`DOCKER`	nat	DNAT for published ports.
`POSTROUTING` (MASQUERADE)	nat	Outbound NAT for container subnets.
`DOCKER`	filter	Accepts traffic to published container ports.
`DOCKER-ISOLATION-STAGE-1/2`	filter	Blocks traffic between different bridge networks.
`DOCKER-USER`	filter	**Your** rules; evaluated first, Docker never touches it.

> **Warning:** Published ports bypass host firewalls like `ufw` and `firewalld` rules in the INPUT chain, because the traffic is DNATed and goes through FORWARD instead. To restrict access to published ports, add rules to `DOCKER-USER`:

```
# Allow only the office network to reach published container ports via eth0
sudo iptables -I DOCKER-USER -i eth0 ! -s 203.0.113.0/24 -j DROP
```

The `userland-proxy` (`docker-proxy` processes) handles edge cases like hairpin traffic from localhost; with `"userland-proxy": false` the kernel handles everything via iptables and you avoid one process per published port.

5.2.7 host and none

```
docker run -d --network host nginx:1.27-alpine   # listens directly on host :80
docker run --rm --network none alpine:3.20 ip addr
```

`host` is useful for performance-sensitive or port-range-heavy workloads (some monitoring agents, VoIP), at the cost of isolation and port conflicts. On Docker Desktop, host networking refers to the VM (support for host networking on Desktop exists but behaves differently from Linux). `none` is for batch jobs that must not touch the network.

5.2.8 Container network sharing

```
docker run -d --name app myapp
docker run --rm -it --network container:app nicolaka/netshoot
```

The second container joins `app`'s network namespace: same IP, same `localhost`. This is exactly how a Kubernetes Pod works (the "pause" container holds the namespace) and is the standard way to debug networking in a minimal container.

5.2.9 macvlan and ipvlan

When a container must appear as a first-class device on the physical LAN (legacy apps, network appliances, DHCP-dependent software):

```
docker network create -d macvlan \
  --subnet 192.168.1.0/24 --gateway 192.168.1.1 \
  --ip-range 192.168.1.192/27 \
  -o parent=eth0 lan
docker run -d --network lan --ip 192.168.1.200 --name legacy nginx:1.27-alpine
```

Caveats: the host cannot talk to its own macvlan containers directly without an extra macvlan interface on the host; many cloud networks and Wi-Fi adapters drop frames from unknown MACs (use `ipvlan` there).

5.2.10 Overlay networks

Overlay networks connect containers across hosts in a Swarm using VXLAN encapsulation (UDP 4789), with control traffic on TCP 2377 and gossip on TCP/UDP 7946.

```
docker network create -d overlay --attachable --opt encrypted appnet
```

`--attachable` allows standalone containers (not only Swarm services) to join; `encrypted` enables IPsec between nodes. Covered further in Part 9.

5.2.11 IPv6

Enable per network:

```
docker network create --ipv6 --subnet 2001:db8:1::/64 v6net
```

Recent Engine versions publish ports on IPv6 by default and support IPv6-only networks; check `ip6tables` rules if v6 traffic misbehaves.

5.3 Networking Troubleshooting Toolkit

```
docker network inspect backend --format '{{range .Containers}}{{.Name}} {{.IPv4Address}}
{{"\n"}}{{end}}'
docker exec api getent hosts db                # DNS resolution from inside
docker run --rm --network container:api nicolaka/netshoot \
  sh -c 'dig db; curl -sv http://db:5432; ss -tlnp'
sudo nsenter -t "$(docker inspect -f '{{.State.Pid}}' api)" -n ss -tlnp   # host-side
```

Symptom	Likely cause
`could not resolve host: db`	Containers on the default bridge, or on different networks.
Connection refused from host on published port	App listens on `127.0.0.1` inside the container; it must bind `0.0.0.0`.
Works locally, not from other machines	Bound to `127.0.0.1:port`, or cloud security group/firewall.

Symptom	Likely cause
No internet from containers	`ip_forward` disabled, broken iptables rules, or DNS unreachable (corporate DNS).
Subnet clash, VPN breaks	Docker network overlaps corporate range; set `default-address-pools`.
Port already allocated	Another container or host process is using that host port.

5.4 Lab: Volumes, Isolated Networks and Firewalling

Goal: build a three-tier topology, verify DNS and isolation, persist data, and restrict a published port.

1. Networks:

```
docker network create frontend
docker network create --internal backend
```

2. Database with a named volume on the internal network:

```
docker run -d --name db --network backend \
  -e POSTGRES_PASSWORD=labpass \
  -v labdb:/var/lib/postgresql/data postgres:16
```

3. An "API" (here `netshoot` as a stand-in) on both networks, and a proxy on frontend only:

```
docker run -d --name api --network backend nicolaka/netshoot sleep infinity
docker network connect frontend api
docker run -d --name proxy --network frontend -p 127.0.0.1:8080:80 nginx:1.27-alpine
```

4. Verify DNS and isolation:

```
docker exec api nc -zv db 5432            # succeeds
docker exec proxy getent hosts api        # resolves
docker exec proxy getent hosts db         # fails: different network
docker exec api ping -c1 -W2 8.8.8.8      # succeeds: api also has egress via frontend
docker run --rm --network backend alpine:3.20 ping -c1 -W2 8.8.8.8   # fails: backend is
internal-only
```

5. Persistence: create a table, destroy the container, recreate it with the same volume, confirm the table is still there.

```
docker exec db psql -U postgres -c 'CREATE TABLE notes(id int);'
docker rm -f db
docker run -d --name db --network backend -e POSTGRES_PASSWORD=labpass \
```

```
  -v labdb:/var/lib/postgresql/data postgres:16
sleep 5; docker exec db psql -U postgres -c '\dt'
```

6. Back up the volume with the tar method in 5.1.7.
7. On a Linux host, inspect the NAT rule for port 8080 and add a `DOCKER-USER` rule that drops traffic from a test range. Remove it afterwards with `iptables -D`.
8. Clean up:

```
docker rm -f db api proxy
docker network rm frontend backend
docker volume rm labdb
```

Key takeaways

- Use named volumes for persistent data, bind mounts for development and config, tmpfs for scratch and sensitive in-memory data.
- Prefer `--mount` in scripts: it is explicit and fails instead of silently creating directories.
- Bind mounts keep host UIDs; mismatched UIDs cause most “permission denied” errors.
- Always use user-defined networks for name-based DNS via `127.0.0.11`; use `--internal` networks for backends.
- Port publishing is iptables DNAT; published ports bypass `ufw`, so filter in `DOCKER-USER` and bind to `127.0.0.1` when possible.
- `--network container:<name>` shares a network namespace, the same model as a Kubernetes Pod, and is the go-to debugging technique.

Part 6 — Docker Compose in Depth

Running a real application with `docker run` means remembering dozens of flags across several containers. **Docker Compose** describes a multi-container application declaratively in YAML and manages it as one unit. It is the standard tool for local development environments, integration tests in CI, and small single-host deployments.

This course uses **Compose v2**: the `docker compose` CLI plugin (written in Go), following the **Compose Specification**. The old Python `docker-compose` v1 is end-of-life, and the top-level `version:` key is obsolete; Compose ignores it and prints a warning.

6.1 Core Concepts

- **Project**: the application, named after the directory by default (override with `-p` or the top-level `name:`). Every resource is prefixed or labeled with the project name.
- **Service**: a container definition (image, config, scale). One service can run several replicas.
- **Networks, volumes, configs, secrets**: top-level resources the services use.

Compose looks for `compose.yaml` (preferred), `compose.yml`, `docker-compose.yaml` or `docker-compose.yml` in the current directory.

By default Compose creates a network called `<project>_default` and attaches every service to it, so services reach each other by **service name** via Docker's embedded DNS.

6.2 A Complete Example

```yaml
# compose.yaml
name: shop

services:
  web:
    build:
      context: ./web
      target: runtime
    image: ghcr.io/acme/shop-web:${TAG:-dev}
    ports:
      - "127.0.0.1:8080:8080"
    environment:
      API_URL: http://api:8000
    depends_on:
      api:
        condition: service_healthy
    networks: [frontend]

  api:
```

```
    build:
      context: ./api
      args:
        PYTHON_VERSION: "3.12"
    image: ghcr.io/acme/shop-api:${TAG:-dev}
    env_file:
      - ./api/.env
    environment:
      DATABASE_URL: postgresql://shop:${DB_PASSWORD:?DB_PASSWORD must be set}@db:5432/shop
      REDIS_URL: redis://cache:6379/0
    depends_on:
      db:
        condition: service_healthy
      cache:
        condition: service_started
      migrate:
        condition: service_completed_successfully
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/healthz')"]
      interval: 10s
      timeout: 3s
      retries: 5
      start_period: 15s
    deploy:
      resources:
        limits:
          cpus: "1.0"
          memory: 512M
    restart: unless-stopped
    networks: [frontend, backend]

  migrate:
    image: ghcr.io/acme/shop-api:${TAG:-dev}
    command: ["python", "-m", "shop.migrate"]
    environment:
      DATABASE_URL: postgresql://shop:${DB_PASSWORD}@db:5432/shop
    depends_on:
      db:
        condition: service_healthy
    restart: "no"
    networks: [backend]

  db:
    image: postgres:16.4-alpine
    environment:
      POSTGRES_USER: shop
      POSTGRES_DB: shop
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets:
      - db_password
    volumes:
      - dbdata:/var/lib/postgresql/data
```

```
      - ./db/init:/docker-entrypoint-initdb.d:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U shop -d shop"]
      interval: 5s
      timeout: 3s
      retries: 10
    networks: [backend]

  cache:
    image: redis:7.4-alpine
    command: ["redis-server", "--appendonly", "yes"]
    volumes:
      - cachedata:/data
    networks: [backend]

  adminer:
    image: adminer:4
    ports: ["127.0.0.1:8081:8080"]
    profiles: [debug]
    networks: [backend]

networks:
  frontend:
  backend:
    internal: true

volumes:
  dbdata:
  cachedata:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

Bring it up:

```
echo "s3cr3t" > secrets/db_password.txt
export DB_PASSWORD=s3cr3t
docker compose up -d --build
docker compose ps
```

```
NAME             IMAGE                          SERVICE   STATUS                    PORTS
shop-api-1       ghcr.io/acme/shop-api:dev      api       Up 20 seconds (healthy)
shop-cache-1     redis:7.4-alpine               cache     Up 32 seconds             6379/tcp
shop-db-1        postgres:16.4-alpine           db        Up 32 seconds (healthy)   5432/tcp
shop-migrate-1   ghcr.io/acme/shop-api:dev      migrate   Exited (0) 22 seconds ago
shop-web-1       ghcr.io/acme/shop-web:dev      web       Up 8 seconds
127.0.0.1:8080->8080/tcp
```

Note the `backend` network is `internal: true`, and the `api` is on both networks. Since `api` must reach the internet (for example, a payment API), it gets egress via `frontend`.

6.3 Service Configuration Reference

6.3.1 build

```yaml
build:
  context: ./api
  dockerfile: Dockerfile.prod      # relative to context
  target: runtime
  args:
    APP_VERSION: ${APP_VERSION:-dev}
  secrets: [npmrc]
  ssh: [default]
  cache_from: ["type=registry,ref=ghcr.io/acme/api:buildcache"]
  cache_to: ["type=registry,ref=ghcr.io/acme/api:buildcache,mode=max"]
  platforms: ["linux/amd64", "linux/arm64"]
  labels:
    org.opencontainers.image.source: https://github.com/acme/shop
```

If both `build` and `image` are set, Compose builds and tags the result with `image`, so `docker compose push` can publish it.

6.3.2 depends_on conditions

Plain `depends_on: [db]` only controls **start order**; it does not wait for the database to be ready. Use the long form:

Condition	Waits until
`service_started`	The dependency container has started (default).
`service_healthy`	The dependency's healthcheck reports `healthy`.
`service_completed_successfully`	The dependency ran to completion with exit code 0 (migrations, seeders).

Additional keys: `restart: true` (restart this service when the dependency is restarted by a Compose operation) and `required: false` (do not fail if the dependency is not enabled, e.g. disabled by a profile).

> **Note:** `depends_on` only applies when Compose starts things. If the DB dies later, your app must still handle reconnects. Readiness gating at startup is not a substitute for retry logic.

6.3.3 healthcheck

```yaml
healthcheck:
  test: ["CMD", "curl", "-fsS", "http://localhost:8080/healthz"]
  interval: 30s
  timeout: 5s
  retries: 3
```

```
    start_period: 30s
    start_interval: 2s
```

`CMD` runs directly (exec), `CMD-SHELL` runs via `/bin/sh -c`. `disable: true` turns off an image-defined healthcheck. The tool you call (`curl`, `wget`) must exist in the image; for distroless images, ship a tiny health binary or build the check into the app.

6.3.4 Environment variables and interpolation

Two separate mechanisms that beginners confuse:

1. **Interpolation**: Compose substitutes `${VAR}` in the YAML itself, using the shell environment and the project `.env` file. This happens on the client, before anything is sent to Docker.
2. **Container environment**: `environment:` and `env_file:` set variables inside the container.

```
services:
  api:
    image: "acme/api:${TAG:-latest}"     # interpolation
    env_file:
      - path: ./api/.env                 # container env from file
        required: true
      - path: ./api/.env.local
        required: false
    environment:
      LOG_LEVEL: ${LOG_LEVEL:-info}      # interpolation feeding container env
      FEATURE_X: "true"
```

Interpolation syntax:

Syntax	Meaning
`${VAR}`	Value of VAR (empty if unset, with a warning).
`${VAR:-default}`	Default if unset or empty.
`${VAR-default}`	Default only if unset.
`${VAR:?error msg}`	Fail with message if unset or empty.
`${VAR:+replacement}`	Replacement if set and non-empty.
`$$`	A literal `$` (e.g. for shell commands in `command:`).

Precedence for a container variable (highest first): `docker compose run -e`, `environment:` (with interpolation from shell/`.env`), `env_file:`, then the image's `ENV`.

```
docker compose config           # show the fully resolved model; your best debugging tool
docker compose --env-file .env.staging config
```

> **Tip:** Commit a `.env.example` and add `.env` to `.gitignore`.

6.3.5 Ports, expose, networks

```yaml
ports:
  - "8080:80"
  - "127.0.0.1:5432:5432"
  - target: 443
    published: "8443"
    protocol: tcp
    host_ip: 0.0.0.0
expose:
  - "9090"           # documentation only; services on the same network can reach all ports anyway
networks:
  backend:
    aliases: [database]
    ipv4_address: 10.30.0.10
```

6.3.6 Volumes

```yaml
services:
  app:
    volumes:
      - dbdata:/var/lib/data                # named volume
      - ./config:/etc/app:ro                # bind mount
      - type: tmpfs
        target: /tmp
        tmpfs:
          size: 67108864
      - type: bind
        source: ./src
        target: /app/src
        bind:
          create_host_path: false
volumes:
  dbdata:
    driver: local
  legacy:
    external: true            # created outside Compose; never deleted by `down -v`
    name: prod_legacy_data
```

6.3.7 Resource limits, restart, security

```yaml
services:
  api:
    restart: unless-stopped
    deploy:
```

```
      replicas: 2
      resources:
        limits: { cpus: "1.5", memory: 768M, pids: 200 }
        reservations: { memory: 256M }
    read_only: true
    tmpfs: [/tmp]
    user: "10001:10001"
    cap_drop: [ALL]
    cap_add: [NET_BIND_SERVICE]
    security_opt:
      - no-new-privileges:true
    init: true
    stop_grace_period: 30s
    ulimits:
      nofile: { soft: 65536, hard: 65536 }
    logging:
      driver: local
      options: { max-size: "10m", max-file: "3" }
```

Compose v2 applies `deploy.resources` limits for plain `docker compose up` (not only Swarm).

6.3.8 Configs and secrets

```
services:
  nginx:
    image: nginx:1.27-alpine
    configs:
      - source: nginx_conf
        target: /etc/nginx/conf.d/default.conf
    secrets:
      - source: tls_key
        target: /run/secrets/tls.key
        mode: 0400
configs:
  nginx_conf:
    file: ./nginx/default.conf
  inline_cfg:
    content: |
      upstream api { server api:8000; }
secrets:
  tls_key:
    file: ./secrets/tls.key
  api_token:
    environment: API_TOKEN       # secret sourced from an env var on the host
```

Secrets are mounted under `/run/secrets/<name>`. Many official images (postgres, mysql, mariadb) support `*_FILE` variables to read secrets from files.

6.4 Profiles

Profiles let one Compose file describe optional services: debug tools, monitoring, seed jobs.

```yaml
services:
  adminer:
    image: adminer:4
    profiles: [debug]
  prometheus:
    image: prom/prometheus:v2.54.1
    profiles: [monitoring]
  loadtest:
    image: grafana/k6:0.54.0
    profiles: [test]
```

```
docker compose up -d                              # only services without a profile
docker compose --profile debug up -d              # plus adminer
COMPOSE_PROFILES=debug,monitoring docker compose up -d
docker compose run --rm loadtest run /scripts/smoke.js   # explicitly running a service
enables its profile
```

6.5 Multiple Files: Overrides, extends, include

6.5.1 Override files

Compose automatically merges `compose.override.yaml` on top of `compose.yaml`. The convention:

- `compose.yaml`: the canonical, production-like definition.
- `compose.override.yaml`: local dev tweaks (bind mounts, debug ports, dev target). Applied automatically.
- `compose.prod.yaml`, `compose.ci.yaml`: applied explicitly with `-f`.

```yaml
# compose.override.yaml (dev)
services:
  api:
    build:
      target: dev
    volumes:
      - ./api/src:/app/src
    environment:
      LOG_LEVEL: debug
    ports:
      - "127.0.0.1:5678:5678"   # debugger
```

```
docker compose up -d                                      # base + override
docker compose -f compose.yaml -f compose.prod.yaml up -d  # base + prod (override NOT
```

```
applied)
docker compose -f compose.yaml -f compose.prod.yaml config > resolved.yaml
```

Merge rules: scalar values are replaced; `ports`, `expose`, `dns` and similar sequences are concatenated; `environment`, `labels`, `volumes` (by target path) are merged by key. Use `!reset` to clear an inherited value and `!override` to replace a sequence entirely:

```yaml
services:
  api:
    ports: !override
      - "9000:8000"
    healthcheck: !reset null
```

6.5.2 extends and include

```yaml
# compose.yaml
include:
  - path: ./observability/compose.yaml     # pull in another project's services
services:
  worker:
    extends:
      file: common.yaml
      service: python-base
    command: ["celery", "-A", "shop", "worker"]
```

`include` imports a whole Compose file (with its own relative paths resolved correctly), which is great for splitting a platform into team-owned pieces.

6.6 Day-to-Day Commands

```
docker compose up -d                  # create/start, recreating changed services
docker compose up -d --build api      # rebuild and restart one service
docker compose up -d --wait           # block until services are running/healthy
docker compose up -d --scale worker=4
docker compose ps
docker compose logs -f --tail 50 api
docker compose exec api sh
docker compose run --rm api pytest    # one-off container with service config
docker compose stop / start / restart
docker compose pull                   # pull newer images
docker compose build --pull
docker compose push
docker compose top
docker compose events
docker compose cp api:/app/report.html .
docker compose down                   # stop and remove containers + networks
docker compose down -v                # also remove named volumes (data loss!)
```

```
docker compose down --rmi local       # also remove locally built images
docker compose ls                     # list running projects
```

`up` is idempotent: Compose computes a config hash per service and recreates only containers whose configuration or image changed.

6.6.1 Compose Watch for development

`develop.watch` syncs files or rebuilds automatically, replacing hand-rolled bind-mount hacks:

```yaml
services:
  web:
    build: .
    develop:
      watch:
        - action: sync
          path: ./src
          target: /app/src
          ignore: [node_modules/]
        - action: rebuild
          path: package.json
        - action: sync+restart
          path: ./config
          target: /app/config
```

```
docker compose watch
# or: docker compose up --watch
```

6.7 Compose in CI and Production

CI integration tests:

```
docker compose -f compose.yaml -f compose.ci.yaml up -d --wait
docker compose run --rm api pytest -q
status=$?
docker compose logs --no-color > compose-logs.txt
docker compose down -v
exit $status
```

Use a unique project name per CI job (`-p ci-$CI_JOB_ID`) so parallel jobs on one runner do not collide.

Single-host production: Compose is acceptable for small, single-host deployments if you add: pinned image tags or digests, `restart: unless-stopped`, healthchecks, resource limits, log rotation, secrets via files, a reverse proxy with TLS, and backups. It does not give you multi-host scheduling, rolling updates with zero downtime, or self-healing across nodes. For those, use Swarm or (much more commonly) Kubernetes.

Deploying to a remote host:

```
docker context create prod --docker "host=ssh://deploy@prod-1"
docker --context prod compose -f compose.yaml -f compose.prod.yaml pull
docker --context prod compose -f compose.yaml -f compose.prod.yaml up -d --remove-orphans
```

6.8 Lab: Build a Full Dev Environment With Compose

Goal: a web API, Postgres and Redis with healthchecks, a migration job, a debug profile and a dev override.

1. Project layout:

```
compose-lab/
  compose.yaml
  compose.override.yaml
  .env
  api/
    Dockerfile
    requirements.txt
    app.py
  secrets/db_password.txt
```

2. `api/app.py` (Flask app that counts hits in Redis and checks Postgres):

```python
import os
import psycopg
import redis
from flask import Flask

app = Flask(__name__)
r = redis.Redis.from_url(os.environ["REDIS_URL"])

@app.get("/healthz")
def healthz():
    return {"ok": True}

@app.get("/")
def index():
    hits = r.incr("hits")
    with psycopg.connect(os.environ["DATABASE_URL"]) as conn:
        version = conn.execute("select version()").fetchone()[0]
    return {"hits": hits, "db": version.split(",")[0]}
```

`api/requirements.txt`:

```
flask==3.0.3
gunicorn==23.0.0
redis==5.0.8
psycopg[binary]==3.2.3
```

Use the multi-stage Python Dockerfile from Part 3, with `CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]` and a `dev` stage that runs `flask --app app run --debug --host 0.0.0.0 --port 8000`.

3. Write `compose.yaml` with services `api`, `db`, `cache`, `adminer` (profile `debug`), using the patterns in 6.2: healthchecks on `db` and `api`, `depends_on` with `service_healthy`, a secret for the DB password, a named volume for Postgres. Publish the API as `"127.0.0.1:8080:8000"`, and set `DATABASE_URL: postgresql://postgres:${DB_PASSWORD}@db:5432/postgres` and `REDIS_URL: redis://cache:6379/0` in its environment. Keep this lab on the single default network to keep it simple.

4. `.env`:

```
TAG=dev
DB_PASSWORD=labpass
```

5. Validate and start:

```
docker compose config --quiet && echo "config OK"
docker compose up -d --build --wait
curl -s localhost:8080/ ; curl -s localhost:8080/
```

```
{"db":"PostgreSQL 16.4","hits":1}
{"db":"PostgreSQL 16.4","hits":2}
```

6. Add `compose.override.yaml` that targets the `dev` stage and adds a `develop.watch` sync of `./api` to `/app`. Run `docker compose watch`, edit `app.py`, and see the reload.

7. Start the debug profile: `docker compose --profile debug up -d` and log in to Adminer at `localhost:8081` (server `db`).

8. Kill the database and watch the API's health and logs: `docker compose kill db`, `docker compose ps`, `docker compose logs api`. Then `docker compose up -d` to restore.

9. Tear down: `docker compose --profile debug down -v`.

Key takeaways

- Use `docker compose` (v2) and `compose.yaml` without a `version:` key.
- Services discover each other by service name on the project network; split networks for isolation.
- `depends_on` with `service_healthy` and `service_completed_successfully` fixes startup ordering, but apps must still retry.
- Distinguish YAML interpolation (`.env`, shell) from container environment (`environment`, `env_file`); `docker compose config` shows the truth.
- Profiles add optional services; override files and `-f` layering adapt one definition to dev, CI and prod.
- `docker compose watch` gives fast inner-loop development; `up --wait` makes CI deterministic.

Part 7 — Runtime Internals: cgroups, Namespaces, Logging, Monitoring and Filesystems

Senior engineers are expected to explain *how* containers work, not just how to run them. That knowledge pays off in troubleshooting: an OOMKilled container, a CPU-throttled service, a host whose disk filled with logs, a "file not found" that exists. This Part covers resource limits and cgroups, namespaces, logging drivers, monitoring, overlayfs, and finally builds a container by hand.

7.1 Resource Limits and cgroups

7.1.1 What cgroups do

Control groups organize processes into a hierarchy and apply resource controllers to each group: `cpu`, `memory`, `io`, `pids`, `cpuset`, `hugetlb`. Modern distributions use **cgroup v2**, a unified hierarchy at `/sys/fs/cgroup`. With the systemd cgroup driver, each container gets a scope like:

```
/sys/fs/cgroup/system.slice/docker-<container-id>.scope/
    cpu.max            memory.max        pids.max
    cpu.stat           memory.current    memory.events
    io.max             memory.high       cgroup.procs
```

Without limits, a container can consume all host CPU and memory. Always set limits in production.

7.1.2 Memory

```
docker run -d --name api --memory 512m --memory-swap 512m myorg/api
docker run -d --memory 1g --memory-reservation 768m myorg/api
```

Flag	Effect
`--memory` (`-m`)	Hard limit (`memory.max`). Exceeding it triggers the kernel OOM killer inside the cgroup.
`--memory-swap`	Memory + swap total. Equal to `--memory` means no swap. `-1` means unlimited swap.
`--memory-reservation`	Soft limit (`memory.low` / `high` behavior) applied under host memory pressure.
`--oom-kill-disable`	Do not OOM-kill; dangerous without a hard limit. Avoid.

When a container exceeds its limit, the kernel kills a process in the cgroup (usually the biggest, typically your app):

```
docker inspect api --format '{{.State.OOMKilled}} {{.State.ExitCode}}'
```

```
true 137
```

```
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' api).scope/memory.events
```

```
low 0
high 0
max 2314
oom 3
oom_kill 3
```

> **Warning:** Language runtimes must know their limit. Modern JVMs (17+) respect cgroup limits automatically (tune with `-XX:MaxRAMPercentage=75`). Node.js sizes its heap from available memory in recent versions but `--max-old-space-size` is still common. Go 1.19+ honors `GOMEMLIMIT`. Python does not manage this at all; watch worker counts.

Note: page cache counts toward the cgroup's memory usage but is reclaimable. `docker stats` subtracts inactive file cache; tools reading raw `memory.current` may show higher numbers.

7.1.3 CPU

```
docker run -d --cpus 1.5 myorg/api            # quota: 150ms per 100ms period
docker run -d --cpu-shares 512 myorg/batch    # relative weight under contention (default 1024)
docker run -d --cpuset-cpus "0,1" myorg/latency-sensitive
```

Flag	cgroup v2 file	Semantics
`--cpus`	`cpu.max` (`150000 100000`)	Hard cap; the process is throttled once it uses its quota in the period.
`--cpu-shares`	`cpu.weight`	Proportional share only when CPUs are contended.
`--cpuset-cpus`	`cpuset.cpus`	Pin to specific cores.

Throttling is a silent latency killer. A multi-threaded app with `--cpus 1` can burn its 100 ms quota in 25 ms on 4 cores and then sit idle for 75 ms. Check:

```
cat /sys/fs/cgroup/system.slice/docker-<id>.scope/cpu.stat
```

```
usage_usec 98234123
user_usec 80211022
system_usec 18023101
nr_periods 51234
nr_throttled 8721
throttled_usec 412938112
```

A high `nr_throttled / nr_periods` ratio means raise the limit or reduce the app's thread pool.

7.1.4 PIDs, I/O and ulimits

```
docker run -d --pids-limit 200 myorg/api                  # fork-bomb protection
docker run -d --device-write-bps /dev/sda:20mb myorg/etl  # I/O throttle
docker run -d --ulimit nofile=65536:65536 myorg/proxy
```

7.1.5 Changing limits at runtime

```
docker update --memory 1g --memory-swap 1g --cpus 2 api
docker stats --no-stream api
```

```
CONTAINER ID   NAME   CPU %    MEM USAGE / LIMIT   MEM %    NET I/O         BLOCK I/O
PIDS
4f2a1b9c8d7e   api    12.31%   312MiB / 1GiB       30.47%   1.2MB / 890kB   0B / 4.1MB    23
```

7.2 Namespaces

Namespace	Isolates	Docker flag to share with host
PID	Process IDs	`--pid host`
NET	Interfaces, routes, ports, iptables	`--network host`
MNT	Mount points	(bind mounts selectively)
UTS	Hostname, domain name	`--uts host`
IPC	SysV IPC, POSIX message queues, shared memory	`--ipc host`
USER	UID/GID mapping	`--userns host` (when userns-remap is on)
CGROUP	View of cgroup hierarchy	`--cgroupns host`
TIME	System clocks (offsets)	Not exposed by Docker by default

Inspect a container's namespaces from the host:

```
PID=$(docker inspect -f '{{.State.Pid}}' api)
sudo ls -l /proc/$PID/ns
sudo nsenter -t "$PID" -n ip addr      # enter only the network namespace
sudo nsenter -t "$PID" -m -u -i -n -p sh   # enter almost everything (like docker exec)
lsns -p "$PID"
```

`nsenter` is how you debug a container that has no shell: you use the host's tools inside the container's namespaces.

User namespaces remap UIDs: root (UID 0) inside the container maps to an unprivileged UID (e.g. 100000) outside. Enable host-wide with `"userns-remap": "default"` in `daemon.json`, or use rootless Docker. This is one of the strongest hardening measures, because even a container breakout yields an unprivileged user.

7.3 Logging

7.3.1 How container logging works

The container's stdout and stderr are captured by the shim and passed to the configured **logging driver**. Applications should write logs to stdout/stderr (twelve-factor), ideally as structured JSON.

Driver	Where logs go	`docker logs` works
`json-file`	JSON lines under `/var/lib/docker/containers/<id>/`	Yes
`local`	Compressed, rotated files; more efficient	Yes
`journald`	systemd journal	Yes
`syslog`	Syslog server	Via dual logging
`fluentd`	Fluentd / Fluent Bit forwarder	Via dual logging
`awslogs`	Amazon CloudWatch Logs	Via dual logging
`gcplogs`	Google Cloud Logging	Via dual logging
`splunk`, `gelf`	Splunk HEC, Graylog	Via dual logging
`none`	Discarded	No

"Dual logging" (Engine 20.10+) keeps a local cache so `docker logs` works even with remote drivers.

7.3.2 Configuration

Daemon-wide in `/etc/docker/daemon.json`:

```
{
  "log-driver": "local",
```

```
  "log-opts": { "max-size": "20m", "max-file": "5" }
}
```

Per container:

```
docker run -d --log-driver json-file --log-opt max-size=10m --log-opt max-file=3 nginx
docker run -d --log-driver awslogs \
  --log-opt awslogs-region=eu-west-1 \
  --log-opt awslogs-group=/prod/api \
  --log-opt awslogs-create-group=true myorg/api
docker run -d --log-driver fluentd --log-opt fluentd-address=127.0.0.1:24224 \
  --log-opt tag="docker.{{.Name}}" myorg/api
```

> **Warning:** Daemon-wide `log-opts` changes apply only to containers created **after** the change. Existing containers keep their old config until recreated.

7.3.3 Delivery mode

`--log-opt mode=non-blocking --log-opt max-buffer-size=4m` makes logging asynchronous. In the default `blocking` mode, if the log destination is slow (e.g. a struggling Fluentd), your app blocks on writes to stdout. Non-blocking trades potential log loss for app availability.

7.3.4 Architecture for production

A common pattern: containers log JSON to stdout -> `local` or `json-file` driver -> a node agent (Fluent Bit, Vector, Grafana Alloy, Datadog Agent) tails the files -> central store (Loki, Elasticsearch/OpenSearch, CloudWatch, Splunk). The agent enriches logs with container labels.

7.4 Monitoring

7.4.1 Built-in tools

```
docker stats
docker system df
docker events --filter event=oom --filter event=die
docker inspect --format '{{json .State.Health}}' api | jq
```

7.4.2 cAdvisor, Prometheus and Grafana

cAdvisor exports per-container metrics (CPU, memory, network, filesystem, throttling) in Prometheus format.

```
# compose.monitoring.yaml
services:
```

```yaml
  cadvisor:
    image: gcr.io/cadvisor/cadvisor:v0.49.1
    privileged: true
    devices: ["/dev/kmsg"]
    volumes:
      - /:/rootfs:ro
      - /var/run:/var/run:ro
      - /sys:/sys:ro
      - /var/lib/docker/:/var/lib/docker:ro
    ports: ["127.0.0.1:8088:8080"]

  node-exporter:
    image: prom/node-exporter:v1.8.2
    pid: host
    volumes: ["/:/host:ro,rslave"]
    command: ["--path.rootfs=/host"]

  prometheus:
    image: prom/prometheus:v2.54.1
    volumes: ["./prometheus.yml:/etc/prometheus/prometheus.yml:ro"]
    ports: ["127.0.0.1:9090:9090"]

  grafana:
    image: grafana/grafana:11.2.0
    ports: ["127.0.0.1:3000:3000"]
```

```yaml
# prometheus.yml
scrape_configs:
  - job_name: cadvisor
    static_configs: [{ targets: ["cadvisor:8080"] }]
  - job_name: node
    static_configs: [{ targets: ["node-exporter:9100"] }]
```

Useful PromQL:

```
# CPU cores used per container
sum by (name) (rate(container_cpu_usage_seconds_total{name!=""}[5m]))

# Memory working set vs limit
container_memory_working_set_bytes{name!=""} / container_spec_memory_limit_bytes{name!=""}

# CPU throttling ratio
rate(container_cpu_cfs_throttled_periods_total[5m]) /
rate(container_cpu_cfs_periods_total[5m])
```

The Docker daemon itself can expose metrics with `"metrics-addr": "127.0.0.1:9323"` in `daemon.json`.

7.4.3 What to alert on

- Container restarts and OOM kills (events, `container_oom_events_total`).

- Memory working set above 90 percent of limit.
- CPU throttling ratio above about 25 percent sustained.
- Unhealthy containers.
- Host disk usage of `/var/lib/docker` above 80 percent.

7.5 Storage Drivers and overlayfs

7.5.1 How overlayfs assembles a container root

```
          Container view (merged)  /
                 ^
                 |  overlay mount
   +-------------+---------------------------------+
   | upperdir    | container writable layer (rw)   |
   +-------------+---------------------------------+
   | lowerdir 3  | image layer: app files   (ro)   |
   | lowerdir 2  | image layer: packages    (ro)   |
   | lowerdir 1  | image layer: base OS     (ro)   |
   +-------------+---------------------------------+
     workdir: scratch space used by overlayfs
```

- **Reads** search from the top layer down; the first match wins.
- **Writes** to an existing lower file trigger **copy-up**: the whole file is copied to `upperdir`, then modified.
- **Deletes** create a **whiteout** file (a character device 0/0) in `upperdir` that hides the lower file.

See it for a running container (classic overlay2 driver):

```
docker inspect api --format '{{json .GraphDriver.Data}}' | jq
mount | grep overlay | head -1
```

```
{
  "LowerDir": "/var/lib/docker/overlay2/7c1e.../diff:/var/lib/docker/overlay2/a9f3.../diff",
  "MergedDir": "/var/lib/docker/overlay2/3be2.../merged",
  "UpperDir": "/var/lib/docker/overlay2/3be2.../diff",
  "WorkDir": "/var/lib/docker/overlay2/3be2.../work"
}
```

With the containerd image store, the snapshots live under `/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/` instead, but the mechanism is the same.

7.6 Building a Container by Hand

The best way to demystify containers is to build one without Docker. Run this on a Linux VM as root (not on your production host).

7.6.1 Step 1: get a root filesystem

```
mkdir -p /tmp/mycontainer/rootfs && cd /tmp/mycontainer
# Export Alpine's filesystem using Docker (or download alpine-minirootfs tarball)
docker export "$(docker create alpine:3.20)" | tar -C rootfs -xf -
ls rootfs
```

```
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

7.6.2 Step 2: chroot only (filesystem isolation)

```
chroot rootfs /bin/sh
ps aux          # empty or errors: /proc is not mounted
mount -t proc proc /proc
ps aux          # you now see ALL host processes: no PID isolation
exit
umount rootfs/proc
```

`chroot` changes the apparent root, but the process still shares every namespace with the host.

7.6.3 Step 3: add namespaces with unshare

```
unshare --pid --fork --mount --uts --ipc --net --mount-proc=rootfs/proc \
  chroot rootfs /bin/sh
```

Inside:

```
hostname mycontainer && hostname   # UTS namespace: does not affect host
ps aux                             # PID 1 is /bin/sh: PID namespace works
ip addr                            # only 'lo', down: empty NET namespace
```

```
PID   USER     TIME  COMMAND
    1 root      0:00 /bin/sh
    3 root      0:00 ps aux
```

(Use `pivot_root` instead of `chroot` for real isolation; runc does that, because chroot can be escaped by a privileged process.)

7.6.4 Step 4: add a cgroup limit

In another host terminal:

```
mkdir /sys/fs/cgroup/mycontainer
echo "50M" > /sys/fs/cgroup/mycontainer/memory.max
echo "50000 100000" > /sys/fs/cgroup/mycontainer/cpu.max   # half a CPU
# find the PID of the containerized sh (the forked child of unshare) and move it in
CPID=$(pgrep -P "$(pgrep -xf 'unshare --pid.*')")
echo "$CPID" > /sys/fs/cgroup/mycontainer/cgroup.procs
```

Inside the container, try to allocate more than 50 MB (for example `head -c 100m /dev/zero | tail`) and it gets OOM-killed.

7.6.5 Step 5: give it a network

```
# Host side
ip link add veth-host type veth peer name veth-ctr
ip link set veth-ctr netns "$CPID"
ip addr add 10.99.0.1/24 dev veth-host && ip link set veth-host up
# Container side (via nsenter)
nsenter -t "$CPID" -n ip addr add 10.99.0.2/24 dev veth-ctr
nsenter -t "$CPID" -n ip link set veth-ctr up
nsenter -t "$CPID" -n ip link set lo up
ping -c1 10.99.0.2
```

That is, in essence, what Docker's bridge driver does, plus a bridge, NAT and iptables rules.

7.6.6 Step 6: an overlay root

```
mkdir -p /tmp/ov/{lower,upper,work,merged}
cp -a /tmp/mycontainer/rootfs/. /tmp/ov/lower/
mount -t overlay overlay -o
lowerdir=/tmp/ov/lower,upperdir=/tmp/ov/upper,workdir=/tmp/ov/work /tmp/ov/merged
echo hi > /tmp/ov/merged/etc/motd
rm /tmp/ov/merged/etc/hostname
ls -la /tmp/ov/upper/etc
```

```
c---------  1 root root 0, 0 Oct 12 10:30 hostname
-rw-r--r--  1 root root    3 Oct 12 10:30 motd
```

You see the copied-up `motd` and the whiteout for `hostname`; `lower` is untouched.

What runc adds beyond this: pivot_root, a minimal `/dev`, dropping capabilities, seccomp filters, AppArmor/SELinux labels, user namespace mapping, masked `/proc` paths, and lifecycle hooks, all driven by the OCI `config.json`. You can generate one with `runc spec`.

7.7 Lab: Limits, OOM, Throttling and Logs

Goal: observe cgroup enforcement and logging behavior directly.

1. Memory limit and OOM:

```
docker run --name oomtest --memory 64m --memory-swap 64m python:3.12-alpine \
  python -c "x = bytearray(200*1024*1024)"
docker inspect oomtest --format 'OOMKilled={{.State.OOMKilled}} Exit={{.State.ExitCode}}'
docker events --since 5m --until "$(date +%s)" --filter event=oom
```

```
OOMKilled=true Exit=137
```

2. CPU throttling:

```
docker run -d --name burner --cpus 0.5 alpine:3.20 sh -c 'while :; do :; done'
docker stats --no-stream burner     # about 50% CPU
CG=/sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' burner).scope
cat $CG/cpu.max; grep throttled $CG/cpu.stat
docker update --cpus 1 burner && sleep 3 && docker stats --no-stream burner
```

3. PID limit:

```
docker run --rm --pids-limit 20 alpine:3.20 sh -c 'for i in $(seq 50); do sleep 30 & done;
wait'
```

Expect `can't fork: Resource temporarily unavailable`.

4. Logging rotation: run a chatty container with `--log-opt max-size=1m --log-opt max-file=2` using the `json-file` driver, then inspect `/var/lib/docker/containers/<id>/` and confirm at most two files of about 1 MB.

```
docker run -d --name chatty --log-driver json-file --log-opt max-size=1m --log-opt max-file=2
\
  alpine:3.20 sh -c 'while :; do echo "$(date) some log line padding padding padding"; done'
sleep 10; sudo ls -lh /var/lib/docker/containers/$(docker inspect -f '{{.Id}}' chatty)/
```

5. Work through section 7.6 (by-hand container) on a scratch VM.

6. Clean up: `docker rm -f oomtest burner chatty`.

Key takeaways

- cgroups enforce limits: `--memory` triggers the OOM killer (exit 137, `OOMKilled=true`), `--cpus` throttles via `cpu.max`.

- CPU throttling hurts latency invisibly; watch `nr_throttled` in `cpu.stat` or cAdvisor's throttling metrics.
- Namespaces isolate PID, NET, MNT, UTS, IPC, USER and CGROUP; `nsenter` lets host tools work inside a container's namespaces.
- Log to stdout, use the `local` driver or configure rotation, and ship logs with a node agent.
- overlayfs stacks read-only image layers under a writable upper layer with copy-up and whiteouts.
- A container is chroot/pivot_root plus namespaces plus cgroups plus security filters; you can build one with `unshare`.

Part 8 — Container Security

Containers are not a security boundary by default, but they can be made into a strong one with layered controls. This Part follows the container lifecycle: secure the **host and daemon**, secure the **image** (minimal, scanned, signed, with an SBOM), secure the **runtime** (non-root, capabilities, seccomp, AppArmor/SELinux, read-only filesystem, limits), and handle **secrets** correctly.



8.1 Threat Model in One Table

Threat	Example	Primary controls
Vulnerable packages	Old OpenSSL in base image	Minimal bases, scanning, regular rebuilds
Malicious image	Typosquatted image on Docker Hub	Trusted sources, signing and verification
Secret leakage	API key in a layer or `ENV`	Secret mounts, runtime secrets, scanning
Container breakout	Kernel exploit, privileged container	Non-root, userns/rootless, seccomp, capabilities, patching
Lateral movement	Compromised web container reaching the DB	Network segmentation, internal networks
Resource exhaustion	Fork bomb, memory leak	cgroup limits, pids limit
Daemon takeover	Exposed Docker socket or TCP API	Never expose socket, TLS for remote API, rootless

8.2 Securing the Host and Daemon

- Keep the kernel, Docker Engine, containerd and runc patched. Several historical breakouts were runc or kernel bugs (for example CVE-2019-5736, CVE-2024-21626 “Leaky Vessels”).
- **Never** expose the Docker API on TCP without mutual TLS (`tcp://0.0.0.0:2375` is a classic way to get cryptominers). Prefer SSH contexts.
- Do not mount `/var/run/docker.sock` into containers unless you fully trust them; it is root on the host. If a tool needs it, consider a socket proxy that allows only read endpoints.

- Restrict the `docker` group. Consider **rootless mode** or **userns-remap**.
- Run the **CIS Docker Benchmark** checks:

```
docker run --rm --net host --pid host --userns host --cap-add audit_control \
  -v /etc:/etc:ro -v /var/lib:/var/lib:ro -v /var/run/docker.sock:/var/run/docker.sock:ro \
  -v /usr/lib/systemd:/usr/lib/systemd:ro \
  docker/docker-bench-security
```

```
[WARN] 2.2  - Ensure network traffic is restricted between containers on the default bridge
[PASS] 2.14 - Ensure Userland Proxy is Disabled
[WARN] 4.1  - Ensure that a user for the container has been created
[WARN] 5.10 - Ensure that the memory usage for containers is limited
...
```

Hardened daemon settings:

```
{
  "icc": false,
  "no-new-privileges": true,
  "live-restore": true,
  "userland-proxy": false,
  "userns-remap": "default",
  "log-driver": "local",
  "log-opts": { "max-size": "10m", "max-file": "3" }
}
```

`"icc": false` disables inter-container communication on the default bridge (user-defined networks are controlled by network membership).

8.3 Securing Images

8.3.1 Minimal, pinned, maintained bases

- Use official, verified, or hardened images (Docker Official Images, Docker Hardened Images, Chainguard, distroless).
- Pin versions and ideally digests: `FROM python:3.12.7-slim-bookworm@sha256:...`. Use Renovate or Dependabot to bump them automatically so pinning does not mean stagnating.
- Rebuild regularly (weekly, or on base image update) to pick up patches even when your code has not changed.

8.3.2 Never bake secrets

```
# BAD: visible in docker history and in the layer forever
ARG GITHUB_TOKEN
ENV API_KEY=abc123
```

```
COPY id_rsa /root/.ssh/id_rsa

# GOOD: available only during this RUN
RUN --mount=type=secret,id=github_token \
    GITHUB_TOKEN=$(cat /run/secrets/github_token) ./fetch-private-deps.sh
```

Prove a leak to yourself:

```
docker history --no-trunc badimage | grep -i token
docker save badimage | tar -xO --wildcards '*/layer.tar' 2>/dev/null | strings | grep -i
'BEGIN OPENSSH'
```

Scan for secrets in images and repositories with Trivy (`--scanners secret`), gitleaks or trufflehog.

8.3.3 Vulnerability scanning with Trivy

Trivy (Aqua Security, open source) scans images, filesystems, repositories, IaC and Kubernetes for CVEs, misconfigurations, secrets and licenses.

```
trivy image nginx:1.27-alpine
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 ghcr.io/acme/api:1.4.2
trivy image --scanners vuln,secret,misconfig ghcr.io/acme/api:1.4.2
trivy config Dockerfile                # Dockerfile misconfigurations
trivy fs --scanners vuln .             # scan lockfiles in the repo
trivy image --format sarif -o trivy.sarif ghcr.io/acme/api:1.4.2
```

```
ghcr.io/acme/api:1.4.2 (debian 12.7)
Total: 3 (HIGH: 2, CRITICAL: 1)

+-----------+----------------+----------+-------------------+---------------+
|  Library  | Vulnerability  | Severity | Installed Version | Fixed Version |
+-----------+----------------+----------+-------------------+---------------+
| libssl3   | CVE-2024-XXXX  | CRITICAL | 3.0.14-1~deb12u1  | 3.0.14-1~deb12u2 |
...
```

Key flags: `--exit-code 1` fails CI, `--ignore-unfixed` reduces noise from CVEs with no fix available, `.trivyignore` documents accepted risks (with expiry and justification).

8.3.4 Docker Scout

Docker Scout is built into the Docker CLI and Docker Hub.

```
docker scout quickview ghcr.io/acme/api:1.4.2
docker scout cves --only-severity critical,high ghcr.io/acme/api:1.4.2
docker scout recommendations ghcr.io/acme/api:1.4.2   # suggests better base image tags
docker scout compare ghcr.io/acme/api:1.4.2 --to ghcr.io/acme/api:1.4.1
```

`recommendations` is particularly useful: it tells you which base image update removes the most CVEs.

> **Tip:** Scanning is a point-in-time result. New CVEs are published daily against images you already shipped. Re-scan images in your registry continuously (ECR enhanced scanning, Harbor scheduled scans, Scout policies), not only at build time.

8.3.5 SBOMs

A **Software Bill of Materials** lists every package in an image. When the next Log4Shell happens, you query SBOMs instead of scanning thousands of images in a panic.

```
# Generate with BuildKit at build time (attached as an attestation)
docker buildx build --sbom=true --provenance=mode=max -t ghcr.io/acme/api:1.4.2 --push .

# Generate with Syft (SPDX or CycloneDX)
syft ghcr.io/acme/api:1.4.2 -o spdx-json > sbom.spdx.json
syft ghcr.io/acme/api:1.4.2 -o cyclonedx-json > sbom.cdx.json

# Scan an SBOM instead of the image
grype sbom:sbom.cdx.json
trivy sbom sbom.cdx.json

# Inspect attestations on a pushed image
docker buildx imagetools inspect ghcr.io/acme/api:1.4.2 --format '{{json .SBOM}}' | jq
'.SPDX.packages | length'
```

Provenance attestations (SLSA) record how, where and from what source an image was built, which lets consumers verify it came from your CI.

8.3.6 Signing and verification with cosign

Sigstore's **cosign** signs images and stores signatures in the registry next to the image.

Keyless signing (recommended in CI with OIDC, e.g. GitHub Actions):

```
# In CI with id-token permission; identity comes from the OIDC token
cosign sign --yes ghcr.io/acme/api@sha256:5e1f...
```

Verification pins the expected identity and issuer:

```
cosign verify ghcr.io/acme/api@sha256:5e1f... \
  --certificate-identity-regexp
'https://github.com/acme/api/.github/workflows/release.yml@refs/tags/v.*' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com
```

Key-based signing (air-gapped or KMS-backed):

```
cosign generate-key-pair                        # or --kms awskms:///alias/cosign
cosign sign --key cosign.key ghcr.io/acme/api@sha256:5e1f...
cosign verify --key cosign.pub ghcr.io/acme/api@sha256:5e1f...
cosign attest --key cosign.key --type cyclonedx --predicate sbom.cdx.json
ghcr.io/acme/api@sha256:5e1f...
```

Always sign **digests**, not tags. Enforce verification at deploy time: Kubernetes admission controllers (Sigstore policy-controller, Kyverno, Connaisseur), Harbor's content trust policies, or a deploy script that runs `cosign verify` before rollout.

> **Note:** Docker Content Trust (Notary v1, `DOCKER_CONTENT_TRUST=1`) is legacy. The ecosystem has moved to Sigstore/cosign and Notary Project (notation) signatures stored as OCI artifacts.

8.4 Securing the Runtime

8.4.1 Run as non-root

Root in a container is real root on the host kernel (without user namespaces), restricted only by capabilities and filters. A breakout from a root container is far worse.

```
RUN groupadd -r -g 10001 app && useradd -r -u 10001 -g app -s /sbin/nologin app
USER 10001:10001
```

```
docker run --user 10001:10001 myorg/api
docker run --rm myorg/api id
```

```
uid=10001(app) gid=10001(app) groups=10001(app)
```

Non-root processes cannot bind ports below 1024 without `CAP_NET_BIND_SERVICE`; listen on 8080 and map `-p 80:8080` instead.

8.4.2 Linux capabilities

Root's power is split into about 40 capabilities. Docker grants a default subset (e.g. `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `SETUID`, `SETGID`, `NET_BIND_SERVICE`, `NET_RAW`, `KILL`, `SYS_CHROOT`, `MKNOD`, `AUDIT_WRITE`, `SETFCAP`, `SETPCAP`, `FSETID`). Dangerous ones like `SYS_ADMIN`, `NET_ADMIN`, `SYS_PTRACE`, `SYS_MODULE` are not granted.

Best practice: drop all, add back only what is needed.

```
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE nginx-custom
docker run --rm --cap-drop ALL alpine:3.20 sh -c 'grep Cap /proc/self/status'
```

```
CapInh: 0000000000000000
CapPrm: 0000000000000000
CapEff: 0000000000000000
CapBnd: 0000000000000000
CapAmb: 0000000000000000
```

Decode with `capsh --decode=00000000a80425fb`.

8.4.3 --privileged: avoid it

`--privileged` grants all capabilities, all devices, disables seccomp and AppArmor, and makes `/sys` writable. It is effectively no isolation. If a tool claims to need it, find the specific capability or device: `--cap-add SYS_ADMIN` (still dangerous), `--device /dev/fuse`, etc.

8.4.4 no-new-privileges

```
docker run --security-opt no-new-privileges:true myorg/api
```

Prevents processes from gaining privileges through setuid/setgid binaries or file capabilities (for example, a `sudo` binary in the image becomes useless to an attacker).

8.4.5 seccomp

seccomp filters which **system calls** a process may make. Docker's default profile blocks around 40-60 dangerous syscalls (`kexec_load`, `mount` without `CAP_SYS_ADMIN`, `keyctl`, `bpf`, `ptrace` on older kernels, and so on) while allowing what nearly all apps need.

```
docker info --format '{{.SecurityOptions}}'       # confirm seccomp is active
docker run --security-opt seccomp=./custom.json myorg/api
docker run --security-opt seccomp=unconfined myorg/api   # disables it: debugging only
```

A custom profile can be tighter. Tools such as the Security Profiles Operator (Kubernetes) or `strace`-based recording can generate an allowlist of the syscalls your app actually uses.

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "architectures": ["SCMP_ARCH_X86_64", "SCMP_ARCH_AARCH64"],
  "syscalls": [
    { "names": ["read", "write", "openat", "close", "fstat", "mmap", "futex",
                "epoll_wait", "accept4", "socket", "bind", "listen", "exit_group"],
      "action": "SCMP_ACT_ALLOW" }
  ]
}
```

8.4.6 AppArmor and SELinux

Mandatory access control limits what files, capabilities and network operations a process can use, independent of UID.

- **AppArmor** (Ubuntu, Debian, SUSE): Docker applies the `docker-default` profile automatically. Load a custom one with `apparmor_parser -r -W myprofile` and run with `--security-opt apparmor=myprofile`.
- **SELinux** (RHEL, Fedora, CentOS): enable with `"selinux-enabled": true`. Containers run as `container_t` with unique MCS labels so one container cannot read another's files. Use `:z` / `:Z` on bind mounts.

```
docker inspect api --format '{{.AppArmorProfile}}'
```

```
docker-default
```

8.4.7 Read-only root filesystem

An attacker who cannot write to the filesystem cannot drop binaries or modify app code.

```
docker run -d --read-only \
  --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --tmpfs /var/run:rw,size=1m \
  -v appdata:/data \
  myorg/api
```

Find what needs to be writable by running normally and checking `docker diff`.

8.4.8 Putting it together

```
docker run -d --name api \
  --user 10001:10001 \
  --read-only --tmpfs /tmp:rw,noexec,nosuid,size=64m \
  --cap-drop ALL \
  --security-opt no-new-privileges:true \
  --memory 512m --memory-swap 512m --cpus 1 --pids-limit 200 \
  --network backend \
  -p 127.0.0.1:8080:8080 \
  --restart unless-stopped \
  --health-cmd 'wget -qO- http://127.0.0.1:8080/healthz || exit 1' \
  ghcr.io/acme/api@sha256:5e1f...
```

The Compose equivalent:

```
services:
  api:
```

```
    image: ghcr.io/acme/api@sha256:5e1f...
    user: "10001:10001"
    read_only: true
    tmpfs:
      - /tmp:rw,noexec,nosuid,size=64m
    cap_drop: [ALL]
    security_opt: ["no-new-privileges:true"]
    deploy:
      resources:
        limits: { cpus: "1", memory: 512M, pids: 200 }
    networks: [backend]
```

8.5 Secrets at Runtime

Method	Security	Notes
Baked into image	Very poor	Anyone with pull access has it, forever.
`-e SECRET=...`	Poor to fair	Visible in `docker inspect`, `/proc/*/environ`, child processes, crash reports.
Bind-mounted file	Fair	File on host disk; restrict permissions.
Compose/Swarm secrets	Good	Mounted at `/run/secrets/`; Swarm encrypts at rest in the Raft log and only sends to nodes that need it.
External secrets manager	Best	Vault, AWS Secrets Manager, GCP Secret Manager, Azure Key Vault; short-lived credentials, rotation, audit.

App pattern: read `*_FILE` env vars pointing to secret files, falling back to the plain var only for local development.

```python
import os

def secret(name: str) -> str:
    path = os.environ.get(f"{name}_FILE")
    if path:
        with open(path) as f:
            return f.read().strip()
    return os.environ[name]
```

8.6 Security Checklist

Area	Control
Host	Patched kernel/Engine, rootless or userns-remap, no TCP socket, CIS benchmark
Image	Minimal pinned base, multi-stage, no secrets, non-root USER, OCI labels

Area	Control
Pipeline	Trivy/Scout gate on HIGH/CRITICAL, SBOM, provenance, cosign keyless signing
Registry	Private, scan on push, immutable tags, retention, pull-through cache
Runtime	cap-drop ALL, no-new-privileges, seccomp default or tighter, AppArmor/SELinux, read-only FS, limits
Network	User-defined/internal networks, bind to 127.0.0.1 where possible, DOCKER-USER rules
Secrets	Files or secrets manager; never in env for production
Ops	Continuous re-scanning, rebuild cadence, audit logs, signature verification at deploy

8.7 Lab: Harden and Verify an Image

Goal: take a root, full-OS image, harden it, scan it, sign it, and verify the signature.

1. Baseline scan of a naive image:

```
docker build -f Dockerfile.naive -t pyapi:naive ~/labs/pyapi   # from Part 3
trivy image --severity HIGH,CRITICAL pyapi:naive | tail -n 5
docker run --rm pyapi:naive id
```

2. Scan the optimized image (`pyapi:opt`) and compare counts. Then try distroless Python (`gcr.io/distroless/python3-debian12:nonroot`) as the runtime stage, copying the venv's site-packages, and scan again.

3. Run the hardened container:

```
docker run -d --name hard -p 127.0.0.1:8000:8000 \
  --read-only --tmpfs /tmp --cap-drop ALL \
  --security-opt no-new-privileges:true --user 10001:10001 \
  --memory 256m --pids-limit 100 pyapi:opt
curl -s localhost:8000/healthz
docker exec hard sh -c 'touch /app/hack' ; echo "exit=$?"     # read-only FS: fails
docker exec hard sh -c 'grep CapEff /proc/self/status'
```

4. Generate an SBOM and scan it:

```
syft pyapi:opt -o cyclonedx-json > sbom.json
grype sbom:sbom.json --fail-on high
```

5. Sign and verify with a local key, pushing to the lab registry from Part 4:

```
docker run -d --name registry -p 5000:5000 registry:2
docker tag pyapi:opt localhost:5000/pyapi:opt && docker push localhost:5000/pyapi:opt
REF=$(docker inspect --format '{{index .RepoDigests 0}}' localhost:5000/pyapi:opt)   #
```

```
localhost:5000/pyapi@sha256:...
cosign generate-key-pair
cosign sign --key cosign.key --allow-insecure-registry --yes "$REF"
cosign verify --key cosign.pub --allow-insecure-registry "$REF" | jq '.[0].critical'
```

6. Tamper test: push a different image to the same tag and verify the tag; verification fails because the new digest has no signature.
7. Run `docker-bench-security` and fix two warnings.
8. Clean up: `docker rm -f hard registry`.

Key takeaways

- Secure the whole lifecycle: host, image, pipeline, registry, runtime, secrets.
- Never bake secrets into images; use BuildKit secret mounts for builds and files or a secrets manager at runtime.
- Scan with Trivy or Docker Scout in CI with a severity gate, and re-scan continuously in the registry.
- Generate SBOMs and provenance; sign digests with cosign (keyless in CI) and verify before deploy.
- Run as non-root, drop all capabilities, keep seccomp/AppArmor/SELinux on, set no-new-privileges and a read-only root.
- `--privileged` and the Docker socket are root on the host; treat both as exceptional.

Part 9 — Docker in CI/CD, Swarm, the Path to Kubernetes, and Production Practices

Up to now you have built and run containers by hand. In real organizations, images are built by pipelines, deployed by orchestrators, and operated under production standards. This Part covers Docker in GitHub Actions, GitLab CI and Jenkins (including caching), Docker Swarm basics, how every Docker concept maps to Kubernetes, and a production best-practice checklist.

9.1 Principles of Container CI/CD

1. **Build once, promote everywhere.** Build an image once per commit, tag it with the Git SHA, and promote that exact digest through test, staging and production. Never rebuild for production.
2. **The pipeline is the only path to the registry.** Developers do not push production images from laptops.
3. **Gate on quality and security**: tests, lint (`docker build --check`, hadolint), vulnerability scan, then sign.
4. **Cache aggressively** so builds stay fast on ephemeral runners.
5. **Configuration lives outside the image**, injected per environment.

```
git push
  |
  v
[lint] -> [build test stage + unit tests] -> [build & push image :sha-abc123 (cache)]
  -> [scan: Trivy/Scout gate] -> [SBOM + provenance + cosign sign]
  -> [integration tests with docker compose up --wait]
  -> [deploy to staging by digest] -> [smoke tests]
  -> [manual approval / tag v1.4.2] -> [retag digest as 1.4.2, deploy prod]
```

9.2 GitHub Actions

```yaml
# .github/workflows/ci.yaml
name: ci
on:
  push:
    branches: [main]
    tags: ["v*"]
  pull_request:

permissions:
  contents: read
  packages: write
  id-token: write        # for keyless cosign signing
  security-events: write # upload SARIF
```

```yaml
env:
  IMAGE: ghcr.io/${{ github.repository }}

jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v4

      - uses: docker/setup-qemu-action@v3
      - uses: docker/setup-buildx-action@v3

      - uses: docker/login-action@v3
        if: github.event_name != 'pull_request'
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - id: meta
        uses: docker/metadata-action@v5
        with:
          images: ${{ env.IMAGE }}
          tags: |
            type=sha,prefix=sha-
            type=ref,event=branch
            type=ref,event=pr
            type=semver,pattern={{version}}
            type=semver,pattern={{major}}.{{minor}}

      - name: Unit tests (test stage)
        uses: docker/build-push-action@v6
        with:
          context: .
          target: test
          cache-from: type=gha
          cache-to: type=gha,mode=max

      - id: build
        name: Build and push
        uses: docker/build-push-action@v6
        with:
          context: .
          target: runtime
          platforms: linux/amd64,linux/arm64
          push: ${{ github.event_name != 'pull_request' }}
          tags: ${{ steps.meta.outputs.tags }}
          labels: ${{ steps.meta.outputs.labels }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
```

```
          sbom: true
          provenance: mode=max

      - name: Scan
        if: github.event_name != 'pull_request'
        uses: aquasecurity/trivy-action@0.28.0
        with:
          image-ref: ${{ env.IMAGE }}@${{ steps.build.outputs.digest }}
          severity: HIGH,CRITICAL
          ignore-unfixed: true
          exit-code: "1"
          format: sarif
          output: trivy.sarif

      - uses: github/codeql-action/upload-sarif@v3
        if: always() && github.event_name != 'pull_request'
        with:
          sarif_file: trivy.sarif

      - uses: sigstore/cosign-installer@v3
        if: github.event_name != 'pull_request'
      - name: Sign
        if: github.event_name != 'pull_request'
        run: cosign sign --yes "${IMAGE}@${{ steps.build.outputs.digest }}"
```

Notes:

- `type=gha` stores BuildKit cache in the GitHub Actions cache service; `mode=max` includes intermediate stages so the `test` and `runtime` stages share dependency layers.
- Pull requests from forks do not get secrets; the workflow builds but does not push.
- Third-party actions should be pinned to a commit SHA in security-sensitive repos (tags are shown here for readability).
- Downstream deploy jobs consume `needs.build.outputs.digest` so they deploy exactly what was scanned and signed.

9.3 GitLab CI

GitLab runners can build images in three ways:

Approach	How	Trade-offs
Docker-in-Docker (dind)	`docker:27` job image + `docker:27-dind` service, privileged runner	Full Docker; requires privileged mode; cache is empty each job unless exported.
Socket binding	Mount host `/var/run/docker.sock` into the job	Fast, shared cache; jobs get root on the runner host and can see each other's containers.

Approach	How	Trade-offs
Daemonless (BuildKit rootless, Kaniko successors, Buildah)	Build without a Docker daemon	No privileged mode; good for Kubernetes runners.

```yaml
# .gitlab-ci.yml
stages: [build, scan, release]

variables:
  IMAGE: $CI_REGISTRY_IMAGE
  DOCKER_TLS_CERTDIR: "/certs"

build:
  stage: build
  image: docker:27
  services: [docker:27-dind]
  before_script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin
"$CI_REGISTRY"
    - docker buildx create --use --driver docker-container
  script:
    - >
      docker buildx build
      --cache-from type=registry,ref=$IMAGE:buildcache
      --cache-to type=registry,ref=$IMAGE:buildcache,mode=max
      --provenance=mode=max --sbom=true
      -t $IMAGE:$CI_COMMIT_SHORT_SHA
      --push .

scan:
  stage: scan
  image:
    name: aquasec/trivy:0.56.2
    entrypoint: [""]
  variables:
    TRIVY_USERNAME: $CI_REGISTRY_USER
    TRIVY_PASSWORD: $CI_REGISTRY_PASSWORD
  script:
    - trivy image --exit-code 1 --severity HIGH,CRITICAL --ignore-unfixed
$IMAGE:$CI_COMMIT_SHORT_SHA

release:
  stage: release
  image: docker:27
  rules:
    - if: $CI_COMMIT_TAG
  script:
    - echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin
"$CI_REGISTRY"
    - docker buildx imagetools create -t $IMAGE:$CI_COMMIT_TAG $IMAGE:$CI_COMMIT_SHORT_SHA
```

`imagetools create` retags in the registry without pulling, preserving multi-arch manifests: this is “build once, promote”.

> **Note:** Kaniko, once the standard daemonless builder for GitLab on Kubernetes, was archived by Google in 2025. Prefer rootless BuildKit (`moby/buildkit:rootless` with `buildctl-daemonless.sh`), Buildah, or a remote BuildKit service.

9.4 Jenkins

```
// Jenkinsfile (declarative)
pipeline {
  agent { label 'docker' }
  environment {
    REGISTRY = 'harbor.corp.example.com'
    IMAGE    = "${REGISTRY}/payments/api"
    TAG      = "${env.GIT_COMMIT.take(7)}"
  }
  options { timestamps() }
  stages {
    stage('Build') {
      steps {
        withCredentials([usernamePassword(credentialsId: 'harbor', usernameVariable: 'U',
passwordVariable: 'P')]) {
          sh '''
            echo "$P" | docker login "$REGISTRY" -u "$U" --password-stdin
            docker buildx build \
              --cache-from type=registry,ref=$IMAGE:buildcache \
              --cache-to type=registry,ref=$IMAGE:buildcache,mode=max \
              -t $IMAGE:$TAG --push .
          '''
        }
      }
    }
    stage('Scan') {
      steps {
        sh 'trivy image --exit-code 1 --severity CRITICAL $IMAGE:$TAG'
      }
    }
    stage('Integration') {
      steps {
        sh '''
          docker compose -p ci-$BUILD_NUMBER -f compose.yaml -f compose.ci.yaml up -d --wait
          docker compose -p ci-$BUILD_NUMBER run --rm api pytest -q
        '''
      }
      post {
        always { sh 'docker compose -p ci-$BUILD_NUMBER down -v' }
      }
    }
```

```
    }
    post { always { sh 'docker logout $REGISTRY || true' } }
}
```

Jenkins tips: use dedicated build agents (or Kubernetes pod agents) rather than building on the controller, use the credentials store, clean up with `docker system prune -f --filter until=24h` in a scheduled job, and give each build a unique Compose project name.

9.5 CI Caching Strategies Compared

Strategy	Speed	Works on ephemeral runners	Notes
Local builder cache on persistent runner	Fastest	No	Self-hosted runners; must prune.
`type=gha`	Fast	Yes	GitHub Actions only; 10 GB per repo limit (by default), eviction.
`type=registry,mode=max`	Good	Yes	Any CI; separate cache tag; needs registry storage.
`type=inline`	OK	Yes	Only final stage layers; easiest.
`type=s3` / `azblob`	Good	Yes	Cloud object storage.
Remote BuildKit / Docker Build Cloud	Fast	Yes	Shared cache across all pipelines and developers.

Plus: order Dockerfile layers well, use cache mounts for package managers, and keep the build context small. Those matter more than any backend.

9.6 Docker Swarm Basics

Swarm mode is Docker's built-in orchestrator. It is simpler than Kubernetes, and still used for small clusters and edge deployments, but Kubernetes dominates the industry. Know the concepts.

Concept	Meaning
Node	A Docker host in the swarm: manager (Raft consensus, scheduling) or worker.
Service	Desired state: image, replicas, ports, networks, update policy.
Task	One container scheduled for a service on a node.
Stack	A set of services deployed from a Compose file.
Routing mesh	Any node accepts traffic on a published port and load-balances to tasks via IPVS.

```
# Manager
docker swarm init --advertise-addr 10.0.0.10
# Prints: docker swarm join --token SWMTKN-1-... 10.0.0.10:2377

# Workers
docker swarm join --token SWMTKN-1-... 10.0.0.10:2377

docker node ls
docker service create --name web --replicas 3 -p 80:80 \
  --update-parallelism 1 --update-delay 10s --update-failure-action rollback \
  nginx:1.27-alpine
docker service ls
docker service ps web
docker service scale web=5
docker service update --image nginx:1.27.2-alpine web    # rolling update
docker service rollback web
docker service logs -f web
```

Stacks reuse Compose files, with a `deploy:` section that Swarm honors fully:

```
# stack.yaml
services:
  api:
    image: ghcr.io/acme/api:1.4.2
    networks: [appnet]
    secrets: [db_password]
    deploy:
      replicas: 3
      update_config: { parallelism: 1, delay: 10s, order: start-first, failure_action:
rollback }
      restart_policy: { condition: on-failure }
      placement:
        constraints: ["node.role == worker"]
      resources:
        limits: { cpus: "0.5", memory: 256M }
networks:
  appnet:
    driver: overlay
secrets:
  db_password:
    external: true
```

```
printf 's3cr3t' | docker secret create db_password -
docker stack deploy -c stack.yaml shop
docker stack services shop
docker stack rm shop
```

Run 3 or 5 managers for fault tolerance (Raft needs a majority: 3 managers tolerate 1 failure, 5 tolerate 2).

9.7 The Path to Kubernetes

Kubernetes runs OCI images through a CRI runtime (containerd or CRI-O), not through dockerd. Your Docker skills transfer almost one to one.

Docker / Compose	Kubernetes	Notes
Image	Image	Same OCI image, same registries.
Container	Container	Inside a Pod.
`--network container:x` / shared netns	Pod	Containers in a Pod share network (and optionally PID) namespaces.
Compose service	Deployment + Service	Deployment manages replicas; Service gives a stable virtual IP and DNS name.
`deploy.replicas`, `--scale`	`spec.replicas`, HPA	HorizontalPodAutoscaler adds metric-driven scaling.
`ports: "8080:80"`	Service (ClusterIP/NodePort/LoadBalancer), Ingress/Gateway	Ingress or Gateway API for HTTP routing and TLS.
User-defined network + DNS	Cluster network + CoreDNS	`api.default.svc.cluster.local`; NetworkPolicy for isolation.
Named volume	PersistentVolumeClaim	Backed by CSI drivers (EBS, PD, Azure Disk, Ceph).
Bind mount of config	ConfigMap	Mounted as files or env.
Compose/Swarm secret	Secret	Base64, not encrypted by default; enable encryption at rest or use External Secrets.
`HEALTHCHECK`	livenessProbe, readinessProbe, startupProbe	Kubernetes ignores Dockerfile HEALTHCHECK.
`depends_on`	initContainers, readiness probes, retries	No direct equivalent; design for retries.
`--memory`, `--cpus`	`resources.limits` and `resources.requests`	Requests drive scheduling; limits enforce cgroups.
`--restart`	`restartPolicy` + controller	Deployments always restart; Jobs for run-to-completion.
`ENTRYPOINT` / `CMD`	`command` / `args`	`command` overrides ENTRYPOINT, `args` overrides CMD.
`--user`, `--cap-drop`, `--read-only`	`securityContext`	`runAsNonRoot`, `capabilities.drop: [ALL]`, `readOnlyRootFilesystem`.

Docker / Compose	Kubernetes	Notes
`docker logs`, `exec`	`kubectl logs`, `kubectl exec`	`kubectl debug` for ephemeral debug containers.
Compose migration migrate job	Job / Helm hook	
Compose file	Manifests, Helm chart, Kustomize	Kompose can convert Compose to manifests as a starting point.

The same API container, translated:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api
spec:
  replicas: 3
  selector:
    matchLabels: { app: api }
  template:
    metadata:
      labels: { app: api }
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 10001
        seccompProfile: { type: RuntimeDefault }
      containers:
        - name: api
          image: ghcr.io/acme/api@sha256:5e1f...
          ports: [{ containerPort: 8080 }]
          envFrom: [{ configMapRef: { name: api-config } }]
          resources:
            requests: { cpu: 250m, memory: 256Mi }
            limits: { memory: 512Mi }
          readinessProbe:
            httpGet: { path: /healthz, port: 8080 }
          livenessProbe:
            httpGet: { path: /healthz, port: 8080 }
            initialDelaySeconds: 10
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities: { drop: [ALL] }
          volumeMounts: [{ name: tmp, mountPath: /tmp }]
      volumes:
        - name: tmp
          emptyDir: { medium: Memory, sizeLimit: 64Mi }
---
apiVersion: v1
kind: Service
```

```
metadata:
  name: api
spec:
  selector: { app: api }
  ports: [{ port: 80, targetPort: 8080 }]
```

For local Kubernetes, use kind (Kubernetes in Docker), k3d, minikube, or Docker Desktop's built-in cluster. `kind load docker-image myapp:dev` copies a locally built image into the cluster nodes without a registry.

9.8 Production Best Practices

9.8.1 Image

- Small, pinned, regularly rebuilt base; multi-stage; non-root numeric UID; no secrets.
- OCI labels with source repo and commit SHA.
- One process per container (sidecars for helpers), exec-form entrypoint, handles SIGTERM, `--init` or tini if it spawns children.

9.8.2 Application behavior

- Log JSON to stdout/stderr.
- Read configuration from env and files; no environment-specific images.
- Health endpoints: liveness ("process is not wedged") separate from readiness ("can serve traffic now").
- Graceful shutdown: on SIGTERM stop accepting new work, finish in-flight requests within the grace period, exit 0.
- Stateless where possible; state in databases, object storage or volumes.
- Retry with backoff for dependencies.

9.8.3 Runtime and operations

- Resource limits on every container; size from observed metrics.
- Restart policy (`unless-stopped`) or orchestrator controller.
- Log rotation (`local` driver) and central shipping.
- Metrics (cAdvisor/Prometheus), alerting on restarts, OOMs, throttling, disk.
- Deploy by digest; keep previous digests for instant rollback.
- Separate `/var/lib/docker` onto its own disk or volume to protect the root filesystem.
- Scheduled cleanup (`docker system prune -f --filter until=72h`, `docker builder prune`).
- Use `live-restore` so Engine upgrades do not restart containers.
- Back up volumes and test restores.

9.8.4 Zero-downtime deploy on a single host

Compose recreates containers, causing a brief gap. Options: run two replicas behind a reverse proxy (Traefik, Caddy, nginx) and update one at a time; use Swarm's `order: start-first`; or graduate to Kubernetes rolling updates.

9.9 Lab: A GitHub Actions Pipeline With Cache, Scan and Sign

Goal: wire the Python API from Part 3 into a real pipeline.

1. Push the `pyapi` project to a new GitHub repository. Add `test` and `runtime` stages to the Dockerfile (the test stage installs `pytest` and runs a one-line test that imports `app`).
2. Add the workflow from section 9.2 at `.github/workflows/ci.yaml`.
3. Open a pull request. Confirm the build and test stages run, but nothing is pushed.
4. Merge to `main`. Confirm in the Actions log: - `importing cache manifest from gha` on the second run, and `CACHED` steps. - Trivy SARIF results appear under the repository's Security tab. - `cosign sign` succeeds with a Fulcio certificate for the workflow identity.
5. Locally verify the signature:

```
cosign verify ghcr.io/<you>/pyapi@sha256:<digest> \
  --certificate-identity-regexp "https://github.com/<you>/pyapi/.*" \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com | jq '.
[0].optional.Subject'
```

6. Tag `v0.1.0` and confirm the semver tags `0.1.0` and `0.1` point to the same digest:

```
docker buildx imagetools inspect ghcr.io/<you>/pyapi:0.1.0 --format '{{.Manifest.Digest}}'
docker buildx imagetools inspect ghcr.io/<you>/pyapi:sha-<short> --format
'{{.Manifest.Digest}}'
```

7. Introduce a vulnerable dependency (an old version of a package with a known HIGH CVE) and confirm the scan gate fails the pipeline. Revert.
8. Optional: deploy the digest to a kind cluster with the manifest from section 9.7.

Key takeaways

- Build once per commit, tag with the SHA, promote the same digest; retag with `imagetools create` rather than rebuilding.
- In GitHub Actions use `setup-buildx`, `metadata-action`, `build-push-action` with `type=gha` cache, then scan and sign by digest.

- GitLab and Jenkins builds use dind, socket binding, or daemonless BuildKit; know the security trade-offs of each.
- Swarm offers services, stacks, overlay networks, secrets and rolling updates with a Compose-like file.
- Kubernetes maps Docker concepts directly: Pod = shared namespaces, Deployment + Service = Compose service, probes = healthchecks, securityContext = run flags.
- Production readiness is a checklist: limits, health, graceful shutdown, logs, metrics, digests, cleanup, backups.

Part 10 — Troubleshooting Playbook

Everything breaks eventually. The difference between a junior and a senior engineer is a systematic approach: gather facts, form a hypothesis, test it, fix, and prevent recurrence. This Part is a field guide organized by symptom.

10.1 The General Method

```
1. What is the state?     docker ps -a, docker compose ps, docker events
2. What did it say?       docker logs --tail 200, docker inspect .State
3. What changed?          image digest, config, host, dependency, traffic
4. Reproduce in isolation docker run -it --entrypoint sh <image>
5. Look from outside      nsenter, netshoot, cgroup files, host logs (journalctl -u docker)
6. Fix, then prevent      healthcheck, limits, alert, test, runbook
```

The first five commands to run on any broken container:

```
docker ps -a --filter name=api
docker logs --tail 100 api
docker inspect api --format 'Status={{.State.Status}} Exit={{.State.ExitCode}} OOM=
{{.State.OOMKilled}} Err={{.State.Error}} Restarts={{.RestartCount}}'
docker inspect api --format '{{json .State.Health}}' | jq '.Log[-3:]'
docker events --since 1h --filter container=api
```

10.2 Container Exits Immediately

Symptom

```
$ docker ps -a
NAMES   STATUS
api     Exited (1) 3 seconds ago
```

Causes and checks

Exit code	Look for
0	The main process finished. A container lives only as long as PID 1. Common with images whose CMD is a shell, or a daemon that forks into the background (`nginx` without `daemon off` , `service x start`).
1	App error: read `docker logs` . Missing env var, bad config, cannot reach dependency.
125	`docker run` itself failed: invalid flag, name conflict, port already allocated.

Exit code	Look for
126	Not executable: missing `+x`, script with Windows CRLF line endings, wrong architecture.
127	Not found: typo, binary not in PATH, or the interpreter in the shebang does not exist (`#!/bin/bash` on Alpine).
137	SIGKILL: OOM, `docker kill`, or stop timeout. Check `OOMKilled`.
139	Segfault: often native library or musl/glibc mismatch.
143	SIGTERM: stopped normally.

Debug by overriding the entrypoint and exploring:

```
docker run --rm -it --entrypoint sh myorg/api:1.4.2
# inside: ls -l /app, cat /app/entrypoint.sh, which python, env
docker run --rm myorg/api:1.4.2 --version     # test args
docker image inspect myorg/api:1.4.2 --format 'EP={{json .Config.Entrypoint}} CMD={{json
.Config.Cmd}} USER={{.Config.User}} WD={{.Config.WorkingDir}}'
```

The tricky "no such file or directory" when the file clearly exists:

- Script has CRLF endings: the kernel looks for `/bin/sh\r`. Fix with `dos2unix` or `.gitattributes` (`*.sh text eol=lf`).
- Dynamically linked binary on `scratch`/Alpine: the ELF loader (`/lib64/ld-linux-x86-64.so.2`) is missing. Build static (`CGO_ENABLED=0`) or use a glibc base.
- Architecture mismatch: `exec format error` when running an amd64 image on arm64 without emulation. Check `docker image inspect --format '{{.Architecture}}'`.

10.3 Restart Loops

```
api   Restarting (1) 4 seconds ago
```

```
docker inspect api --format '{{.RestartCount}}'
docker logs --tail 50 api
docker update --restart no api     # stop the loop so you can investigate
```

Typical causes: dependency not ready at startup (add retries or `depends_on: condition: service_healthy`), migration failing, missing secret file, healthcheck killing the container under an orchestrator. Remember that Docker's backoff grows to one minute, so a looping container may look "stuck".

10.4 OOMKilled

```
docker inspect api --format 'OOM={{.State.OOMKilled}} Exit={{.State.ExitCode}} Limit=
{{.HostConfig.Memory}}'
journalctl -k | grep -i -E 'oom|killed process' | tail
docker stats --no-stream api
```

```
OOM=true Exit=137 Limit=536870912
kernel: Memory cgroup out of memory: Killed process 48211 (java) total-vm:3194824kB, anon-
rss:523112kB
```

Decision tree:

1. Is the limit realistic? Compare peak working set from metrics with the limit.
2. Does the runtime know the limit? JVM `-XX:MaxRAMPercentage`, Node `--max-old-space-size`, Go `GOMEMLIMIT`, worker counts for Python/Ruby.
3. Is there a leak? Memory climbs steadily under constant load.
4. Is it page cache or tmpfs? A tmpfs mount counts against the container's memory. Writing large files to `--tmpfs /tmp` can OOM-kill an app.
5. Exit 137 with `OOMKilled=false`: something else sent SIGKILL (stop timeout, `docker kill`, host-level OOM killer outside the cgroup, orchestrator).

10.5 Networking Problems

10.5.1 Cannot reach the app from the host

```
docker port api                                    # is it published?
docker exec api ss -tlnp 2>/dev/null || docker run --rm --network container:api
nicolaka/netshoot ss -tlnp
curl -v http://127.0.0.1:8080/
```

```
LISTEN 0  128  127.0.0.1:8080  0.0.0.0:*  users:(("node",pid=1,fd=18))
```

The app listens on `127.0.0.1` **inside** the container, which is the container's own loopback. Port forwarding arrives on the container's `eth0`. Fix: bind to `0.0.0.0` (e.g. `--host 0.0.0.0`, `server.address=0.0.0.0`).

Other causes: wrong container port in `-p host:container`, publishing to `127.0.0.1` while testing from another machine, cloud security groups, `DOCKER-USER` rules.

10.5.2 Containers cannot reach each other

```
docker inspect api db --format '{{.Name}} {{range $k, $v := .NetworkSettings.Networks}}{{$k}}
{{end}}'
```

```
docker exec api getent hosts db
docker exec api nc -zv db 5432
```

- Different networks: connect them with `docker network connect`.
- Default `bridge` network: no DNS by name. Use a user-defined network.
- Using `localhost:5432` from the app container: `localhost` is the app container itself. Use the service name `db`.
- Calling the **published** host port from another container instead of the container port.

10.5.3 Container has no internet / DNS fails

```
docker run --rm alpine:3.20 ping -c1 1.1.1.1        # IP connectivity
docker run --rm alpine:3.20 nslookup example.com    # DNS
sysctl net.ipv4.ip_forward                          # must be 1
sudo iptables -t nat -S POSTROUTING | grep MASQ
cat /etc/resolv.conf                                # host resolvers
```

- `ip_forward=0` (some hardening scripts disable it): re-enable.
- Host uses `127.0.0.53` (systemd-resolved); Docker detects this and uses upstream servers, but on custom setups set `"dns": ["10.0.0.2"]` in `daemon.json`.
- Corporate proxy: set `HTTP_PROXY` / `HTTPS_PROXY` / `NO_PROXY` for the daemon (systemd drop-in) for pulls, and in `~/.docker/config.json` `proxies` for containers and builds.
- Firewalld or nftables flushing Docker's rules: restart Docker after firewall reloads, or integrate properly.
- VPN or subnet overlap: route conflicts with `172.17.0.0/16`-style ranges. Set `default-address-pools` and `bip`.

10.5.4 "Port is already allocated"

```
sudo ss -tlnp | grep ':8080'
docker ps --filter publish=8080
```

10.6 Disk Space

The most common production Docker incident: `/var/lib/docker` fills the disk and everything on the host fails.

```
df -h /var/lib/docker
docker system df
docker system df -v | head -40
sudo du -sh /var/lib/docker/* 2>/dev/null | sort -h
```

Usual suspects:

Culprit	Detect	Fix
Container logs without rotation	`sudo du -sh /var/lib/docker/containers/*/*-json.log`	`log-opts` max-size/max-file, `local` driver; recreate containers. Emergency: `truncate -s 0` the log file.
Old images	`docker image ls`, `docker system df`	`docker image prune -a --filter until=168h`
Build cache	`docker buildx du`	`docker builder prune --keep-storage 20gb`
Dangling/anonymous volumes	`docker volume ls -f dangling=true`	`docker volume prune` (check first!)
Writable layers	`docker ps -s`	App writing to container FS; move to a volume or fix the app.
Stopped containers	`docker ps -a -f status=exited`	`docker container prune`

Safe cleanup sequence on a server:

```
docker container prune -f --filter "until=24h"
docker image prune -a -f --filter "until=168h"
docker builder prune -f --keep-storage 10gb
docker network prune -f
# volumes: only after reviewing
docker volume ls -f dangling=true
```

> **Warning:** Do not delete files under `/var/lib/docker` by hand (except truncating log files). You will corrupt the daemon's metadata. If the disk is completely full and the daemon cannot start, free space elsewhere first.

Also watch **inodes** (`df -i`): many small layers can exhaust inodes before bytes.

10.7 Build Failures

Error	Cause	Fix
`failed to compute cache key: "/app/x" not found`	File excluded by `.dockerignore` or wrong context path.	Check `.dockerignore`, the `-f` and context arguments.
`COPY failed: forbidden path outside the build context`	`COPY ../shared`	Move the context up a level and use `-f`, or use named contexts: `--build-context shared=../shared` and `COPY --from=shared`.

Error	Cause	Fix
Stale dependencies	Cached `apt-get update` layer	Combine update+install; `--pull`; `--no-cache-filter`.
`exec format error` during RUN	Building for another platform without QEMU	Install binfmt, or cross-compile.
`no space left on device`	Builder cache or disk	`docker builder prune`.
Network timeouts in RUN	Proxy/DNS inside build	`--build-arg HTTPS_PROXY=...`, `--network host` for build, daemon DNS.
Secrets not found	Missing `--secret id=...`	Match the `id` in Dockerfile and CLI.

Get the full log and debug interactively:

```
docker build --progress=plain --no-cache . 2>&1 | tee build.log
docker buildx build --target build -t debug-stage . && docker run --rm -it debug-stage sh
BUILDX_EXPERIMENTAL=1 docker buildx debug --invoke /bin/sh build .   # drop into a failed step
```

10.8 Permission Problems

```
PermissionError: [Errno 13] Permission denied: '/data/app.db'
```

```
docker exec api id
docker exec api ls -ln /data
ls -ln ./data          # host side for bind mounts
```

- UID mismatch between container user and host directory owner (see Part 5).
- Read-only root filesystem: the app writes somewhere you did not provide as tmpfs/volume.
- SELinux denials: `ausearch -m avc -ts recent`; add `:Z` to bind mounts.
- Capability dropped: e.g. `cap_drop: [ALL]` breaks an entrypoint that runs `chown` (needs `CHOWN`) or `su-exec` (needs `SETUID/SETGID`). Fix the image so it does not need them.

10.9 Performance Problems

```
docker stats
cat /sys/fs/cgroup/system.slice/docker-<id>.scope/cpu.stat    # throttling
cat /sys/fs/cgroup/system.slice/docker-<id>.scope/memory.stat  # anon vs file
cat /sys/fs/cgroup/system.slice/docker-<id>.scope/io.stat
```

- CPU throttling from tight `--cpus` limits (Part 7).
- Heavy writes to the overlay writable layer: move to a volume.
- Bind mount performance on Docker Desktop: use named volumes for dependency directories, enable VirtioFS or synchronized file shares.
- Logging in blocking mode to a slow remote driver.
- DNS latency: `ndots` and search domains causing multiple lookups; use FQDNs.
- Emulated architecture (`--platform linux/amd64` on Apple Silicon) running 5-10 times slower.

10.10 Daemon-Level Problems

```
systemctl status docker containerd
journalctl -u docker --since "30 min ago" --no-pager | tail -50
sudo dockerd --debug          # foreground with debug (stop the service first)
docker info 2>&1 | grep -i -E 'warning|error'
```

Enable debug without restart: set `"debug": true` in `daemon.json` and send `SIGHUP` to dockerd (`sudo kill -HUP $(pidof dockerd)`). Send `SIGUSR1` to dump goroutine stacks to the log when the daemon hangs.

Common issues: invalid JSON in `daemon.json` (daemon will not start; validate with `jq . /etc/docker/daemon.json`), conflicting flags between `daemon.json` and the systemd unit (`-H` defined in both), storage driver errors after kernel changes, containerd not running.

10.11 Quick Reference: Symptom to First Command

Symptom	First command
Container not running	`docker ps -a` then `docker logs`
Exit 137	`docker inspect -f '{{.State.OOMKilled}}'`
Unhealthy	`docker inspect -f '{{json .State.Health}}' api`
Cannot connect from host	`docker port`, then `ss -tlnp` inside
Cannot connect between containers	`docker network inspect`, `getent hosts`
No internet	`ping 1.1.1.1` then `nslookup` from a container
Disk full	`docker system df`, `du` on `/var/lib/docker/containers`
Slow	`docker stats`, `cpu.stat` throttling
Build cannot find file	`.dockerignore`, context path
Permission denied	`id` inside, `ls -ln` on the mount
Daemon errors	`journalctl -u docker`

10.12 Lab: Break It and Fix It

Each scenario: run the broken command, diagnose with the playbook, write down the root cause, then fix.

1. **The disappearing container**

```
docker run -d --name s1 ubuntu:24.04
```

Hint: what is the default CMD, and is there a TTY?

2. **The invisible port**

```
docker run -d --name s2 -p 8080:8000 python:3.12-alpine python -m http.server 8000 --bind
127.0.0.1
curl -m 3 localhost:8080
```

Fix: bind to `0.0.0.0`.

3. **The memory hog**

```
docker run -d --name s3 --memory 50m redis:7.4-alpine
docker exec s3 redis-cli DEBUG POPULATE 2000000 key 64
docker inspect s3 --format '{{.State.OOMKilled}}'
```

Fix: raise the limit or set `maxmemory` and an eviction policy in Redis config so the app stays under the cgroup limit.

4. **The lonely database**

```
docker run -d --name s4db -e POSTGRES_PASSWORD=x postgres:16
docker run --rm postgres:16 pg_isready -h s4db
```

Fix: put both on a user-defined network.

5. **The CRLF entrypoint**

```
mkdir -p ~/labs/crlf && cd ~/labs/crlf
printf '#!/bin/sh\r\necho hi\r\n' > entry.sh
printf 'FROM alpine:3.20\nCOPY --chmod=755 entry.sh /entry.sh\nENTRYPOINT ["/entry.sh"]\n' >
Dockerfile
docker build -t crlf . && docker run --rm crlf
```

Fix: convert line endings (`sed -i 's/\r$//' entry.sh`) and add a `.gitattributes` rule.

6. **The full disk**: on a scratch VM, run the `chatty` container from Part 7 *without* log rotation for a minute, measure its log size, then configure rotation and recreate it.

7. Clean up: `docker rm -f s1 s2 s3 s4db`.

Key takeaways

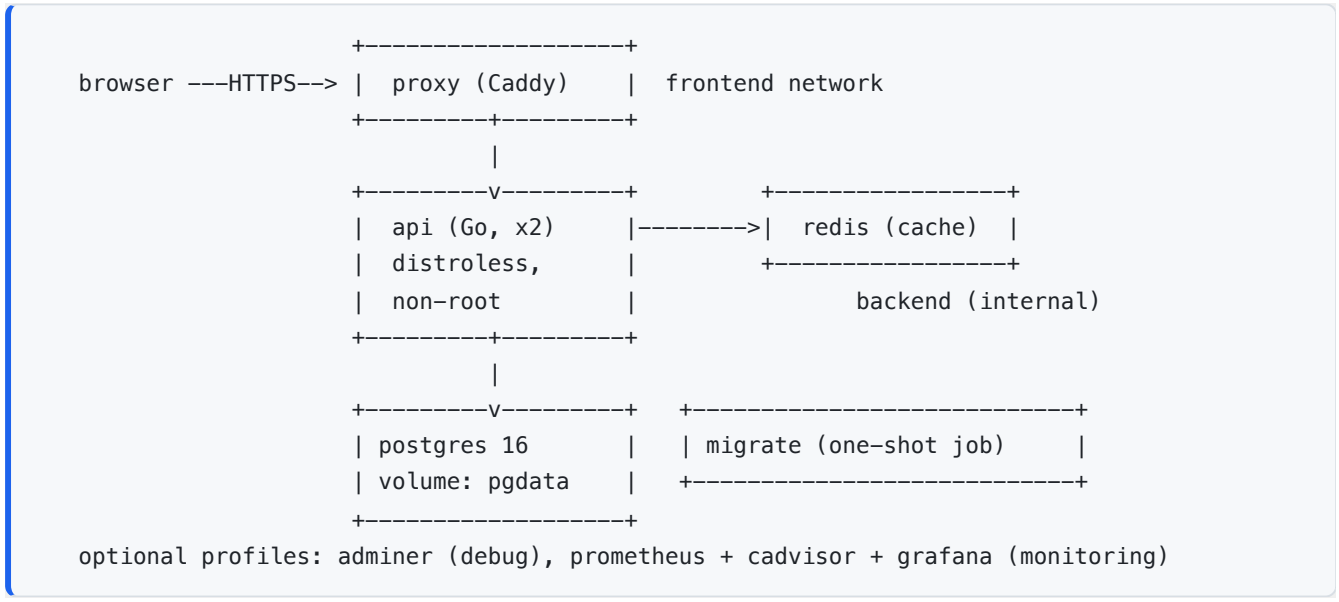
- Start every investigation with `ps -a`, `logs`, `inspect .State`, and `events`.
- Exit codes point the way: 125/126/127 startup problems, 137 kill/OOM, 139 segfault.
- “Connection refused” through a published port usually means the app is bound to `127.0.0.1` inside the container.
- Container-to-container problems are almost always networks, default bridge DNS, or using `localhost`.
- Disk incidents come from logs, images, build cache and volumes; prevent them with rotation and scheduled pruning.
- Debug minimal images from outside with `nsenter`, `--network container:`, `--pid container:` and netshoot.

Part 11 — Capstone Project, Interview Preparation and Cheat Sheet

This final Part pulls everything together. You will build a production-style multi-service application end to end: hardened images, a Compose environment with healthchecks and profiles, a CI pipeline with caching, scanning, SBOM and signing, and a deployment by digest. Then you will test yourself with more than forty interview questions, and finish with a dense command cheat sheet to keep at hand.

11.1 Capstone: “Linkly”, a URL Shortener Platform

11.1.1 Architecture



Requirements:

- `api` written in Go, built multi-stage and multi-arch, final image on distroless static, UID 65532, under 20 MB.
- Read-only root filesystem, all capabilities dropped, `no-new-privileges`, resource limits.
- Postgres password from a secret file; no secrets in env for production.
- Healthchecks everywhere; `api` waits for healthy `db` and successful `migrate`.
- Backend network is internal; only the proxy publishes ports.
- CI: lint, test stage, build with registry/GHA cache, Trivy gate, SBOM + provenance, cosign keyless signing, push by SHA, promote on tag.

11.1.2 Repository layout

```
linkly/
  api/
    cmd/linkly/main.go
```

```
    internal/...
    go.mod
    go.sum
    Dockerfile
    .dockerignore
  proxy/
    Caddyfile
  db/
    migrations/001_init.sql
  secrets/
    db_password.txt        (gitignored)
  compose.yaml
  compose.override.yaml
  compose.prod.yaml
  monitoring/
    compose.yaml
    prometheus.yml
  .env.example
  .github/workflows/ci.yaml
```

11.1.3 Step 1: the application

A minimal but realistic Go service (shortened for space):

```go
package main

import (
    "context"
    "database/sql"
    "encoding/json"
    "log/slog"
    "net/http"
    "os"
    "os/signal"
    "strings"
    "syscall"
    "time"

    _ "github.com/jackc/pgx/v5/stdlib"
)

func readSecret(name string) string {
    if p := os.Getenv(name + "_FILE"); p != "" {
        b, err := os.ReadFile(p)
        if err == nil {
            return strings.TrimSpace(string(b))
        }
    }
    return os.Getenv(name)
}

func main() {
```

```
    log := slog.New(slog.NewJSONHandler(os.Stdout, nil))
    dsn := "postgres://linkly:" + readSecret("DB_PASSWORD") + "@db:5432/linkly?
sslmode=disable"
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        log.Error("db open", "err", err)
        os.Exit(1)
    }

    mux := http.NewServeMux()
    mux.HandleFunc("GET /healthz", func(w http.ResponseWriter, r *http.Request) {
        w.WriteHeader(http.StatusOK)
    })
    mux.HandleFunc("GET /readyz", func(w http.ResponseWriter, r *http.Request) {
        ctx, cancel := context.WithTimeout(r.Context(), time.Second)
        defer cancel()
        if err := db.PingContext(ctx); err != nil {
            http.Error(w, "db not ready", http.StatusServiceUnavailable)
            return
        }
        json.NewEncoder(w).Encode(map[string]string{"status": "ready"})
    })
    // ... POST /links, GET /{code} handlers ...

    srv := &http.Server{Addr: ":8080", Handler: mux, ReadHeaderTimeout: 5 * time.Second}
    go func() {
        log.Info("listening", "addr", srv.Addr)
        if err := srv.ListenAndServe(); err != nil && err != http.ErrServerClosed {
            log.Error("serve", "err", err)
            os.Exit(1)
        }
    }()

    // Graceful shutdown on SIGTERM (docker stop) or SIGINT
    ctx, stop := signal.NotifyContext(context.Background(), syscall.SIGTERM, syscall.SIGINT)
    defer stop()
    <-ctx.Done()
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 20*time.Second)
    defer cancel()
    srv.Shutdown(shutdownCtx)
    log.Info("shut down cleanly")
}
```

Note the production behaviors: JSON logs to stdout, `*_FILE` secrets, separate liveness (`/healthz`) and readiness (`/readyz`), graceful shutdown on SIGTERM. Add a `healthcheck` subcommand (or a tiny separate binary) that GETs `/healthz`, because distroless has no `curl`.

11.1.4 Step 2: the hardened Dockerfile

```
# syntax=docker/dockerfile:1
# check=error=true
```

```
ARG GO_VERSION=1.23

FROM --platform=$BUILDPLATFORM golang:${GO_VERSION} AS deps
WORKDIR /src
COPY go.mod go.sum ./
RUN --mount=type=cache,target=/go/pkg/mod go mod download

FROM deps AS test
COPY . .
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    go vet ./... && go test ./...

FROM deps AS build
ARG TARGETOS TARGETARCH
ARG VERSION=dev
COPY . .
RUN --mount=type=cache,target=/go/pkg/mod \
    --mount=type=cache,target=/root/.cache/go-build \
    CGO_ENABLED=0 GOOS=$TARGETOS GOARCH=$TARGETARCH \
    go build -trimpath -ldflags="-s -w -X main.version=${VERSION}" \
      -o /out/linkly ./cmd/linkly

FROM gcr.io/distroless/static-debian12:nonroot AS runtime
ARG VERSION=dev
ARG GIT_SHA=unknown
LABEL org.opencontainers.image.title="linkly-api" \
      org.opencontainers.image.version="${VERSION}" \
      org.opencontainers.image.revision="${GIT_SHA}" \
      org.opencontainers.image.source="https://github.com/acme/linkly"
COPY --from=build /out/linkly /linkly
USER 65532:65532
EXPOSE 8080
HEALTHCHECK --interval=15s --timeout=3s --start-period=10s --retries=3 \
  CMD ["/linkly", "healthcheck"]
ENTRYPOINT ["/linkly"]
```

`.dockerignore`:

```
.git
**/*_test.go.orig
bin/
dist/
*.md
.env*
```

Build and check:

```
docker build --check api/
docker build --target test api/
```

```
docker buildx build --platform linux/amd64,linux/arm64 -t linkly-api:dev --target runtime
api/
docker image ls linkly-api
```

```
REPOSITORY   TAG   IMAGE ID       CREATED          SIZE
linkly-api   dev   5d3b...        12 seconds ago   14.2MB
```

11.1.5 Step 3: Compose

```yaml
# compose.yaml
name: linkly

x-hardening: &hardening
  read_only: true
  cap_drop: [ALL]
  security_opt: ["no-new-privileges:true"]
  init: false

services:
  proxy:
    image: caddy:2.8-alpine
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./proxy/Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy_data:/data
    cap_drop: [ALL]
    cap_add: [NET_BIND_SERVICE]
    depends_on:
      api:
        condition: service_healthy
    networks: [frontend]
    restart: unless-stopped

  api:
    <<: *hardening
    build:
      context: ./api
      target: runtime
    image: ghcr.io/acme/linkly-api:${TAG:-dev}
    environment:
      DB_PASSWORD_FILE: /run/secrets/db_password
      REDIS_ADDR: cache:6379
    secrets: [db_password]
    depends_on:
      db: { condition: service_healthy }
      cache: { condition: service_healthy }
      migrate: { condition: service_completed_successfully }
    deploy:
      replicas: 2
```

```
      resources:
        limits: { cpus: "0.50", memory: 128M, pids: 100 }
    networks: [frontend, backend]
    restart: unless-stopped

  migrate:
    image: migrate/migrate:v4.17.1
    command: ["-path=/migrations", "-database", "postgres://linkly:${DB_PASSWORD:?set
DB_PASSWORD}@db:5432/linkly?sslmode=disable", "up"]
    volumes:
      - ./db/migrations:/migrations:ro
    depends_on:
      db: { condition: service_healthy }
    networks: [backend]
    restart: "no"

  db:
    image: postgres:16.4-alpine
    environment:
      POSTGRES_USER: linkly
      POSTGRES_DB: linkly
      POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    secrets: [db_password]
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U linkly -d linkly"]
      interval: 5s
      timeout: 3s
      retries: 10
    deploy:
      resources:
        limits: { memory: 512M }
    networks: [backend]
    restart: unless-stopped

  cache:
    <<: *hardening
    image: redis:7.4-alpine
    command: ["redis-server", "--maxmemory", "64mb", "--maxmemory-policy", "allkeys-lru", "--
save", ""]
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 5s
      retries: 5
    deploy:
      resources:
        limits: { memory: 96M }
    networks: [backend]
    restart: unless-stopped

  adminer:
    image: adminer:4
```

```
    profiles: [debug]
    ports: ["127.0.0.1:8081:8080"]
    networks: [backend, frontend]

networks:
  frontend:
  backend:
    internal: true

volumes:
  pgdata:
  caddy_data:

secrets:
  db_password:
    file: ./secrets/db_password.txt
```

Points worth noticing:

- `x-hardening` is an **extension field** with a YAML anchor reused by several services.
- `migrate` gets the password via interpolation only because the migrate tool needs a URL; in a stricter setup, wrap it in a script that reads the secret file.
- The `api` replicas share the service DNS name `api`; Caddy load-balances with `reverse_proxy api:8080`. No host ports on `api`, so scaling does not cause port conflicts.
- Redis gets an internal `maxmemory` below its cgroup limit, so it evicts keys instead of being OOM-killed.

`proxy/Caddyfile`:

```
:80 {
    reverse_proxy api:8080 {
        health_uri /readyz
        lb_policy round_robin
    }
}
```

`compose.override.yaml` (dev):

```
services:
  api:
    build:
      target: build
    read_only: false
    develop:
      watch:
        - action: rebuild
          path: ./api
```

`compose.prod.yaml`:

```yaml
services:
  api:
    image: ghcr.io/acme/linkly-api@${API_DIGEST:?set API_DIGEST}
    build: !reset null
    logging:
      driver: local
      options: { max-size: "10m", max-file: "5" }
  db:
    logging:
      driver: local
      options: { max-size: "10m", max-file: "5" }
```

Run it:

```
cp .env.example .env && openssl rand -hex 16 > secrets/db_password.txt
echo "DB_PASSWORD=$(cat secrets/db_password.txt)" >> .env
docker compose config --quiet
docker compose up -d --build --wait
docker compose ps
curl -s -X POST localhost/links -d '{"url":"https://docs.docker.com"}'
```

11.1.6 Step 4: monitoring profile

Add `monitoring/compose.yaml` with cAdvisor, Prometheus and Grafana from Part 7 (each with `profiles: [monitoring]`) and include it:

```yaml
# at the top of compose.yaml
include:
  - monitoring/compose.yaml
```

```
docker compose --profile monitoring up -d
```

Build a Grafana panel for CPU throttling and memory working set versus limit for the `api` containers.

11.1.7 Step 5: CI pipeline

Use the workflow from Part 9 with these changes:

- `context: ./api`.
- A `lint` job: `docker build --check ./api` and `hadolint api/Dockerfile`.
- An `integration` job that `needs: build`, pulls the image by digest, and runs:

```
export API_DIGEST=${{ needs.build.outputs.digest }}
openssl rand -hex 16 > secrets/db_password.txt
export DB_PASSWORD=$(cat secrets/db_password.txt)
docker compose -f compose.yaml -f compose.prod.yaml up -d --wait
curl -fsS localhost/readyz
```

```
docker compose logs --no-color > compose.log
docker compose down -v
```

- A `release` job on tags that promotes the digest: `docker buildx imagetools create -t ghcr.io/acme/linkly-api:${TAG} ghcr.io/acme/linkly-api@${DIGEST}`.

11.1.8 Step 6: deploy and verify

On the production host (via a Docker context over SSH):

```
DIGEST=sha256:...   # from the CI release job
cosign verify ghcr.io/acme/linkly-api@$DIGEST \
  --certificate-identity-regexp
'https://github.com/acme/linkly/.github/workflows/ci.yaml@refs/tags/v.*' \
  --certificate-oidc-issuer https://token.actions.githubusercontent.com > /dev/null \
  && echo "signature OK"
export API_DIGEST=$DIGEST
docker --context prod compose -f compose.yaml -f compose.prod.yaml pull
docker --context prod compose -f compose.yaml -f compose.prod.yaml up -d --wait --remove-
orphans
```

Rollback = redeploy with the previous digest.

11.1.9 Acceptance checklist

Check	Command	Expected
Image size	`docker image ls`	under 20 MB
Non-root	`docker image inspect -f '{{.Config.User}}' linkly-api:dev`	`65532:65532`
Read-only	`docker inspect -f '{{.HostConfig.ReadonlyRootfs}}'`	`true`
Capabilities	`docker inspect -f '{{.HostConfig.CapDrop}}'`	`[ALL]`
No published DB port	`docker compose port db 5432`	error / empty
Vulnerabilities	`trivy image --severity HIGH,CRITICAL`	0
Signed	`cosign verify ...`	success
Graceful stop	`time docker compose stop api`	under 2 s, exit 0
Data persists	`down` then `up` without `-v`	links still resolve
Backend isolation	`docker compose exec cache ping -c1 1.1.1.1`	fails

11.2 Interview Questions and Answers

11.2.1 Beginner

1. What is the difference between an image and a container? An image is an immutable, layered filesystem plus metadata (a template). A container is a running (or stopped) instance of an image with its own writable layer, namespaces and cgroup.

2. What is the difference between a container and a VM? A VM virtualizes hardware and runs its own kernel under a hypervisor. A container is a process isolated by kernel namespaces and constrained by cgroups, sharing the host kernel. Containers are lighter and faster; VMs offer stronger isolation.

3. What does `docker run` actually do? Pulls the image if missing, creates a container (writable layer, network endpoint, mounts, config), starts it via containerd/runc, and attaches or detaches.

4. `CMD` vs `ENTRYPOINT`? `ENTRYPOINT` sets the executable; `CMD` sets default arguments (or the default command if there is no entrypoint). Arguments to `docker run` replace `CMD`; `--entrypoint` replaces `ENTRYPOINT`.

5. `COPY` vs `ADD`? Both copy files into the image. `ADD` also extracts local tar archives and can fetch URLs and Git repos. Prefer `COPY` for clarity unless you need those features.

6. What is a Docker volume and why use it? Docker-managed persistent storage outside the container's writable layer. Data survives container removal, performs better than the overlay layer, and can be shared and backed up.

7. Volume vs bind mount? A volume is managed by Docker under its storage area; a bind mount maps any host path. Bind mounts are good for dev and config; volumes for persistent app data.

8. What does `EXPOSE` do? Documents which ports the app listens on and enables `-P`. It does not publish ports.

9. How do you see logs of a container? `docker logs [-f] [--tail N] <container>`, provided the logging driver supports reading.

10. How do you get a shell in a running container? `docker exec -it <container> sh` (or bash). For images without a shell, use a debug container sharing namespaces or `docker debug`.

11. What is a Dockerfile layer? The filesystem change produced by an instruction like `RUN`, `COPY` or `ADD`. Layers are cached, content-addressed and shared between images.

12. What is `.dockerignore` for? Excludes files from the build context, making builds faster and preventing secrets, `.git` and local artifacts from entering images.

13. What is Docker Compose? A tool to define and run multi-container apps from a YAML file (`compose.yaml`), managing services, networks, volumes and secrets as one project with `docker compose up/down`.

14. What does `latest` mean? Only the default tag name when none is specified. It does not mean newest and should not be used for deployments.

11.2.2 Intermediate

15. How does the build cache work, and how do you optimize for it? Each instruction is cached by its text and inputs (file checksums for `COPY`). A miss invalidates all subsequent steps. Copy dependency manifests and install dependencies before copying source code; use cache mounts for package managers.

16. What are multi-stage builds and why use them? Multiple `FROM` stages in one Dockerfile; the final stage copies only artifacts from earlier stages. This removes compilers and dev dependencies from the runtime image, reducing size and attack surface, and enables test/dev stages.

17. Why should a container's main process use the exec form? Shell form runs under `/bin/sh -c`, which becomes PID 1 and does not forward signals, so SIGTERM never reaches the app and `docker stop` ends with SIGKILL after the timeout.

18. What is the PID 1 problem? PID 1 gets no default signal handlers and must reap zombies. Apps not written for that ignore SIGTERM and leak zombies. Solutions: handle signals, use `--init` (tini) or `exec` in entrypoint scripts.

19. What does exit code 137 mean? 128 + 9: the process received SIGKILL, commonly from the OOM killer (check `OOMKilled`) or a stop timeout.

20. How do containers on the same host talk to each other? Via a user-defined bridge network, using container or service names resolved by Docker's embedded DNS at 127.0.0.11. The default bridge has no name resolution.

21. How does port publishing work under the hood? Docker adds iptables DNAT rules in the `nat` table's `DOCKER` chain to forward host port traffic to the container IP and port, plus filter rules and MASQUERADE for outbound traffic. Published ports bypass host INPUT firewall rules; use `DOCKER-USER`.

22. Explain `depends_on` conditions in Compose. `service_started` (default) only orders startup; `service_healthy` waits for the dependency's healthcheck; `service_completed_successfully` waits for a one-shot job to exit 0. Apps still need retry logic at runtime.

23. Difference between `ARG` and `ENV`? `ARG` exists only at build time (and is visible in history); `ENV` persists into the image and running containers.

24. How do you pass secrets during a build safely? BuildKit secret mounts: `RUN --mount=type=secret,id=x` with `docker build --secret id=x,src=...`. The secret is never written into a layer. Never use `ARG` / `ENV` for secrets.

25. How do you limit a container's resources? `--memory`, `--memory-swap`, `--cpus`, `--cpu-shares`, `--pids-limit`, `--cpuset-cpus`, or `deploy.resources` in Compose. These configure cgroup controllers.

26. What are Docker's logging drivers and what is the risk with the default? Drivers send stdout/stderr to `json-file`, `local`, `journald`, `syslog`, `fluentd`, `awslogs`, and others. The default `json-file` does not rotate unless configured, so logs can fill the disk.

27. How would you reduce an image from 1 GB to under 100 MB? Multi-stage build, slim/alpine/distroless base, production-only dependencies, clean package caches in the same layer, tight `.dockerignore`, static binaries for compiled languages, and inspect with `dive`.

28. What is the difference between `docker stop` and `docker kill`? `stop` sends the stop signal (SIGTERM by default), waits for the grace period (10 s), then SIGKILL. `kill` sends SIGKILL immediately (or a chosen signal with `-s`).

29. How do you make Compose work for dev, CI and prod? A canonical `compose.yaml`, an auto-loaded `compose.override.yaml` for dev, explicit `-f compose.prod.yaml` or `compose.ci.yaml` overlays, profiles for optional services, and `.env` interpolation; verify with `docker compose config`.

30. What is the difference between a tag and a digest? A tag is a mutable, human-readable pointer; a digest (`sha256:...`) is the immutable content hash of a manifest or index. Deploy by digest for reproducibility and safety.

11.2.3 Advanced

31. Walk through the Docker architecture from CLI to process. The CLI calls the dockerd REST API; dockerd asks containerd (gRPC) to create and start the container; containerd spawns a shim per container; the shim invokes runc with an OCI bundle; runc creates namespaces, cgroups, mounts, applies seccomp/capabilities/LSM, execs the entrypoint, and exits. The shim stays as the parent, holding stdio and exit status.

32. What is OCI and why does it matter? The Open Container Initiative defines the image, runtime and distribution specs. They make images portable across Docker, Podman, containerd and Kubernetes, and allow swapping runtimes like runc, crun, gVisor or Kata.

33. Kubernetes removed dockershim. Do Docker-built images still work? Yes. Kubernetes uses CRI runtimes (containerd, CRI-O) that run any OCI image. Only using dockerd as the node runtime was removed.

34. Explain how overlayfs implements image layers. Read-only lower directories (image layers) and a writable upper directory are merged into one view. Reads hit the topmost copy; writes to lower files trigger copy-up; deletions create whiteout files. That is why deleting a file in a later layer does not shrink an image.

35. Which namespaces does a container use, and how would you enter them without docker exec? PID, NET, MNT, UTS, IPC, USER (optional), CGROUP. Find the PID with `docker inspect -f '{{.State.Pid}}'` and use `nsenter -t <pid> -n -m -p ...`, which lets you use host tools inside a minimal container.

36. How do cgroups v2 enforce `--cpus` and `--memory`, and what is CPU throttling? `--cpus` writes a quota/period to `cpu.max`; once the group uses its quota within a period, it is throttled until the next period. `--memory` sets `memory.max`; exceeding it invokes the OOM killer within the cgroup. Throttling shows in `cpu.stat` (`nr_throttled`) and causes latency spikes for multi-threaded apps.

37. How do you harden a container at runtime? Run as non-root (numeric UID), `--cap-drop ALL` plus minimal adds, `no-new-privileges`, keep default seccomp and AppArmor/SELinux or tighten them, `--read-only` with tmpfs for scratch, resource and PID limits, internal networks, no Docker socket, no `--privileged`, and user namespaces or rootless mode.

38. Why is access to the Docker socket dangerous? The daemon runs as root and will start any container you ask for, including one that bind-mounts the host root filesystem with full privileges. Socket access equals root on the host.

39. Rootless Docker vs userns-remap? Rootless runs the daemon and containers as an unprivileged user with user namespaces, so even the daemon is not root. userns-remap keeps a root daemon but maps container root to an unprivileged UID range. Both reduce breakout impact; rootless has more networking and feature limitations.

40. How do you build multi-architecture images efficiently? `docker buildx build --platform linux/amd64,linux/arm64 --push`, producing an image index. Use `FROM --platform=$BUILDPLATFORM` and `TARGETOS` / `TARGETARCH` to cross-compile natively instead of emulating with QEMU, or use native builder nodes per architecture.

41. How do you speed up Docker builds on ephemeral CI runners? Export and import BuildKit cache (`type=gha`, `type=registry,mode=max`, `s3`), use a persistent or remote builder (Build Cloud, BuildKit on Kubernetes), order layers well, use cache mounts, and keep the context small.

42. What is an SBOM and how is it produced and used? A Software Bill of Materials listing every package and version in an image (SPDX or CycloneDX). Produce it with `docker buildx build --sbom=true` or Syft, attach it as an attestation, and scan it with Grype or Trivy to answer "where are we affected?" quickly when a CVE appears.

43. How does keyless signing with cosign work? In CI, cosign obtains an OIDC token (e.g. from GitHub Actions), gets a short-lived certificate from Fulcio binding the workload identity, signs the image digest, and records the signature in the Rekor transparency log. Verification checks the certificate identity and issuer and the log entry; no long-lived keys to manage.

44. A container is OOMKilled but its memory graph never reached the limit. Why? Metrics are sampled and may miss short spikes; the graph may show working set excluding page cache while the kernel counts some of it; tmpfs mounts count toward memory; or a child process spiked. Check `memory.events`, kernel logs, and `memory.stat`.

45. Published port works locally but not from another host. Diagnose. Check the binding (`127.0.0.1:` vs `0.0.0.0`), the app's listen address inside the container, cloud security groups/NACLs, `DOCKER-USER` rules, and whether host firewalls (firewalld zones) block FORWARD. Use `tcpdump` on the host interface and `docker0`.

46. How do you achieve zero-downtime deployments with containers? Run multiple replicas behind a load balancer with readiness checks, update gradually (start new before stopping old), drain connections with graceful SIGTERM handling, and keep backward-compatible database migrations. Swarm (`order: start-first`) and Kubernetes rolling updates automate it.

47. How does Docker's embedded DNS work, and what is `ndots`? Containers on user-defined networks use 127.0.0.11, which the daemon intercepts to resolve container names, service names and aliases, forwarding other queries to upstream resolvers. `ndots` controls when search domains are appended; in Kubernetes the default `ndots:5` can cause many extra lookups for external names.

48. Map Docker Compose concepts to Kubernetes. Service -> Deployment + Service; ports -> Service/Ingress; named volume -> PVC; config bind mount -> ConfigMap; secrets -> Secret; healthcheck -> liveness/readiness probes; resource limits -> requests/limits; `depends_on` -> init containers and retries; container network sharing -> Pod.

11.3 Command Cheat Sheet

11.3.1 Containers

```
docker run -d --name app -p 8080:80 -e K=V -v vol:/data --network net IMAGE
docker run --rm -it --entrypoint sh IMAGE
docker run --init --restart unless-stopped --memory 512m --cpus 1 --pids-limit 200 IMAGE
docker run --read-only --tmpfs /tmp --cap-drop ALL --security-opt no-new-privileges:true -u
10001 IMAGE
docker ps -a --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'
docker logs -f --tail 100 --since 10m app
docker exec -it app sh          ;  docker exec -u 0 app id
docker cp app:/path ./local     ;  docker cp ./local app:/path
docker stop -t 30 app ; docker kill -s HUP app ; docker restart app
docker rm -f app ; docker container prune -f
docker inspect -f '{{.State.Status}} {{.State.ExitCode}} {{.State.OOMKilled}}' app
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' app
docker stats --no-stream ; docker top app ; docker diff app ; docker port app
docker update --memory 1g --cpus 2 --restart on-failure:5 app
docker events --since 1h --filter type=container
docker wait app
```

11.3.2 Images and builds

```
docker build -t img:tag .                       # BuildKit
docker build -f Dockerfile.prod --target runtime --build-arg V=1 -t img:1 .
docker build --check .                          # lint
docker build --progress=plain --no-cache .
docker build --secret id=npmrc,src=$HOME/.npmrc --ssh default .
docker build --target export --output type=local,dest=./out .
docker buildx create --name b --driver docker-container --use --bootstrap
docker buildx build --platform linux/amd64,linux/arm64 -t reg/img:1 --push .
docker buildx build --cache-from type=registry,ref=reg/img:cache --cache-to
type=registry,ref=reg/img:cache,mode=max .
docker buildx build --sbom=true --provenance=mode=max --push -t reg/img:1 .
docker buildx bake --push
docker buildx imagetools inspect reg/img:1
docker buildx imagetools create -t reg/img:prod reg/img@sha256:...
docker buildx du ; docker builder prune --keep-storage 10gb
docker image ls ; docker image ls --digests ; docker image history img
docker image inspect -f '{{json .Config}}' img | jq
docker tag img reg/img:1 ; docker push reg/img:1 ; docker pull --platform linux/arm64 img
```

```
docker image save -o img.tar img ; docker image load -i img.tar
docker image prune -a -f --filter until=168h
```

11.3.3 Volumes and networks

```
docker volume create v ; docker volume ls ; docker volume inspect v ; docker volume rm v
docker volume prune                     # anonymous unused
docker run --rm -v v:/data -v "$PWD":/b alpine tar czf /b/v.tgz -C /data .
docker network create --subnet 10.20.0.0/24 net
docker network create --internal backend
docker network connect net app ; docker network disconnect net app
docker network inspect net ; docker network ls ; docker network prune
docker run --rm --network container:app nicolaka/netshoot
docker run --rm --network host IMAGE ; docker run --rm --network none IMAGE
```

11.3.4 Compose

```
docker compose up -d --build --wait
docker compose -f compose.yaml -f compose.prod.yaml up -d --remove-orphans
docker compose --profile debug up -d       # or COMPOSE_PROFILES=debug
docker compose config                      # resolved model
docker compose ps ; docker compose logs -f api ; docker compose top
docker compose exec api sh ; docker compose run --rm api pytest
docker compose up -d --scale worker=3
docker compose watch
docker compose pull ; docker compose build --pull ; docker compose push
docker compose down ; docker compose down -v --rmi local
docker compose ls
```

11.3.5 System, daemon and contexts

```
docker version ; docker info
docker system df -v
docker system prune ; docker system prune -a --volumes   # careful
docker context create prod --docker host=ssh://user@host ; docker context use prod
docker login reg ; docker logout reg
journalctl -u docker -f
sudo kill -HUP "$(pidof dockerd)"                         # reload daemon.json (supported
keys)
```

11.3.6 Security and supply chain

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 img
trivy config Dockerfile ; trivy fs .
docker scout quickview img ; docker scout cves img ; docker scout recommendations img
syft img -o cyclonedx-json > sbom.json ; grype sbom:sbom.json
cosign sign --yes reg/img@sha256:...
```

```
cosign verify reg/img@sha256:... --certificate-identity-regexp '...' --certificate-oidc-issuer '...'
docker run --rm hadolint/hadolint < Dockerfile
```

11.3.7 Swarm

```
docker swarm init --advertise-addr IP ; docker swarm join-token worker
docker node ls ; docker node update --availability drain NODE
docker service create --name web --replicas 3 -p 80:80 nginx:1.27-alpine
docker service ls ; docker service ps web ; docker service logs -f web
docker service scale web=5 ; docker service update --image nginx:1.27.2-alpine web
docker service rollback web
printf 'x' | docker secret create s - ; docker stack deploy -c stack.yaml app ; docker stack rm app
```

11.3.8 Internals and debugging

```
PID=$(docker inspect -f '{{.State.Pid}}' app)
sudo nsenter -t "$PID" -n ss -tlnp
sudo nsenter -t "$PID" -m -u -i -n -p sh
lsns -p "$PID"
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' app).scope/memory.events
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}' app).scope/cpu.stat
docker inspect -f '{{json .GraphDriver.Data}}' app | jq
sudo iptables -t nat -L DOCKER -n ; sudo iptables -L DOCKER-USER -n
```

11.3.9 Exit codes

Code	Meaning
0	Success
1	Application error
125	Docker run error
126	Not executable
127	Not found
137	SIGKILL (OOM / kill / timeout)
139	SIGSEGV
143	SIGTERM

11.4 Where to Go Next

- Kubernetes: Pods, Deployments, Services, Ingress/Gateway API, Helm, operators.
- Supply chain: SLSA levels, in-toto attestations, admission policies (Kyverno, policy-controller).
- Runtime security: Falco, Tetragon, gVisor and Kata for sandboxing.
- Platform engineering: remote builders, internal base image programs, golden paths.

Key takeaways

- The capstone combines everything: multi-stage multi-arch distroless images, hardened runtime flags, Compose with healthchecks, profiles, secrets and internal networks, and a CI pipeline that tests, caches, scans, generates SBOMs, signs and promotes by digest.
- Production readiness is verifiable: write an acceptance checklist and test it with commands.
- Interviews reward depth: be ready to go from `docker run` down to containerd, runc, namespaces, cgroups and overlayfs.
- Keep the cheat sheet nearby; fluency with `inspect`, `logs`, `events`, `system df`, `nsenter` and `buildx` is what makes you fast in real incidents.